



NATIONAL BOARD FOR TECHNICAL EDUCATION

HIGHER NATIONAL DIPLOMA (HND)

SOFTWARE AND WEB DEVELOPMENT

CURRICULUM AND COURSE SPECIFICATIONS

SEPTEMBER, 2023

PLOT B, BIDA ROAD, P.M.B. 2239, KADUNA – NIGERIA

www.nbte.gov.ng

Foreword

The Higher National Diploma (HND) Software and Web Development curriculum is designed to be used by training institutions to produce manpower with skills for the development and deployment of Software and Web Solutions.

There is therefore the need to produce professional practitioners with good ethics and prospects for career progression, through the acquisition of desirable knowledge and skills. This necessitated the production of this National Curriculum.

It is my belief that this curriculum and course specifications, which is the minimum required to produce technologists with sound knowledge and skills in Software and Web Development, if properly implemented with the required resources along with qualified candidates being admitted into the programme, will lead to the production of competent and skilled technologists.

I sincerely appreciate all those who made the development of this curriculum possible. The invaluable contributions of all the members of the committee and resource persons during the pre-critique and national critique workshops are appreciated.

I hope that the curriculum would be properly implemented so as to produce the required Work Force of our dream.

Prof. Idris M. Bugaje
Executive Secretary,
NBTE, Kaduna

GENERAL INFORMATION

1.0 TITLE OF THE PROGRAMME:

Higher National Diploma SOFTWARE AND WEB DEVELOPMENT

2.0 GOAL AND OBJECTIVES:

2.1 GOAL: To enable students acquire knowledge and skills for the development and deployment of software and web solutions.

2.2 OBJECTIVES OF THE PROGRAMME

A diplomate of HND SOFTWARE AND WEB DEVELOPMENT should be able to:

- I. Apply knowledge of Operating System and HCI effectively in Software and Web Development
- II. Apply appropriate database models in specific tasks
- III. Design and develop embedded systems
- IV. Apply the concepts of Software Engineering in solutions development
- V. Design, develop and deploy secured software and web solutions.
- VI. Apply algorithmic solutions to real-world problems
- VII. Obtain industry certification(s)
- VIII. Apply ethical and professional practices in Software and Web Development.
- IX. Carry out Software Testing and Quality Assurance
- X. Conduct data analysis using modern tools for decision making

3.0 ENTRY REQUIREMENTS

The general entry requirements for the HND SOFTWARE AND WEB DEVELOPMENT Programme are:

(a) National Diploma (ND) computer Science entry requirements (See ND computer Science curriculum) or National Innovation Diploma (NID) Computer Software Engineering entry requirements (See NID Computer Software Engineering curriculum)

(b) Lower credit or above in ND computer Science or National Innovation Diploma (NID) Computer Software Engineering or Nigerian Skills Qualification

(NSQ) level 3-5

(c) In exceptional cases, ND diplomates with a pass (CGPA of 2.0-2.49) in the ND examination with two or more years of cognate experience in the specific field may be considered for admission into the HND SWD programme

4.0 STRUCTURE OF PROGRAMME

This is a two-year Programme i.e. four semesters of classroom, laboratory, field and workshop activities in the institution. Each semester shall be of 17 weeks duration made up as follows: 15 Contact weeks of teaching, i.e. recitation, practical exercises, quiz, tests, etc and 2 weeks for registration and examination.

5.0 EVALUATION SCHEME

The HND SWD Examination questions must be externally moderated. In grading the students, theory shall constitute 40% while practical 60%.

6.0.ACCREDITATION

Each Programme offered at the HND level shall be accredited by the NBTE before the Diplomates can be awarded the Higher National Diploma certificates. Details about the process of accrediting a Programme for the award of the HND are available from the Executive Secretary, National Board for Technical Education, Plot B, Bida Road, P.M.B. 2239, Kaduna, Nigeria.

7.0 CONDITIONS FOR THE AWARD OF HND SOFTWARE AND WEB DEVELOPMENT

Institutions offering this Programme will award the HND certificate to candidates who successfully completed the Programme after passing prescribed course work, examinations, Project and NBTE approved certification. Such candidates should have completed a minimum of 90 credit units.

A candidate with Recognition of Prior Learning (RPL) equivalent and NSQ can proceed to graduate upon achieving the required credit units specified above.

7.1 GRADING OF COURSES

Courses shall be graded as follows:

MARKED RANGE	LETTER GRADE	WEIGHTING
75% and above	A	4.00
70% – 74%	AB	3.50
65% – 69%	B	3.25
60% – 64%	BC	3.00
55% – 59%	C	2.75
50% – 54%	CD	2.50
45% – 49%	D	2.25
40% – 44%	E	2.00

7.2 CLASSIFICATION OF DIPLOMAS

Higher National Diploma Certificates shall be awarded based on the following classifications:

Distinction	-	CGPA 3.50-4.00
Upper Credit	-	CGPA 3.00-3.49
Lower Credit	-	CGPA 2.50-2.99
Pass	-	CGPA 2.00-2.49

8.0 QUALIFICATION OF TEACHERS

8.1 Holders of B.Sc/HND qualifications in Computer Science or related disciplines.

8.2 In addition, teachers of this programme should have been trained and certified by recognized industrial bodies or possesses higher degree or its equivalent in related discipline.

9.0 GUIDANCE NOTES FOR TEACHERS OF THE PROGRAMME

The new curriculum is drawn in unit courses. This is in keeping with the provisions of the National Policy on Education which stresses the need to introduce the semester credit units which will enable a student who so wish to transfer the units already completed in an institution of similar standard from which he is transferring.

As the success of the credit unit system depends on the articulation of programmes between the institutions and industry, the curriculum content has been written in behavioral objectives, so that it is clear to all, the expected performance of the student who successfully completed some of the courses or the diplomates of the programme. There is a slight departure in the presentation of the performance-based curriculum which requires the conditions under which the performance is expected to be carried out and the criteria for the acceptable levels of performance. It is a deliberate attempt to further involve the staff of the department teaching the programme to write their own curriculum stating the conditions existing in their institution under which the performance can take place and to follow that with the criteria for determining an acceptable level of performance.

The Academic Board of the institution may vet departmental submission on the final curriculum. Our aim is to continue to see to it that a solid internal evaluation system exists in each institution for ensuring minimum standard and quality of education in the programmes offered throughout the Technical and Vocational Education (TVE) system. The teaching of the theory and practical work should, as much as possible, be integrated. Practical exercises, especially those in professional courses and laboratory work should not be taught in isolation from the theory. For each course, there should be a balance of theory to practical in the ratio of 40:70.

10.0 MANDATORY SKILLS QUALIFICATIONS

TABLE OF CONTENTS

Foreword	<i>Error! Bookmark not defined.</i>
GENERAL INFORMATION.....	<i>Error! Bookmark not defined.</i>
TABLE OF CONTENTS.....	7
CURRICULUM TABLE.....	<i>Error! Bookmark not defined.</i>
YEAR ONE SEMESTER ONE COURSES.....	<i>Error! Bookmark not defined.</i>
Operating System	<i>Error! Bookmark not defined.</i>
Database Design I.....	<i>Error! Bookmark not defined.</i>
C++ Programming	36
Operations Research.....	<i>Error! Bookmark not defined.</i>

Data Communication and Networks	Error! Bookmark not defined.
Introduction toSoftware Engineering	80
YEAR ONE SEMESTER TWO COURSES	88
Python Programming.....	88
Database Design II.....	105
Front End Development I.....	111
Back End Development I.....	121
Advanced Statistics for Computing	Error! Bookmark not defined.
Research Methodology in SWD.....	Error! Bookmark not defined.
YEAR TWO SEMESTER ONE COURSES	Error! Bookmark not defined.
Project Management in SWD.....	Error! Bookmark not defined.
Embedded System Development.....	Error! Bookmark not defined.

Front End Development II	Error! Bookmark not defined.
Back End Development II.....	Error! Bookmark not defined.
Software Design and Architechture	Error! Bookmark not defined.
Advanced Data Structure and Algorithms	202
YEAR TWO SEMESTER TWO COURSES.....	218
Human Computer Interface(HCI)	218
Ethical and Professional Practice in SWD	231
Software Testing and Quality Assuarance.....	242
List of Tools/Equipment for HND Software and Web Development.....	Error! Bookmark not defined.66
LIST OF PARTICIPANTS PRE-CRITIQUE WORKSHOP.....	Error! Bookmark not defined.9
LIST OF PARTICIPANTS FINAL CRITIQUE WORKSHOP.....	Error! Bookmark not defined.0

**HIGHER NATIONAL DIPLOMA IN SOFTWARE AND WEBDEVELOPMENT
YEAR I SEMESTER I**

S/N	Course Code	Course Title	L	P	CU	CH	Prerequisite
1.	SWD 311	Operating System	2	2	3	4	
2.	SWD 312	Database Design I	2	2	3	4	
3.	SWD 313	C++Programming	2	3	3	5	
4.	SWD 314	Operations Research	1	2	2	3	
5.	SWD 315	Data Communication and Networks.	2	2	3	4	
6.	SWD316	Introduction to Software Engineering	3	2	3	5	
7.	AIT 311	Computer Architecture	2	2	3	4	
8.	AIT 313	Artificial Intelligence	2	2	3	4	
9.	GNS 301	Use of English III	2	0	2	2	
Total			18	17	25	35	

HIGHER NATIONAL DIPLOMA IN SOFTWARE AND WEB DEVELOPMENT
YEAR I SEMESTER II

S/N	Course Code	Course Title	L	P	CU	CH	Prerequisite
1.	SWD 321	Python Programming	2	3	3	5	
2.	SWD 322	Database Design II	2	3	3	5	SWD 312
3.	SWD 323	Front End Development I	2	4	3	6	
4.	SWD 324	Back End Development I	2	4	3	6	
5.	SWD 325	Advanced Statistics for Computing	2	1	2	3	
6.	SWD 326	Research Methods in SWD	2	2	3	4	
7.	SWD 327	Mandatory Skills Qualification I	0	4	4	4	
8.	GNS 302	Communication in English III	2	0	2	2	
9.	ENT 326	Practice of Entrepreneurship I	2	2	2	4	
Total			16	23	25	39	

HIGHER NATIONAL DIPLOMA IN SOFTWARE AND WEB DEVELOPMENT
YEAR II SEMESTER I

S/N	Course Code	Course Title	L	P	CU	CH	Prerequisite
1.	SWD 411	Project Management in SWD	2	1	2	3	
2.	SWD 412	Embedded System Development	2	3	3	5	
3.	SWD 413	Front End Development II	2	4	3	6	SWD 323
4.	SWD 414	Back End Development II	2	4	3	6	SWD 324
5.	SWD 415	Software Design and Architecture	2	4	3	6	
6.	SWD 416	Advanced Data Structures and Algorithms	2	4	3	6	
7.	SWD 417	Mandatory Skills Qualifications II	0	4	4	4	
8.	ENT 416	Entrepreneurship Development	2	2	4	4	
Total			14	26	25	40	

**HIGHER NATIONAL DIPLOMA IN SOFTWARE AND WEB DEVELOPMENT
YEAR II SEMESTER II**

S/N	Course Code	Course Title	L	P	CU	CH	Prerequisite
1.	SWD 421	Human Computer Interface (HCI)	2	1	2	3	
2.	SWD 422	Ethical and Professional Practice in SWD	2	1	2	3	
3.	SWD 423	Software Testing and Quality Assurance	2	2	3	4	
4.	SWD 424	Seminar in SWD	2	2	1	4	
5.	SWD 425	Security in SWD	2	3	3	5	
6.	SWD 426	Project	0	6	6	6	
Total			10	15	17	25	

YEAR I SEMESTER I

PROGRAMME: HIGHER NATIONAL DIPLOMA SOFTWARE AND WEB DEVELOPMENT		
COURSE: Operating System	COURSE CODE: SWD311	CONTACT HOURS: 4hrs
	CREDIT UNIT: 3	THEORETICAL: 2
YEAR: I SEMESTER: I	PRE-REQUISITE:	PRACTICAL: 2
GOAL: This course is designed to acquaint students with knowledge and skills in Operating System		
GENERAL OBJECTIVES: On completion of this course the student should be able to: 1. Comprehend the Fundamentals of Operating Systems. 2. Understand Linux and its basic usage 3. Know User and File Management 4. Understand Process Management 5. Comprehend Memory Management and Virtualization 6. Understand Networking and Communication		

PROGRAMME: HIGHERNATIONAL DIPLOMA SOFTWARE AND WEB DEVELOPMENT						
COURSE: Operating System			COURSE CODE:SWD311		CONTACT HOURS: 4hrs	
			CREDIT UNIT:3		THEORETICAL: 2	
YEAR: I SEMESTER: I			PRE-REQUISITE:		PRACTICAL:2	
COURSE SPECIFIFCATION: THEORETICAL AND PRACTICAL						
GOAL: This course is designed to acquaint students with knowledge and skills in Operating System						
GENERAL OBJECTIVE 1.0: Comprehend the Fundamentals of Operating Systems						
THEORETICAL CONTENT				PRACTICAL CONTENT		
Week	Specific Learning Outcome	Teacher’s Activities	Resources	Specific Learning Outcome	Teacher’s Activities	Evaluation
1-2	1.1 DefineOperatingSystem(OS) 1.2 Explain the types of Operating System. 1.3 Explain the Evolution of OS 1.4 List the Importance and features ofOS, etc. 1.5 Describe OS, it’s modeofoperation and the architecture	Describe OS and its Evolution, the types of OS such as: Microsoft Windows, AppleMacOS , Linux: Linux Distributions Red	Virtualization software Operating System ISO files Presentationpack age MultimediaProj ector	Install virtualization softwarewithdifferent Operating Systems Manipulate User Account Management. Create, Delete, Copy and Move Files and Directories using different OSs.	Guide Students on how to: Install virtualization softwarewithd ifferent Operating Systems Create,	

	1.6 Explain System calls and APIs in terms of it's characteristics and examples	hat and Embedded Systems - Explain the Importance and features of OS. - Describe OS into batch, real time, timesharing and networking environment. - Describe mode of operation and the Architecture of OS (Monolithic, Micro-kernel, Layered Kernel) - Discuss System calls and APIs in	Text Books Lecture Notes	Use Task Manager / Activity Monitor to view and monitor system resources, running processes etc. Practice basic Command-Line Interface (CLI): Create, rename, move, and delete files and folders on your Windows operating system. Explore different file attributes and permissions.	modify, and delete user accounts. Understand user privileges and experiment with different user roles. Create, Delete, Copy and Move Files and Directories using Windows OS/Linux Use the built-in task manager (Windows) or Activity Monitor (macOS/Linux) to monitor system resources, view running processes, and terminate	
--	---	--	-----------------------------	--	---	--

		terms of its characteristics and examples			unresponsive applications. Practice basic command-line operations. Navigate directories, list files, copy, move, and delete files using commands like ls, cd, cp, mv, and rm. Create, rename, move, and delete files and folders on your Windows operating system. Explore different file attributes and permissions.	
--	--	---	--	--	--	--

GENERAL OBJECTIVE: 2.0 Understand Linux and its basic usage						
3-4	2.1 Define Linux Operating System 2.2 State the history and evolution of Linux 2.3 Explain why Linux is Everywhere 2.4 Describe the Open Source Philosophy and the GNU General Public License (GPL) 2.5 Describe Linux Distributions and Package Management System 2.6 Explain Installation Process (dual-boot or Virtual Machine set up) 2.7 Describe the basic Linux commands and command-line interface (CLI) 2.8 Describe File System hierarchy and Navigation 2.9 Explain the usage of "Getting Help"	- Explain Linux Operating System - Describe the Open Source Philosophy such as: Free Redistribution, Access to Source Code, Technology Neutrality, Transparency etc - Explain Linux Distribution and Package Management - Describe the Installation Process and	Computer System Internet PC Loaded with virtualization software Presentation package Multimedia Projector Linux Distribution (Distro) eg Ubuntu, Fedora etc., Installation Media, Text books Marker/Marker Board	Analyze real-world projects embracing the Open Source Philosophy Examine the use of GPL in notable software projects Compare popular Linux distributions and their use cases. Identify specific use cases for different distributions Utilize a package manager to install and update software Diagnose and address issues with package repositories and apply package management commands effectively Set up a basic Linux networking environment Experiment with help-related commands	Guide Students on how to: Examine the use of GPL in notable software projects Identify and Compare popular Linux distributions and their use cases. Utilize a package manager to install and update software Set up a basic Linux networking environment Experiment	

		the basic Linux Command-line Interface • Explain the File System hierarchy and Navigation, • Describe “Getting Help” eg: Introduction, Man Pages, Finding Commands and Documentati on, Info Documentati on etc		Explore commands to search for commands based on keywords etc.	with help-related commands like ‘info’, which provides comprehensive documentation for some commands and topics. Explore ‘apropos’ or ‘whatis’ commands to search for commands based on keywords. Practice using the ‘—help’ option with various commands to access built-in command-line documentation. For example, ‘ls —help’ or ‘mkdir —help’.	
--	--	--	--	---	--	--

GENERAL OBJECTIVE: 3.0 User and File Management						
5-6	1 Describe user accounts and Groups 3.2 Explain user authentication, Password Policies, file permissions and ownership. 3.3 Explain the File manipulation and basic text editing	- Describe user account such as: Username, Password, Home Directory, Access Privileges, Personal Settings and Resource Usage. User Groups such as: Primary Group, Secondary Groups, Access Control etc. - Explain User authentication, Password Policies, File permissions	Computer Hardware, Linux Distribution (Distro) e.g. Ubuntu, Fedora etc., Installation Media Marker Board Text books	Configure user authentication and permissions Create, Delete, Copy and Move Files and Directories Navigate, Move, Search and Replace using text editors. Create and Edit Scripts Set up file sharing between two computers. Share a folder from one system and access it from another.	Guide Students on how to: Configure user authentication and permissions Create, rename, move, and delete files and folders on your operating system. Explore different file attributes and permissions. Use Text Editors Navigation	

		and ownership Describe File Manipulation and basic text editing such as: Copying and Moving Files, Changing Permissions, Formatting and Indentation, Inserting and Deleting Text etc.			and Movement Search and Replace. Creating and Editing Scripts Set up file sharing between two computers using protocols like SMB (Server Message Block) or NFS (Network File System).	
GENERAL OBJECTIVE: 4.0 Process Management						
7-9	1 Explain processes and threads 4.2 Explain Process states and management commands. 4.3 Explain Process Scheduling and Priorities, Process Creation and Termination, Concurrency and	- Describe Processes and threads, - Describe process states and Management	Multimedia Projector Textbooks, Online Articles and Tutorials.	Monitor and control processes using Linux and Windows tools Implement and analyse different process scheduling algorithms	Guide Student how to: Monitor and control processes	

	Parallelism and Inter-process Communication(IPC) 4.4 Explain Deadlock Handling, Load Balancing and Resource Management 4.5 Explain Security and Isolation	command, • Explain process Scheduling considering factors like: Scheduling with I/O requirement and Scheduling without I/O requirement, • Discuss process priority, communication delays, and fairness across networked machines. • Describe Concurrency, Parallelism, Process Creation and Termination, Inter Process Communication	Video Lectures and Courses. Computer System Marker Marker board	and manage process priorities Create, terminate, and monitor processes.	using Linux / Windows tools Implement and analyse different process scheduling algorithms and manage process priorities Create, terminate, and monitor processes; Create a parent and child process that communicate by sending messages or data between them.	
--	--	---	---	---	--	--

		on etc. Describe the concept of Deadlock E.g. Resource Allocation Graph (RAG), Deadlock Detection, Timeouts and Aborts, Resource Pre-emption and Distributed Deadlock Detection.				
GENERAL OBJECTIVE: 5.0 Memory Management and Virtualization						
10-12	5.1 Describe the overview of memory Management 5.2 Explain Virtual memory and Paging 5.3 Explain Swap space Configuration 5.4 Describe memory Virtualization	Describe Memory Management with respect to: Memory Hierarchy, Memory Allocation, Memory Protection,	Computer system, Internet connectivity LAN Multimedia Projector	Capture and analyze network traffic Set up network services (DNS, DHCP) Perform subnetting, configure IP addresses and routing.	Guide Students on how to: Capture and analyze network traffic Set up network services (DNS,	

		Virtual Memory, Swapping, Paging, Demand Paging, Page Replacement Algorithms and Fragmentation. Describe the concept of virtualization	Textbooks, Online Articles Cheat Sheets and Quick References Practical Labs and Simulators. ping, traceroute, Wireshark, ethereal and packet tracer	Use network utilities and commands to diagnose, troubleshoot, and manage network connectivity issues Configure firewalls, analyze network traffic patterns, and enhance network security using Linux tools. Use the ping command to check the reachability of a remote host and measure round-trip latency Check Network Ports and Analyse Network Traffic using Wireshark Set Up a Basic Web Server with HTTP Test HTTPS Connections	DHCP) and Configure IP addresses, subnetting, and routing Use network utilities and commands to diagnose, troubleshoot, and manage network connectivity issues Configure firewalls, analyze network traffic patterns, and enhance network security using Linux tools Use the ping command to check the reachability of a remote host	
--	--	---	---	---	--	--

					and measure round-trip latency Check Network Ports and Analyse Network Traffic using Wireshark Set Up a Basic Web Server with HTTP Test HTTPS Connections	
GENERAL OBJECTIVE: 6.0 Understanding Networking and Communication						
13-15	6.1 Explain Networking Protocols and Stacks 6.2 Explain IP Addressing and Subnetting 6.3 Explain Sockets, Socket	Describe OSI Model. Describe IP Addressing, (IPv4& IPv6)	Computer system, Internet connectivity LAN	Capture and analyze network traffic Set up network services (DNS, DHCP)	Guide Students on how to: Capture and analyze network traffic	

	APIs, Network Utilities and Commands 6.4 Describe Firewalls and Security 6.5 Explain Network Monitoring and Analysis 6.6 Describe networking troubleshooting tools and techniques 6.7 Describe APIs for Communication 6.8 Describe Security Protocols, Web Communication, and Real-Time Communication	Describe Network Utilities and Commands Explain the Key Functions of Firewalls, Types of Firewalls, Security Mechanisms, Firewall Rules and Importance of Firewalls in Security Describe Network Monitoring and Analysis Discuss Networking troubleshooting tools and techniques Describe APIs for Communication such as Winsock API in Windows, C and POSIX API in Linux or the Berkeley Sockets API in Unix-like Systems	Multimedia Projector Textbooks, Online Articles Cheat Sheets and Quick References Practical Labs and Simulators. ping, traceroute, Wireshark, etheral and packet tracer	Perform subnetting, configure IP addresses and routing. Use network utilities and commands to diagnose, troubleshoot, and manage network connectivity issues Configure firewalls, analyze network traffic patterns, and enhance network security using Linux tools. Use the ping command to check the reachability of a remote host and measure round-trip latency Check Network Ports and Analyse Network Traffic using Wireshark Set Up a Basic Web Server with HTTP	Set up network services (DNS, DHCP) and Configure IP addresses, subnetting, and routing Use network utilities and commands to diagnose, troubleshoot, and manage network connectivity issues Configure firewalls, analyze network traffic patterns, and enhance network security using Linux tools Use the ping command to	
--	---	---	---	--	--	--

		Describe Security protocols such as SSL/TLS, Web Communication such as HTTP and HTTPS, Web APIs and Real-Time Communication		Test HTTPS Connections	check the reachability of a remote host and measure round-trip latency Check Network Ports and Analyse Network Traffic using Wireshark Set Up a Basic Web Server with HTTP Test HTTPS Connections	
--	--	---	--	------------------------	---	--

PROGRAMME: HIGHER NATIONAL DIPLOMA SOFTWARE AND WEB DEVELOPMENT		
COURSE: Database Design I	COURSE CODE: SWD312	CONTACT HOURS: 4hrs
	CREDIT UNIT: 3	THEORETICAL: 2
YEAR: I SEMESTER: I	PRE-REQUISITE:	PRACTICAL: 2
GOAL: This course is designed to introduce student to computer database design		
GENERAL OBJECTIVES: : Upon completion of this course, the student should be able to: 1. Understand the organizational information need and database concepts 2. Understand the various types of database models 3. Know how to model data 4. Understand the design of relational database 5. Understand the Structured Query Language (SQL) 6. Comprehend database systems architecture		

PROGRAMME: HIGHERNATIONAL DIPLOMA SOFTWARE AND WEB DEVELOPMENT						
COURSE: Database Design I			COURSE CODE:SWD312		CONTACT HOURS: 4hrs	
			CREDIT UNIT:3		THEORETICAL: 2	
YEAR: I SEMESTER: I			PRE-REQUISITE:		PRACTICAL:2	
COURSE SPECIFIFCATION: THEORETICAL AND PRACTICAL						
GOAL: This course is designed to introduce student to computer database design						
GENERAL OBJECTIVE 1.0: Understand the organizational information need and database concepts						
THEORETICAL CONTENT				PRACTICAL CONTENT		
Week	Specific Learning Outcome	Teacher’s Activities	Resources	Specific Learning Outcome	Teacher’s Activities	Evaluation
1-2	1.1 Define Database & Database management system 1.2 Explain types of information need 1.3 List purpose of database management systems 1.4 Explain data view and data models. 1.5 Explain database administrators, users and languages	Explain databases and database management systems are Explain the types of information which organizations use Explain the different purpose for database management systems	White board, PCs/Laptops, Network/ Internet, Multimedia, Database software such as WAMP, XAMP and flip chart White board, PCs/Laptops, Network/	Install database software. Design and implement type of database models.	Guide students to install database software. Guide students to design and implement type of database	Explain purpose of database management system Explain data view and data model

		Explain data view and models. Explain the different types of data models. Discuss different types of database languages. Explain database administrator and users.	Internet, Multimedia, Database software and flip chart		models.	
GENERAL OBJECTIVE: 2.0 Understand the various types of database models						
3-4	2.1 Explain relational databasemodels 2.2 Explain hierarchical database model 2.3 Explain Network database model	Explain the basic concepts of relational database model. Explain the basic concepts of hierarchical database model Explain the basic concepts of network database model	White board, PCs/Laptops, Network/ Internet, Multimedia, Database software and flip chart	Design and implement various types of databases models	Guide students to: Design Relational, hierarchical and network database models.	Explain different types of database models
GENERAL OBJECTIVE: 3.0 : Know how to model data						

	3.1 Explain the concept of E-R, entity sets, entity relationship, weak entity sets. 3.2 Explain E-R database schema 3.3 Explain reduction of E-R schema into tables	Describe the basic concepts of E-R Explain entity set and entity relationship diagram Explain weak entity sets Discuss the design of E-R database schema Explain reduction of E-R schema into tables	White board, PCs/Laptops, Network/Internet, Multimedia, Database software and flip chart White board, PCs/Laptops, Network/Internet, Multimedia, Database software and flip chart	Design and implement E-R database schema and reduce the E-R schema into table.	Guide students to design and implement E-R database schema and reduction of E-R schema into table	Explain entity set and entity relationship diagram Explain E-R database schema Give examples of E-R database schema
General Objective 4.0: Understand the design of relational database						
7	4.1 Explain pitfalls in relational-database design 4.2 Explain Decomposition and Normalization 4.3 Explain domain-key normal form 4.4 Review alternative approaches to database design	Explain the pitfalls in relational database design Explain decomposition and normalization	White board, PCs/Laptops, Network/Internet, Multimedia, Database software and flip chart	Design and implement relational database and normalize it Design and implement relational database and normalize it	Guide students to accomplish the design of relational database and normalize it	Explain pitfalls in relational database design Describe database

		Explain domain-key normal form. Discuss the alternative approaches to database design	White board, PCs/Laptops, Network/Internet, Multimedia, Database software and flip chart		Guide students to accomplish the design of relational database and normalize it Guide students to design a database using alternative approaches	decomposition and normalization Explain domain-key normal form Explain alternative approaches to database design
GENERAL OBJECTIVE: 5.0 Understand structured query language (SQL)						
9 - 12	5.1 Explain the background of SQL 5.2 Explain the basic structure and syntax of SQL 5.3 Explain nested sub-queries 5.4 Explain derived relations 5.5 Explain views 5.6 Explain joined relations 5.7 Explain data definition language and embedded SQL	Discuss the background of SQL Discuss the syntax and basic structures of SQL Describe nested sub-queries Explain derived relations and views Explain views and how databases can be modified	White board, PCs/Laptops, Network/Internet, Multimedia, Database software and flip chart White board, PCs/Laptops, Network/Internet, Multimedia, Database software and flip chart	Use SQL commands Use SQL commands Create views using SQL commands Create and alter database schemas	Guide students in the use of basic SQL commands Guide students in the use of SQL commands Guide students to create views using SQL commands	Explain the concept of SQL Explain nested sub-queries Describe derived relations Explain views Describe joined relations Explain data definition language and

		Discuss joined relations Describe implementation and data definition language and embedded SQL	White board, PCs/Laptops, Network/Internet, Multimedia, Database software and flip chart White board, PCs/Laptops, Network/Internet, Multimedia, Database software and flip chart		Guide students to create and alter database schemas	embedded SQL
GENERAL OBJECTIVE: 6.0 Comprehend database systems architecture						
13 - 15	6.1 Define centralised systems 6.2 Explain client-server systems 6.3 Explain parallel systems 6.4 Explain distributed systems and network types	Explain centralized systems Explain client-server systems Explain parallel systems Explain distributed	White board, PCs/Laptops, Network/Internet, Multimedia, Database software and flip chart White board, PCs/Laptops,	Demonstrate database systems architecture Create a scenario based analysis of parallel systems Create a scenario based analysis of distributed network systems	Guide students to demonstrate database systems architecture such as centralized and client server systems Guide	Explain centralized systems Describe client-server systems with example Networked PC in a lab loaded with database packages

		and networked system types Differentiate between distributed systems and networked systems	Network/ Internet, Multimedia, Database software and flip chart		students to demonstrate a scenario based analysis of parallel systems Guide students to demonstrate a scenario based analysis of distributed network systems	Explain distributed systems and network types
--	--	---	---	--	---	---

Assessment: Give details of assignments to be used:

Coursework/ Assignments %; Course test %; Practical %; Projects %; Examination %

Type of Assessment	Purpose and Nature of Assessment (SWD 312)	Weighting (%)
Examination	Final Examination (written) to assess knowledge and understanding	40
Test	At least 2 progress tests for feedback.	10
Practical	At least 5 hands-on to be assessed by the teacher	50
Total		100

PROGRAMME: HIGHER NATIONAL DIPLOMA SOFTWARE AND WEB DEVELOPMENT		
COURSE: C++ Programming	COURSE CODE: SWD313	CONTACT HOURS: 5hrs
	CREDIT UNIT: 3	THEORETICAL: 2
YEAR: I SEMESTER: I	PRE-REQUISITE:	PRACTICAL: 3
GOAL: This course is designed to enable students to acquire knowledge and Skills in OO Programming Using C++ Language		
GENERAL OBJECTIVES: Upon completion of this course, the student should be able to: 1. Understand the basics of computer programming, variables, comments, basic I/O operations. 2. Understand flow control (if-else, switch; loops) and data aggregates 3. Understand extending expressive power: pointers, functions, and memory 4. Understand accessing data: arrays of pointers, conversions, strings, namespaces, and exceptions 5. Know object-oriented programming concepts (classes and objects) 6. Know the concept of inheritance, encapsulation, polymorphysm and Abstraction. 7. Know the concept of exceptions 8. Comprehend operators and enumerated types 9. Understand how to apply C++ programming to database development		

PROGRAMME: HIGHERNATIONAL DIPLOMA SOFTWARE AND WEB DEVELOPMENT							
COURSE: C++ Programming			COURSE CODE:SWD313		CONTACT HOURS: 5hrs		
			CREDIT UNIT:3		THEORETICAL: 2		
YEAR: I SEMESTER: I			PRE-REQUISITE:		PRACTICAL:3		
COURSE SPECIFIFCATION: THEORETICAL AND PRACTICAL							
GOAL: This course is designed to enable students to acquire knowledge and Skills in OO Programming Using C++ Language							
GENERAL OBJECTIVE 1.0: Understand the basics of computer programming, variables, comments, basic I/O operations.							
THEORETICAL CONTENT				PRACTICAL CONTENT			
Week	Specific Learning Outcome		Teacher’s Activities	Resources	Specific Learning Outcome	Teacher’s Activities	Evaluation
1 - 3	1.1	Explain the difference between machine and high-level languages	Discuss the difference between machine and high-level languages Explain machine code and compilation Explain variables,	White board, PCs/Laptops, Network/ Internet, Cisco LMS content, Multimedia, IDE / Cisco Sandbox / Code Editor, and flip chart White board, PCs/Laptops, Network/	Install and configure IDE / Code editor	Guide students to install and configure IDE / code editor	Explain the difference between machine and high-level languages Outline examples of variable, integer and character
	1.2	Explain the machine code and compilation			Analyse and compare different code editors		
	1.3	Define variables, integers and characters			Write and test simple C++ programs		
	1.4	Explain the concept of comments in C++				Guide students to analyse different code editors	
	1.5	Explain streams and basic I/O operations					

	1.6 Explain declaration and initialization structures	integers and characters with examples Explain how comments are inserted in C++ program Explain stream and I/O operations Explain declaration and initialization structures	Internet, Cisco LMS content, Multimedia, IDE / Cisco Sandbox / Code Editor, and flip chart	Demonstrate declaration and initialization of structures	Guide students to write and test simple C++ programs Guide students to demonstrate declaration and initialization of structures	State how comments are inserted in C++ program Explain streams and basic I/O operations Discuss declaration and initialization of structures
GENERAL OBJECTIVE: 2.0 Understand flow control (if, else, switch; loops) and data aggregates						
3-5	2.1 Explain flow control of program 2.2 Explain conditional statements /instructions: if, else, switch 2.3 Explain loop execution (for, while, do while, repeat) 2.4 Describe logic, bitwise and arithmetic operators 2.5 Explain vectors and multidimensional arrays	Explain flow control of program Explain conditional statements /instructions: if else, switch Explain loop execution (for, while, do while, repeat) Explain logic, bitwise and arithmetic operators Explain vectors and	White board, PCs/Laptops, Network/ Internet, Cisco LMS content, Multimedia, IDE / Cisco Sandbox / Code Editor, and flip chart White board, PCs/Laptops, Network/ Internet, Cisco LMS content, Multimedia, IDE / Cisco Sandbox	Create programs to demonstrate code control flow and conditional statements (if, else, switch) Demonstrate loop control execution Demonstrate logic, bitwise and arithmetic operators	Guide students to demonstrate program control flow Demonstrate conditional statements/ instructions (if, else, switch) Demonstrate loop execution (for, while, do while, repeat) Guide students	Explain how to control the flow of program Explain conditional instructions Explain logic, bitwise and arithmetic operators Describe vectors and multidimensional arrays

		multidimensional arrays	/ Code Editor, and flip chart		to write program to demonstrate logic, bitwise and arithmetic operators	
GENERAL OBJECTIVE: 3.0 Understand extending expressive power: pointers, functions, and memory						
5-6	3.1 Explain the concept of pointers 3.2 Explain the design, declaration, and invocation of functions 3.3 Explain different methods of passing parameters and their purpose 3.4 Explain default parameters and inline functions 3.5 Explain overloaded functions 3.6 Explain concept of sorting and memory on demand	Explain the concept of pointers Explain the design, declaration, and invocation of functions Discuss different methods of passing parameters and their purpose Explain default parameters and inline functions Explain overloaded functions	White board, PCs/Laptops, Network/ Internet, Cisco LMS content, Multimedia, IDE / Cisco Sandbox / Code Editor, and flip chart	Demonstrate the design, declaration and invocation of functions Demonstrate parameters passing and functions' default parameters Demonstrate functions overload Write program to sort given items	Guide students to: write programs that demonstrate the definition, declaration and invocation of functions Write programs to demonstrate parameters passing and functions' default parameters	Describe pointers and default parameters Describe the process of designing, declaring and invoking functions Explain overloaded functions Discuss sorting and memory on demand

		Discuss concept of sorting and memory on demand			Demonstrate functions overload write program to sort given items using function.	
GENERAL OBJECTIVE: 4.0 Understand accessing data: arrays of pointers, conversions, strings, namespaces, and exceptions						
7 - 8	4.1 Explain arrays of pointers and conversions 4.2 Explain strings: declarations, initializations and assignments 4.3 Explain strings as an example of objects (methods and properties) 4.4 Describe how to declare and use namespaces 4.5 Discuss dealing with exceptions	Explain arrays of pointers and conversions Discuss strings: declarations, initializations and assignments Explain strings as an example of objects: (methods and properties) Discuss how to declare and use namespaces	White board, PCs/Laptops, Network/ Internet, Cisco LMS content, Multimedia, IDE / Cisco Sandbox / Code Editor, and flip chart	Illustrate and test use of arrays pointers Demonstrate strings declarations, initializations and assignments Illustrate and test namespace declaration and use Demonstrate exceptions	Guide students to: illustrate and test use of arrays pointers Demonstrate strings declarations, initializations and assignments Illustrate and test namespace declaration and use demonstrate exceptions	Discuss arrays of pointers and conversion Discuss strings declaration, initialization, and assignment Describe strings as an example of object Explain how to deal with exceptions

		Discuss dealing with exceptions				
GENERAL OBJECTIVE: 5.0 Know object-oriented programming concepts (classes, objects)						
9 - 10	5.1 Explain the concepts of OOP 5.2 Explain stack: the procedural-oriented approach vs. OOP 5.3 Explain the anatomy of classes and static components 5.4 Explain objects vs. Pointers and objects inside objects	Discuss the concepts of OOP Explain stack: the procedural-oriented approach vs. OOP Explain the anatomy of classes and static components Explain objects vs. Pointers and objects inside objects	White board, PCs/Laptops, Network/ Internet, Cisco LMS content, Multimedia, IDE / Cisco Sandbox / Code Editor, and flip chart	Compare procedural and OO programming approaches Design programs to demonstrate: 1) classes and static components, 2) objects inside objects, 3) compare objects and pointers	Guide students to: write codes to compare procedural and OO programming approaches Design programs that demonstrate: 1) classes and static components, 2) objects inside objects, 3) compare objects and pointers	Explain the concept of OOP as compared to procedural-oriented programming Compare objects and pointers Explain the concept of objects inside objects
GENERAL OBJECTIVE: 6.0 Know the concept of inheritance, encapsulation, polymorphysm and Abstraction						
11- 12	6.1 Explain class hierarchies 6.2 Explain classes, inheritance, and type	Discuss class hierarchies	White board, PCs/Laptops, Network/	Design programs to illustrate class hierarchies, inheritance and type	Guide students to: Design programs to	Explain the concepts of class hierarchies,

	compatibility 6.3 Explain Abstraction, polymorphism, encapsulation, and virtual methods 6.4 Explain objects as parameters, and dynamic casting 6.5 Explain various supplements and <i>const</i> keyword	Explain classes, inheritance, and type compatibility Discuss Abstraction, polymorphism, encapsulation, and virtual methods Explain objects as parameters, and dynamic casting Discuss various supplements and <i>const</i> keyword	Internet, Cisco LMS content, Multimedia, IDE / Cisco Sandbox / Code Editor, and flip chart	compatibility Demonstrate 1) Abstraction, polymorphism Inheritance, Encapsulation and virtual methods. , 2) objects as parameters and dynamic casting, 3) use of various supplements and <i>const</i> keyword	illustrate class hierarchies, inheritance and type compatibility Create program codes that demonstrate 1) Abstraction, polymorphism Inheritance, Encapsulation and virtual methods. 2) objects as parameters and dynamic casting, 3) use of various supplements and <i>const</i> keyword	inheritance and type compatibility Discuss 1) virtual methods and polymorphism, 2) objects as parameters and dynamic casting, 3) use of various supplements and <i>const</i> keyword
--	---	--	---	---	---	---

GENERAL OBJECTIVE: 7.0 Know the concept of exceptions

13-14	7.1 Explain the concept of exceptions 7.2 Explain <i>throw</i> and <i>catch</i> statements and their use 7.3 Outline exception categories 7.4 Explain exceptions	Discuss the concept of exceptions Explain <i>throw</i> and <i>catch</i> statements and their usage Discuss exception categories Explain exceptions	White board, PCs/Laptops, Network/Internet, Cisco LMS content, Multimedia, IDE / Cisco Sandbox / Code Editor, and flip chart	Design programs to practice exceptions with <i>throw</i> and <i>catch</i> statements Create codes with errors to debug or resolve using exceptions techniques	Guide students to: Design programs to practice exceptions with <i>throw</i> and <i>catch</i> statements Create codes with errors to debug or resolve using exceptions technique	Group students to work as team to achieve simple normal tasks using C++
GENERAL OBJECTIVE: 8.0 : Comprehend operators and enumerated types						
15	8.1 Outline overloading operators 8.2 Explain enumerated types	Discuss overloading operators Explain enumerated types	White board, PCs/Laptops, Network/Internet, Cisco LMS content, Multimedia, IDE / Cisco Sandbox / Code Editor, and flip chart	Create programs to demonstrate overloading operators and enumerated types	Guide students to create programs to demonstrate overloading operators and enumerated types	Group students to work as team to achieve simple normal tasks using C++

GENERAL OBJECTIVE: 9.0 Understand how to apply C++ programming to database development						
	9.1 Define Database 9.2 Define relational database 9.3 Explain steps of database Design 9.4 Explain general object-oriented design guidelines 9.5 Explain object-oriented Database 9.6 Define Object database 9.7 Explain Object database Model 9.8 Explain how to design object database 9.9 Explain how to apply C++ in database development	Define database and relational database Explain steps of database design and general object-oriented design guidelines Define object database Explain object-oriented database and object database model Explain how to apply C++ in database development	White board, PCs/Laptops, Network/ Internet, Cisco LMS content, Multimedia, IDE / Cisco Sandbox / Code Editor, and flip chart	Design and connect to a database using OOP Perform CRUD Operations in database application	Guide students to: Illustrate how to design and connect to a database using OOP Illustrate how to apply C++ in database development Illustrate how to connect to database using C++ Illustrate how to perform CRUD Operations in database application.	Explain what data database is and the steps of database design Define general object-oriented design guidelines

Assessment: Give details of assignments to be used:

Coursework/ Assignments %; Course test %; Practical %; Projects %; Examination %

Type of Assessment	Purpose and Nature of Assessment (SWD 313)	Weighting (%)
Examination	Final Examination (written) to assess knowledge and understanding	40
Test	At least 2 progress tests for feedback.	10
Practical	At least 5 hands-on to be assessed by the teacher	50
Total		100

PROGRAMME: HIGHER NATIONAL DIPLOMA SOFTWARE AND WEB DEVELOPMENT		
COURSE: Operations Research	COURSE CODE: SWD314	CONTACT HOURS: 3hrs
	CREDIT UNIT: 2	THEORETICAL: 1
YEAR: I SEMESTER: I	PRE-REQUISITE:	PRACTICAL: 2
GOAL: This course is designed to produce diplomates with the knowledge and skills to optimize decision-making.		
GENERAL OBJECTIVES: On completion of the course, the student should be able to: 1. Understand the history and development of Operations Research (OR) 2. Understand modelling in Operations Research 3. Understand optimization problem 4. Know linear programming (LP) 5. Know linear programming (Simplex Method) 6. understand integer programming 7. Understand dynamic programming 8. Understand transportation model 9. Understand assignment model		

PROGRAMME: HIGHERNATIONAL DIPLOMA SOFTWARE AND WEB DEVELOPMENT							
COURSE: Operations Research			COURSE CODE: SWD314		CONTACT HOURS: 3		
			CREDIT UNIT: 2		THEORETICAL: 1		
YEAR: I SEMESTER: I			PRE-REQUISITE:		PRACTICAL: 2		
COURSE SPECIFIFCATION: THEORETICAL AND PRACTICAL							
GOAL: This course is designed to produce diplomates with the knowledge and skills to optimize decision-making.							
GENERAL OBJECTIVE 1.0: Understand the history and development of Operations Research (OR)							
THEORETICAL CONTENT				PRACTICAL CONTENT			
Week	Specific Learning Outcome		Teacher’s Activities	Resources	Specific Learning Outcome	Teacher’s Activities	Evaluation
1	1.1.	Explain the historical context and origins of OR.	Explain the historical context and origins of OR, including its emergence during World War II and its evolution over time. Discuss historical development of OR, emphasizing key milestones, figures,	Whiteboard, Marker, Projector, PowerPoint, Google Slides	visually depict the historical context and origins of OR.	Guide students to Create timelines or infographics that visually depict the evolution of OR including significant developments and contributions.	
	1.2.	Identify key figures and milestones in the development of OR.			Identify key figures and milestones in the development of OR.		
	1.3.	Describe the various fields and industries where OR has made			Identify the various fields		

	significant contributions.	and historical contexts. Discuss key pioneers and contributors to OR Discuss the scientific nature of Operations Research, its importance and uses		and industries where OR has made significant contributions.		
GENERAL OBJECTIVE: 2.0 Understand modeling in Operations Research						
2	2.1. Define mathematical modeling and its role in OR. 2.2. Explain mathematical models to represent real-world decision-making problems. 2.3. Explain the appropriateness of different modeling approaches for various types of problems.	Discuss the concept of mathematical modeling, its purpose, and its importance in OR. Discuss software tools commonly used for mathematical modeling in OR, such as optimization software and modeling platforms Discuss mathematical models, the variables,	Whiteboard, Marker, Projector, PowerPoint, Google Slides	Analyze optimization software and modelling platforms.	Guide students to: Analyze optimization software and modelling platforms.	

		constraints, objectives, and how these models can inform decision-making.				
GENERAL OBJECTIVE: 3.0 Understand optimization problems						
3	3.1 Explain optimization problems in different contexts. 3.2. Formulate optimization problems using objective functions and constraints. 3.3. Differentiate between linear and nonlinear optimization problems.	Discuss optimization problems in different contexts, including resource allocation, cost minimization, and profit maximization. Discuss how to formulate optimization problems using objective functions and constraints. Discuss how to differentiate between linear and nonlinear optimization problems.	Whiteboard, Marker, Projector, PowerPoint, Google Slides	Conduct exercises to validate and test the accuracy of mathematical models by comparing model predictions with real-world data	Guide student to: Conduct exercises to validate and test the accuracy of mathematical models by comparing model predictions with real-world data.	
GENERAL OBJECTIVE: 4.0 : Know linear programming (LP)						

4-5	4.1. Define linear programming and its applications.	Explain linear programming and its applications.	Whiteboard, Marker, Projector, PowerPoint, Google Slides, Excel Solver, GAMS, or PuLP in Python	Solve LP problems graphically.	Guide students to: Solve LP problems graphically.	
	4.2. Explain how to formulate linear programming problems with linear objective functions and constraints.	Discuss how to formulate linear programming problems with linear objective functions and constraints.		Demonstrate the process of identifying feasible regions, corner points, and optimizing the objective function.	Demonstrate the process of identifying feasible regions, corner points, and optimizing the objective function.	
	4.3. Explain how to solve linear programming problems graphically and algebraically.			Demonstrate using LP software tools (e.g., Excel Solver, GAMS, or PuLP in Python) to solve LP problems with real world data.	Demonstrate using LP software tools (e.g., Excel Solver, GAMS, or PuLP in Python) to solve LP problems with real world data.	
	4.4 Explain the fundamental concepts of linear programming, including objective functions, constraints, decision variables, and the graphical representation of LP problems.	Discuss how to solve linear programming problems graphically and algebraically.				
	4.5 Explain sensitivity analysis techniques by varying coefficients, constraints, or resource availability to understand how LP solutions change in response.	Discuss the fundamental concepts of linear programming, including objective functions, constraints, decision variables, and the graphical representation of LP problems.				
	4.6 Explain the concept of LP duality and its practical implications.					
	4.7 Explain the interpretation of dual variables and shadow prices.					
	4.8 Explain the assumptions and limitations of LP models, including linearity and the applicability of LP to specific problem	Discuss sensitivity analysis techniques by varying				

	types. 4.9 Explain how to interpret LP solution outputs, including optimal values, decision variable values, and sensitivity analysis results.	coefficients, constraints, or resource availability to understand how LP solutions change in response. Explain the concept of LP duality and its practical implications. Discuss the interpretation of dual variables and shadow prices. Discuss the assumptions and limitations of LP models, including linearity and the applicability of LP to specific problem types. Explain how to interpret LP solution outputs, including optimal values, decision variable values, and sensitivity analysis				
--	--	--	--	--	--	--

		results.				
GENERAL OBJECTIVE: 5.0 Know linear programming (Simplex Method)						
6-7	5.1. Explain the Simplex Method as an algorithm for solving linear programming problems. 5.2. Explain how to apply the Simplex Method to optimize linear objective functions subject to linear constraints. 5.3. Explain how to interpret the results of Simplex Method iterations. 5.4 Explain the geometric interpretation of the Simplex Method by visually representing linear programming problems in two or three dimensions. 5.5 Explain how the method moves from one vertex to another to optimize the objective function. 5.6 Explain a high-level overview of the Simplex Method algorithm, emphasizing the steps involved, such as selecting entering and leaving variables, performing pivot	Discuss the Simplex Method as an algorithm for solving linear programming problems. Discuss the fundamentals of linear programming, including objective functions, constraints, decision variables, and the basic idea behind the Simplex Method. Discuss how to apply the Simplex Method to optimize linear objective functions subject to linear constraints. Discuss how to interpret the results of Simplex Method	Whiteboard, Marker, Projector, PowerPoint, Google Slides, Excel Solver, online LP solvers	Demonstrate the Simplex Method using a simple example on a whiteboard, chalkboard, or using software tools. Demonstrate using graphical representations to illustrate how linear programming problems can be visualized in two or three dimensions. Demonstrate how the Simplex Method moves between feasible solutions. Solve Simplex tableaus step by step, highlighting the identification of pivot elements and pivot operations	Guide students to: Demonstrate the Simplex Method using a simple example on a whiteboard, chalkboard, or using software tools. Demonstrate using graphical representations to illustrate how linear programming problems can be visualized in two or three dimensions. Demonstrate	

	operations, and updating the tableau.	iterations. Discuss the geometric interpretation of the Simplex Method by visually representing linear programming problems in two or three dimensions. Explain how the method moves from one vertex to another to optimize the objective function. Discuss a high-level overview of the Simplex Method algorithm, emphasizing the steps involved, such as selecting entering and leaving variables, performing pivot operations, and updating the tableau.			how the Simplex Method moves between feasible solutions. Solve Simplex tableaus step by step, highlighting the identification of pivot elements and pivot operations.	
--	---------------------------------------	---	--	--	---	--

GENERAL OBJECTIVE: 6.0 Understand integer programming (IP)

8-9	6.1. Explain the concept of integer programming and when it is applicable. 6.2. Explain how to formulate integer programming problems with discrete decision variables. 6.3. Explain how to apply integer programming techniques to solve optimization problems with integer solutions. 6.4 Explain different types of IP problems, including pure IP, mixed-integer programming (MIP), and binary integer programming (BIP). Provide examples of each type. 6.5 Explain the computational complexity of IP problems and introduce the concept of NP-hard problems. 6.6 Explain how IP problems can become challenging as problem size and complexity increases. 6.7 Explain heuristic and metaheuristic	Discuss the fundamentals of integer programming, emphasizing the difference between continuous and integer decision variables. Discuss the concept of integer programming and when it is applicable. Discuss how to formulate integer programming problems with discrete decision variables. Discuss how to apply integer programming techniques to solve optimization problems with integer solutions.	Whiteboard, Marker, Projector, PowerPoint, Google Slides, Excel Solver	Use IP solver software to formulate, solve, and analyze IP models based on real-world datasets.	Guide students to use IP solver software to formulate, solve, and analyze IP models based on real-world datasets.	
-----	---	---	--	---	---	--

	methods (e.g., greedy algorithms, simulated annealing, or genetic algorithms) that can be used to find approximate solutions to IP problems. 6.8 Explain various applications of IP in different industries, such as finance, transportation, and telecommunications; and explore how IP models impact decision-making.	Explain different types of IP problems, including pure IP, mixed-integer programming (MIP), and binary integer programming (BIP). Provide examples of each type. Discuss the computational complexity of IP problems and introduce the concept of NP-hard problems. Explain how IP problems can become challenging as problem size and complexity increases. Discuss heuristic and metaheuristic methods (e.g., greedy algorithms, simulated annealing,				
--	---	---	--	--	--	--

		or genetic algorithms) that can be used to find approximate solutions to IP problems. Discuss various applications of IP in different industries, such as finance, transportation, and telecommunications; and explore how IP models impact decision-making.				
GENERAL OBJECTIVE: 7.0 Understand Dynamic Programming (DP)						
10-11	8.1. Define dynamic programming and its applications in solving optimization problems with sequential decision-making. 8.2. Explain how to develop dynamic programming models for problems involving stages and state transitions.	Explain dynamic programming and its applications in solving optimization problems with sequential decision-making. Discuss key dynamic programming terminology, such as	Whiteboard, Marker, Projector, PowerPoint, Google Slides	Use DP concepts to solve optimization problems based on real-world datasets.	Guide students to use DP concepts to solve optimization problems based on real-world datasets.	

	8.3. Explain how to solve dynamic programming problems using the Bellman equation or recursive approaches.	memoization, optimal substructure, and overlapping subproblems, to provide a solid foundation for understanding DP concepts. Discuss how to develop dynamic programming models for problems involving stages and state transitions. Discuss how to solve dynamic programming problems using the Bellman equation or recursive approaches.				
GENERAL OBJECTIVE: 8.0 Understand Transportation Model						
12-13	8.1. Explain how to formulate transportation problems to optimize the distribution of goods from multiple sources to multiple destinations.	Discuss the fundamentals of transportation modeling,	Whiteboard, Marker, Projector, PowerPoint, Google Slides,	Formulate transportation problems to optimize the distribution of goods from multiple sources to	Guide students on how to: Formulate transportation	

	8.2. Explain how to solve transportation problems using transportation algorithms. 8.3 Explain the relationship between transportation models and network flow problems, highlighting similarities and differences.	emphasizing its role in solving distribution and logistics problems. Discuss how to formulate transportation problems to optimize the distribution of goods from multiple sources to multiple destinations. Discuss how to solve transportation problems using transportation algorithms, such as the North-West Corner Rule and the Vogel's Approximation Method (VAM). Discuss the relationship between transportation models and network flow problems,	Excel Solver, LP solvers, or specialized transportation modeling software	multiple destinations. Solve transportation problems using transportation algorithms.	problems as linear programming (LP) models, focusing on objective functions, decision variables, and constraints. Solve transportation problems using LP techniques. Demonstrate how to use transportation algorithms (e.g., the North-West Corner Rule, Vogel's Approximation Method). Perform sensitivity analysis for transportation models, including the impact of changes in	
--	---	--	--	---	--	--

		highlighting similarities and differences.			supply, demand, or transportation costs on the optimal solution	
GENERAL OBJECTIVE: 9.0 Understand Assignment Model						
14 -15	9.1. Define assignment problems and their significance. 9.2. Explain how to formulate assignment problems as linear programming models. 9.3. Explain how to apply Hungarian algorithm or other assignment methods to solve assignment problems optimally.	Discuss assignment problems and their significance in matching tasks or assignments to resources. Discuss how to formulate assignment problems as linear programming models. Discuss how to apply Hungarian algorithm or other assignment methods to solve assignment problems optimally. Discuss the components of an assignment problem, including tasks, resources,	Whiteboard, Marker, Projector, PowerPoint, Google Slides	Demonstrate how to formulate assignment problems as linear programming models. Apply Hungarian algorithm or other assignment methods to solve assignment problems optimally.	Guide students on how to: Demonstrate how to formulate assignment problems as linear programming (LP) Demonstrate how to apply Hungarian algorithm or other assignment methods to solve assignment problems optimally.	

		assignment costs, and the objective of minimizing or maximizing the total cost or profit.				
--	--	---	--	--	--	--

PROGRAMME: HIGHERNATIONAL DIPLOMA SOFTWARE AND WEB DEVELOPMENT						
COURSE: Data Communication and Networks		COURSE CODE:SWD315		CONTACT HOURS: 4hrs		
		CREDIT UNIT:3		THEORETICAL: 2		
YEAR: I SEMESTER: I		PRE-REQUISITE:		PRACTICAL:2		
COURSE SPECIFIFCATION: THEORETICAL AND PRACTICAL						
GOAL: This course provides students with knowledge and skills on networking relevant to software and web development						
GENERAL OBJECTIVE 1.0: Know the concepts of today’s networks						
THEORETICAL CONTENT				PRACTICAL CONTENT		
Week	Specific Learning Outcome	Teacher’s Activities	Resources	Specific Learning Outcome	Teacher’s Activities	Evaluation
1-2	1.1 Explain how networks affect our daily lives. 1.2 Outline how host and network devices are used 1.3 Outline network representations and how they are used in network topologies. 1.4 List the characteristics of common types of networks. 1.5 Outline how LANs and WANs interconnect to the internet.	Explain how networks affect our daily lives. Explain how host and network devices are used Explain network representations and how they are used in	Multi-media Projector, Lecture notes, Tutorials, Pictorials Packet tracer or any other suitable simulator	Identify network devices Design LAN/WAN using the simulator Design types of network topologies using the simulator	Guide students to: Demonstrate Network Representation using simulation software. Demonstrate	Outline network representations and how they are used in network topologies List the characteristics of a good network Describe types of

	1.6 Explain the four basic requirements of a reliable network. 1.7 Outline how trends such as BYOD, online collaboration, video, and cloud computing are changing the way we interact 1.8 Explain some basic security threats and solutions for all networks 1.9 Outline opportunities in the networking field	network topologies. Compare the characteristics of common types of networks. Explain how LANs and WANs interconnect to the internet. Describe the four basic requirements of a reliable network. Explain how trends such as BYOD, online collaboration, video, and cloud computing are changing the way we interact Identify some basic security threats and solution for all networks Outline opportunities in the networking			how to design LAN/WAN using the simulator Demonstrate how to design network topologies using the simulator	network topologies
--	--	---	--	--	--	--------------------

		field				
GENERAL OBJECTIVE: 2.0 : Understand protocols and models						
3-4	2.1 Outline the types of rules that are necessary to successfully communicate. 2.2 Narrate why protocols are necessary for network communication 2.3 Outline the purpose of adhering to a protocol suite. 2.4 Outline the role of standards organizations in establishing protocols for network interoperability. 2.5 Outline how the TCP/IP model and the OSI model are used to facilitate standardization in the communication process. 2.6 Outline how data encapsulation allows data to be transported across the network. 2.7 Outline how local hosts access local resources on a network. 2.8 Contrast the roles of the MAC address and the IP address. 2.9 Explain the purpose of ARP. 2.10 Explain the operation of IPv6 neighbour discovery. 2.11 Outline how ICMP is used to test network connectivity. 2.12 Explain the techniques of ensuring successful communication between devices	Describe the TCP / IP and OSI Models. Explain why protocols are necessary in network communication Explain the purpose of adhering to a protocol suite. Explain the role of standards organizations in establishing protocols for network interoperability. Explain how the TCP/IP model and the OSI model are used to facilitate standardization in the communication process.	Multi-media Projector, Lecture notes, Tutorials, Pictorials Wireshark packet tracer or other network simulators	Investigate the TCP/IP and OSI Models using packet tracer Install Wireshark to view network traffic. Apply ping and traceroute utilities to test network connectivity	Guide students to; - Investigate the TCP/IP and OSI Models in using packet tracer - Install Wireshark to view Network Traffic Use ping and traceroute	Explain why protocols are necessary in network communication Explain the purpose of adhering to a protocol suite.

		Explain how data encapsulation allows data to be transported across the network. Explain how local hosts access local resources on a network. Compare the roles of the MAC address and the IP address. Describe the purpose of ARP. Describe the operation of IPv6 neighbour discovery. Explain how ICMP is used to test network connectivity. Explain network utilities like ping, traceroute, etc. utilities for testing network connectivity			utilities to test network connectivity	
--	--	--	--	--	---	--

GENERAL OBJECTIVE: 3.0 Comprehend Open System Interconnect (OSI) model						
5-7	3.1 Elaborate the purpose and functions of the physical layer in the network. 3.2 Elaborate the characteristics of the physical layer. 3.3 Explain the basic characteristics of copper cabling. 3.4 Describe how UTP cable is used in Ethernet networks. 3.5 Explain fiber optic cabling and its main advantages over other media. 3.6 Explain Network devices using wired and wireless media. 3.7 List the purpose and function of the data link layer in preparing communication for transmission on specific media. 3.8 List the characteristics of media access control methods on WAN and LAN topologies. 3.9 Explain the characteristics and functions of the data link frame. 3.10 Explain how the network layer uses IP protocols for reliable communications	- Elaborate the purpose and functions of the physical layer in the network. - Elaborate the characteristics of the physical layer. - Explain the basic characteristics of copper cable. - Describe how UTP cable is used in Ethernet networks. - Explain fiber optic cabling	Multi-media Projector, Lecture notes, Tutorials, Pictorials Wireshark packet tracer or other network simulators networking tool kit networking cable	- Make crossover, rollover and straight-through cables - Connect wired and wireless LAN. - View Wired and Wireless NIC Information using Packet Tracer. - Observe TCP and UDP Communications - Observe DNS Resolution	Guide students to: - Make crossover, rollover and straight-through cables - Connect a Wired and Wireless LAN - View Wired and Wireless NIC Information	List the characteristics of media access control methods on WAN and LAN topologies

	3.11 Explain the role of the major header fields in the IPv4 packet. 3.12 Explain the role of the major header fields in the IPv6 packet. 3.13 Explain how network devices use routing tables to direct packets to a destination network. 3.14 Explain the functions of fields in the routing table 3.15 Explain the purpose of the transport layer in managing the transportation of data in end-to-end communication. 3.16 List the characteristics of TCP. 3.17 List characteristics of UDP. 3.18 Describe how TCP and UDP use port numbers. 3.19 Explain how TCP session establishment and termination processes facilitate reliable communication. 3.20 Explain how TCP protocol data units are transmitted and acknowledged to guarantee delivery. 3.21 Explain the operations of transport layer protocols in supporting end-to-end communication. 3.22 Explain how the functions of the application layer, presentation layer, and session layer work together to provide network services to end-user applications.	and its main advantages over other media. ● Explain Network devices using wired and wireless media. ● Explain the purpose and function of the data link layer in preparing communication for transmission on specific media. ● Explain the characteristics of media access control methods on WAN and LAN			tion using Packet Tracer – TCP and UDP Communications ● Observe DNS Resolution ●	
--	--	--	--	--	--	--

	3.23 Explain how end-user applications operate in a peer-to-peer network. 3.24 Explain how web and email protocols operate. 3.25 Explain how DNS and DHCP operate. 3.26 Explain how file transfer protocols operate.	topologies. ● Explain the characteristics and functions of the data link frame. ● Explain how the network layer uses IP protocols for reliable communications ● Explain the role of the major header fields in the IPv4 packet. ● Explain the role of the major header fields in the IPv6 packet. ● Explain how network devices use				
--	--	--	--	--	--	--

		routing tables to direct packets to a destination network. ● Explain the function of fields in the routing table of a router. ● Explain the purpose of the transport layer in managing the transportation of data in end-to-end communication. ● Explain the characteristics of TCP. ● Explain the characteristics of UDP.				
--	--	--	--	--	--	--

		• Describe how TCP and UDP use port numbers. • Explain how TCP session establishment and termination processes facilitate reliable communication. • Explain how TCP protocol data units are transmitted and acknowledged to guarantee delivery. • Explain the operations of transport layer protocols in				
--	--	---	--	--	--	--

		supporting end-to-end communication. ● Explain how the functions of the application layer, presentation layer, and session layer work together to provide network services to end-user applications. ● Explain how end-user applications operate in a peer-to-peer network. ● Explain how web and email protocols				
--	--	--	--	--	--	--

		operate. ● Explain how DNS and DHCP operate. ● Explain how file transfer protocols operate.				
GENERAL OBJECTIVE: 4.0 Understand Ethernet switching						
8-9	4.1 Describe how the Ethernet sublayers are related to the frame fields. 4.2 Describe the Ethernet MAC address. 4.3 Describe how a switch builds its MAC address table and forwards frames. 4.4 Describe switch forwarding methods and port settings available on Layer 2 switch ports.	● Explain how the Ethernet sublayers are related to the frame fields. ● Describe the Ethernet MAC address. ● Explain how a switch builds its MAC address table and forwards frames.	Multi-media Projector, Lecture notes, Tutorials, Pictorials Multi-media Projector, Lecture notes, Tutorials, Pictorials Computer Routers Switches	● Use Wireshark to Examine Ethernet Frames ● View Network Device MAC Addresses ● View the Switch MAC Address Table	Guide students to; ● Use Wireshark to Examine Ethernet Frames ● View Network Device MAC	Describe how a switch builds its MAC address table and forwards frames

		Describe switch forwarding methods and port settings available on Layer 2 switch ports.			Addresses View the Switch MAC Address Table	
GENERAL OBJECTIVE: 5.0 Understand Number systems and IP addressing						
10	5.1 Explain number systems 5.2 Explain how to convert numbers between decimal and binary systems. 5.3 Explain how to convert numbers between decimal and hexadecimal systems. 5.4 Explain the structure of an IPv4 address including the network portion, the host portion, and the subnet mask. 5.5 Analyse the characteristics and uses of the unicast, broadcast and multicast IPv4 addresses. 5.6 Describe public, private, and reserved IPv4 addresses. 5.7 Describe how subnetting segments a network to enable better communication. 5.8 Compute IPv4 subnets for a /24 prefix. 5.9 Describe the need for IPv6 addressing. 5.10 Describe how IPv6 addresses are	- Explain how to convert decimal and binary systems. - Explain how to convert numbers between decimal and hexadecimal systems. - Describe the structure of an IPv4 address including the network portion, the host portion, and the subnet mask. - Explain the characteristics and uses of the	Multi-media Projector, Lecture notes, Tutorials, Pictorials	Convert binary numbers to decimal numbers Convert decimal numbers to Hexadecimal numbers	Guide students to: Demonstrate how to convert binary numbers to decimal numbers Demonstrate the conversion of decimal numbers to Hexadecimal numbers	Explain the structure of an IPv4 address including the network portion, the host portion, and the subnet mask.

	represented. 5.11 Contrast types of IPv6 network addresses. 5.12 Describe how to configure static global unicast and link-local IPv6 network addresses. 5.13 Describe how to configure global unicast addresses dynamically.	unicast, broadcast and multicast IPv4 addresses. • Explain public, private, and reserved IPv4 addresses. • Explain how subnetting segments a network to enable better communication. • Explain how to calculate IPv4 subnets for a /24 prefix. • Explain the need for IPv6 addressing. • Explain how IPv6 addresses are represented. • Explain the types of IPv6 network addresses(Unicast and Anycast) • Explain how to configure static global unicast				
--	--	---	--	--	--	--

		- and link-local IPv6 network addresses. - Explain how to configure global unicast addresses dynamically.				
GENERAL OBJECTIVE: 6.0 Know Basic router and switch configurations						
11-12	6.1 Explain how to access IOS or VRP device for configuration purposes. 6.2 Explain how to navigate IOS or VRP to configure network devices. 6.3 Explain the command structure of IOS or VRP software. 6.4 Explain IOS or VRP device using CLI. 6.5 Explain IOS or VRP commands to save the running configuration. 6.6 Explain how devices communicate across network media. 6.7 Explain a host device with an IP address.	- Explain how to access Internetwork Operating System(IOS) or Virtual Router Protocol (VRP) device for configuration purposes. - Explain how to navigate IOS or VRP to configure network devices.	Multimedia Projector, Routers and Switches, Packet Tracer or eNSP or GNS3, Laptop Internet Books, Journals, Marker board, Marker, Textbooks, Tera Term, Putty	- Configure a Switch and navigate through the IOS or VRP - Configure InitialSwitch Settings	Guide student to - Navigate the IOS or VRP by Using Tera Term for Console Connectivity - Configure InitialSwitch Settings	Explain how to navigate IOS or VRP to configure network devices.

	6.8 Explain connectivity between two end devices. 6.9 Explain initial settings on an IOS or VRP router. 6.10 Explain two active interfaces on IOS or VRP router. 6.11 Explain how devices use the default gateway.	● Explain the command structure of IOS or VRP software. ● Explain IOS or VRP device using CLI. ● Explain IOS or VRP commands to save the running configuration. ● Explain how devices communicate across network media. ● Explain a host device with an IP address.				
--	--	---	--	--	--	--

		● Explain connectivity between two end devices. ● Explain initial settings on an IOS or VRP router. ● Explain two active interfaces on IOS or VRP router. ● Explain how devices use the default gateway.				
GENERAL OBJECTIVE: 7.0 Understand Network Security fundamentals						
13-14	7.1 Explain why basic security measures are necessary on network devices. 7.2 List general security mitigation techniques. 7.3 Explain network devices with device hardening features to mitigate security	● Explain why basic security measures are necessary on network devices. ● Explain	Multimedia Projector, Routers and Switches, Packet Tracer or eNSP or GNS3, Laptop Internet	● Configure secure passwords and SSH on network devices.	● Guide the students to configure secure password	List general mitigation techniques

	threats.	general security mitigation techniques. Explain network devices with device hardening features to mitigate security threats.	Books, Journals, Marker board, Marker, Textbooks, TeraTerm, Putty		ds and SSH on network devices.	
GENERAL OBJECTIVE: 8.0 : Understand how to build small networks						
14-15	8.1 Explain the devices used in a small network. 8.2 Explain the protocols and applications used in a small network. 8.3 Explain how a small network serves as the basis of larger networks. 8.4 Explain common network troubleshooting methodologies.	- Explain the devices used in a small network. - Explain the protocols and applications used in a small network. - Explain how	Multimedia Projector, Routers and Switches, Packet Tracer or eNSP or GNS3, Laptop Internet Books, Journals, Marker board, Marker, Textbooks, TeraTerm, Putty	- Design a small network, and use ping and traceroute to test for latency and connectivity. - Troubleshoot connectivity issues.	Guide the students to: Design a small network, and use ping and traceroute to test for latency	Explain the protocol and application used in a small network

	8.5 List common connectivity problems associated with network devices	a small network serves as the basis of larger networks. ● Explain common network troubleshooting methodologies. ● Explain common connectivity problems associated with network			and connectivity. ● Troubleshoot connectivity issues.	
--	---	--	--	--	--	--

PROGRAMME: HIGHER NATIONAL DIPLOMA SOFTWARE AND WEB DEVELOPMENT		
COURSE: Introduction to Software Engineering	COURSE CODE: SWD316	CONTACT HOURS: 5
	CREDIT UNIT: 3	THEORETICAL: 3
YEAR: I SEMESTER: I	PRE-REQUISITE:	PRACTICAL: 2
GOAL: This course provides students with knowledge and skills in Software Engineering		
GENERAL OBJECTIVES: On completion of this course the student should be able to: 1.0 Understand the Fundamentals of Software Engineering 2.0 Know Software Processes and Models 3.0 Understand Software Requirements 4.0 Comprehend Software Design Process 5.0 Understand Software Development 6.0 Understand Software Testing 7.0 Understand Software Management		

PROGRAMME: HIGHERNATIONAL DIPLOMA SOFTWARE AND WEB DEVELOPMENT						
COURSE: Introduction to Software Engineering		COURSE CODE:SWD316		CONTACT HOURS: 5hrs		
		CREDIT UNIT:3		THEORETICAL: 3		
YEAR: I SEMESTER: I		PRE-REQUISITE:		PRACTICAL:2		
COURSE SPECIFIFCATION: THEORETICAL AND PRACTICAL						
GOAL: This course provides students with knowledge and skills in Software Engineering						
GENERAL OBJECTIVE 1.0: Understand the Fundamentals of Software Engineering						
THEORETICAL CONTENT				PRACTICAL CONTENT		
Week	Specific Learning Outcome	Teacher’s Activities	Resources	Specific Learning Outcome	Teacher’s Activities	Evaluation
1-3	1.1 Define SoftwareEngineering 1.2 Explain the need for SoftwareEngineering 1.3 Explain the characteristics of goodsoftware 1.4 Explain SoftwareEvolution 1.5 Explain abstraction and decomposition	Discuss Software Engineering and the need forit Explain the Characteristics of a good software Explain the	White Board. PC loaded with Presentation package and connected to multimedia Projector			What are the characteristics of good software? What are the laws used in the evolution of computer

	in software engineering	evolution of computer software. Explain the laws used for software evolution. Explain the techniques of software evolution. Explain abstraction and decomposition in software engineering	Online lecture notes			software? What are the 3Rs of software evolution?
GENERAL OBJECTIVE: 2.0 Know Software Process and Models						
4-5	2.1 Explain Software Process and its activities 2.2 Explain Software generic Process model 2.3 Explain Software process models i.e. waterfall, Spiral, V-model, Rapid Application Development, Agile etc. 2.4 Explain Software Prototyping and Types 2.5 Explain advantages and Disadvantages of Prototyping	Explain Software development Process and its activities Explain various software development process models Explain Software Prototyping, its types, advantages and Disadvantages	White Board and marker. PC loaded with Presentation package and connected to multimedia Projector figma wireframe Online lecture	Identify the strengths and weaknesses of various software process models Demonstrate the use of prototyping software.	Guide students to: Identify and compare the strengths and weaknesses of various models Demonstrate the use of Prototyping software such as figma, wireframe	What are the strengths and weaknesses of various software process models? What are the types of Software Prototyping? Discuss the advantages

			notes			and disadvantage s of Prototyping.
GENERAL OBJECTIVE: 3.0 Understand Software Requirements						
6-7	3.1 Explain Requirement Engineering Process 3.2 Explain various techniques for requirements elicitation e.g. Data Flow technique, structured natural language 3.3 Explain software specification and its Importance 3.4 Explain the structure and contents of Software Requirements Specification (SRS) Document.	Explain Requirement Engineering Process Explain various techniques for requirements elicitation Explain software specification and its importance Explain the structure and contents of Software Requirements Specification (SRS) documents	White Board and marker PC loaded with Presentation package and connected to multimedia Projector Online lecture notes	Determine specification for systems using various techniques Identify the contents of a SRS document. Create SRS Document of systems.	Guide students to: Determine the specification for systems using various techniques Create SRS Document	Demonstrate how to determine specifications and create SRS documents.

GENERAL OBJECTIVE: 4.0 Comprehend Software Design Process						
8 - 10	4.1 Explain Software Design and its importance 4.2 Explain stages in Software design process 4.3 Explain software design tools e.g. Algorithm, UML, XML etc. 4.4 Explain Software Design Strategies 4.5 Explain Software Design model: data/class, architectural design, interface design and component design 4.6 Explain object oriented design: classes, objects, components etc. 4.7 Explain Software Architecture and architectural styles 4.8 Explain design quality attributes e.g. cohesion, coupling, understand ability, adaptability etc. 4.9 Explain Software Design Verification	Explain Software Design and its importance Explain stages in Software design process Explain software design tools e.g. Algorithm, UML, XML etc. Explain Software Design Strategies Discuss the features of different software design tools. Explain Software Design model Discuss object oriented design Explain Software	White Board and marker. PC loaded with Presentation package and connected to multimedia Projector Networked PCs Loaded With XML, UML White Board and marker. PC loaded with Presentation package	Create simple object-oriented designs of some systems using flowcharts Compare the features of different software design tools. Create object-oriented designs of some systems using XML and UML Carryout verification for object-oriented designs	Guide students to : Create simple object-oriented designs of some systems using flowcharts Compare the features of different software design tools. Guide students to: Create simple object-oriented designs of some	Demonstrate how to use XML and UML to create simple object-oriented designs of some systems. Make a comparison between different software design tools. Use XML and UML to create simple object-oriented designs of some systems. Verify software designs.

		Architecture, architectural styles and design quality attributes. Explain software design verification	and connected to multimedia Projector Networked PCs Loaded With XML, UML		systems using XML and UML Verify object-oriented designs.	
GENERAL OBJECTIVE: 5.0 Understand Software Development						
11-12	5.1 Define Software Development 5.2 Explain the activities involved in software development 5.3 Explain Application Program Interface (APIs) and their uses 5.4 Explain various software Development tools and their Uses 5.5 explain the process of developing software using various development tools e.g. Notepad++, PHP, Python, Visual Studio, C++, Java etc	Explain Software Development and its activities Explain APIs and its uses Explain various software development tools and their Uses Explain the process of	White Board and marker. PC loaded with Presentation package and connected to multimedia Projector Networked PCs Loaded With software develop	Develop software using suitable software development tools e.g. PHP, C++, Java etc	Guide students to develop software using suitable software development tools	What are the strengths and weaknesses of various software development tools? Demonstrate how to develop software using suitable software development tools

		developing software using various development tools	ment tools			
GENERAL OBJECTIVE: 6.0 Understand Software Testing						
13-14	6.1 Define software Testing and its importance 6.2 Explain Test Phases: unit testing, integration testing, system testing etc. 6.3 Explain Test Methods: black box, white box, static, dynamic etc. 6.4 Explain software testing types: installation testing, usability testing, recovery testing, security testing etc. 6.5 Explain Software Testing Techniques: fault injection, Application program Interface (API) Testing, static testing etc. 6.6 Distinguish between software Validation and Verification 6.7 Explain the process of software validation and verification	Explain Software Testing, its types and importance Explain Test Phases, Test Methods, and Techniques Explain the process of software validation and verification Distinguish between software	White Board and marker. PC loaded with Presentation package and connected to multimedia Projector Networked PCs Loaded With software development	Draw up testing schedule and perform the testing accordingly. Carryout different test types. Use of different testing methods and types. Validate and verify software.	Guide students to: Draw up testing schedule and perform the testing Accordingly using software testing tools such as selenium, apache jmeter Use of different testing methods and types. Validate and verify	Distinguish between Test Types, Test Phases, Test Methods, and Test Techniques. Compare dynamic and static testing. Compare Black box and White box testing.

		Validation and Verification	tools selenium apache jmeter		software.	
GENERAL OBJECTIVE: 7.0 Understand Software Management						
15	7.1 Explain Software Project Management 7.2 Explain the activities involved in software management: proposal writing, project planning and scheduling, project monitoring, etc. 7.3 Explain Software project Scoping 7.4 Explain Software project planning and its activities 7.5 Explain Software Quality Management: quality control and quality assurance	Explain Software Project Management and its activities Discuss Software project planning and its activities Explain Software project Scoping Discuss Software Quality Management	White Board and marker. PC loaded with Presentation package and connected to multimedia Projector Networked PCs Loaded With software development tools Gantt chart Online lecture materials	Write project proposal Perform Project planning and scheduling	Guide students to: Write project proposal. Perform Project planning and scheduling. Write a software project scope.	Demonstrate how to write project proposal, perform project planning and scheduling. Carryout quality management. Write a project scope.

YEAR 1 SEMESTER II

PROGRAMME: HIGHER NATIONAL DIPLOMA SOFTWARE AND WEB DEVELOPMENT		
COURSE: Python Programming	COURSE CODE: SWD321	CONTACT HOURS: 5
	CREDIT UNIT: 3	THEORETICAL: 2
YEAR: I SEMESTER: II	PRE-REQUISITE:	PRACTICAL: 3
GOAL: This course is designed to enable students evolve critical thinking and problem-solving skills using algorithmic approach		
GENERAL OBJECTIVES: Upon completion of this course, the student should be able to: 1. Comprehend general computer programming concepts and techniques 2. Understand Data types, variables, basic input-output operations, and basic operators 3. Know Boolean values, conditional execution, loops, lists and list processing, logical and bitwise operations 4. Understand python functions, tuples, dictionaries, and data processing 5. Understand python modules, libraries, packages, and PIP 6. Understand strings, string and list methods, and exceptions 7. Know the concepts and application of Object-Oriented Programming 8. Understand working with Databases in Python 9. Understand Basics Data Analysis with Python		
.		

PROGRAMME: HIGHERNATIONAL DIPLOMA SOFTWARE AND WEB DEVELOPMENT							
COURSE: Python Programming			COURSE CODE: SWD321		CONTACT HOURS: 5hrs		
			CREDIT UNIT: 3		THEORETICAL: 2		
YEAR: I SEMESTER: II			PRE-REQUISITE:		PRACTICAL: 3		
COURSE SPECIFIFCATION: THEORETICAL AND PRACTICAL							
GOAL: This course is designed to enable students evolve critical thinking and problem-solving skills using algorithmic approach							
GENERAL OBJECTIVE 1.0: : Comprehend general computer programming concepts and techniques							
THEORETICAL CONTENT				PRACTICAL CONTENT			
Week	Specific Learning Outcome		Teacher’s Activities	Resources	Specific Learning Outcome	Teacher’s Activities	Evaluation
1	1.1	Explain the fundamentals of computer programming	Explain how the computer works and how program is executed	White board, PCs/Laptops, Internet, Cisco LMS content, Multimedia, and flip chart	Install and configure Python Interpreter, IDE / Code editor	Guide students to: Install and configure Python, IDE / code editor	Explain the difference between compilation and interpretation
	1.1	Explain the difference between compilation and interpretation			Analyse and compare different code editors		
	1.2	Describe what python is					Analyse different code editors
			Explain how the python programming language is defined and constructed				
			Explain and differentiate between compilation and interpretation of program				

		Explain Python's position among other programming languages, and what distinguishes the different versions of Python				
GENERAL OBJECTIVE: 2.0 Understand Data types, variables, basic input-output operations, and basic operators						
2 -3	2.1 Explain how to write and run simple Python programs 2.2 Describe what Python literals, operators and expressions are 2.3 Explain what variable are and what are the rules that govern them 2.4 Explain how to perform basic Python input and output operations	Explain how to write and run simple Python programs Explain Python literals and operations with examples Discuss Python expressions with examples Discuss variables, enumerates its attributes and rules that govern them Discuss how to perform basic Python input and output	White board, PCs/Laptops, Internet, Cisco LMS content, Multimedia, IDE / Cisco Sandbox / Code Editor, and flip chart	Construct simple Python codes and test them Design programs to perform basic Python input and output operations	Guide students to: Construct simple Python codes and test them Design programs to perform basic Python input and output operations	Take Quiz using tools like slido, kahoot, poll everywhere, etc. Take assessment on Cisco NetAcad LMS

		operations				
GENERAL OBJECTIVE: 3.0 : Know Boolean values, conditional execution, loops, lists and list processing, logical and bitwise operations						
4	3.1 Explain the Boolean data type 3.2 Describe relational operators 3.3 Explain conditional statements and/or executions 3.4 Describe how to repeat code execution using loops 3.5 Explain how to perform logic and bitwise operations in Python 3.6 Explain lists in Python and related operations 3.7 Explain the concept of sorting list elements or contains 3.8 Explain multidimensional lists and their applications	Explain Boolean data type and enumerates examples Discuss relational operators with examples Discuss conditional statements in Python (using if, if-else, if-elif-else statements) Discuss how to control looping using <i>break</i> and <i>continue</i> instructions Discuss repeating code execution using loop statements (e.g., while, for)	White board, PCs/Laptops, Internet, Cisco LMS content, Multimedia, IDE / Cisco Sandbox / Code Editor, and flip chart	Design Python codes to demonstrate 1) Boolean operations, 2) relational operators, 3) conditional statements using if, if-else, if-elif-else statements Construct Python codes to perform logic and bitwise operations Write Python codes to repeat code execution using while and for loops statements – and how to control the loops using <i>break</i> and <i>continue</i> instructions Design Python programs to perform 1) list indexing, 2) list slicing, 3) list	Guide students to: Design Python codes to demonstrate 1) Boolean operations, 2) relational operators, 3) conditional statements (using if, if-else, if-elif-else) Construct Python codes to perform logic and bitwise operations Write Python	Perform lab activities on Cisco LMS Sandbox (virtual lab) Take assessment (test) on Cisco NetAcad LMS Collaborate on group tasks to demonstrate teamwork as potential developers Take Quiz using tools like slido, kahoot, poll everywhere, etc.

		Discuss how to perform logic and bitwise operations in Python with examples Explain lists in Python Discuss list indexing, slicing, comprehension, and content manipulation Discuss how to sort a list using bubble-sort algorithms Discuss list methods and how they are used Discuss multidimensional lists and their applications		comprehension, 4) list content manipulation Write Python codes to sort list elements Design Python programs to demonstrates list methods	codes to repeat code execution (while and for loops statements) Design Python programs to perform 1) list indexing, 2) list slicing, 3) list comprehension , 4) list content manipulation Collaborate on writing Python codes to sort list elements Collaborate on designing Python programs to demonstrates various list methods	
--	--	--	--	---	---	--

GENERAL OBJECTIVE: 4.0 Understand python functions, tuples, dictionaries, and data processing						
5-6	4.1 Explain code structuring and the concept of function 4.2 Describe function invocation and returning a result from a function 4.3 Define name scopes and variable shadowing 4.4 Explain tuples and their purpose, constructing and using tuples 4.5 Explain dictionaries and their purpose, constructing and using dictionaries 4.6 Explain exceptions – the <i>try</i> statement the <i>except</i> clause, Python built-in exceptions, code testing and debugging	Discuss code structuring in relation to the concept of function – decomposition, positional parameter passing, keyword parameter passing with examples Discuss function construction using <i>def</i>, function invocation, and returning result with examples Discuss the concept of namespaces and variable shadowing with examples Discuss tuples, its structure, purpose and immutable nature.	White board, PCs/Laptops, Internet, Cisco LMS content, Multimedia, IDE / Cisco Sandbox / Code Editor, and flip chart	Illustrate problem decomposition Demonstrate function positional and keyword parameter passing Construct functions to solve simple problems and invoke them to return results Construct functions with variable that the namespace is inside the function and variable outside the function that could be shadow inside the function Construct tuples and manipulate the contents and test the immutablenature of tuples Construct dictionaries and	Guide students to : Collaborate on task to be decomposed into smaller tasks Write programs that pass parameters into functions using 1) positional parameter, 2) keyword parameter, 3) default parameter, 4) combine all Write functions to solve real life challenges Construct	Perform lab activities on Cisco LMS Sandbox (virtual lab) Take assessment (test) on Cisco NetAcad LMS Collaborate on group tasks to demonstrate teamwork as potential developers Take Quiz using tools like slido, kahoot, poll everywhere, etc.

		Explain how to use tuples with examples Compare tuples with list – similarities and differences Discuss dictionaries, its structure, purpose and nature (mutable and non-sequence). Explain how to use dictionaries with examples Compare dictionaries with list and tuples – similarities and differences Discuss Python exception, the use of <i>try</i> statement and <i>except</i> clause with examples Discuss Python built-		manipulate the contents and test the mutability and non-sequential nature of dictionary Design programs that use <i>try</i> statement and <i>except</i> clause to test Python built-in exceptions Construct Python programs and manipulate them to test and debug errors	functions that demonstrate variable shadowing Construct tuples and manipulate the contents and test the immutable nature of tuples Construct dictionaries, manipulate the contents and test the mutability and non-sequential nature of dictionary Design programs that use <i>try</i> statement and <i>except</i> clause to test Python	
--	--	--	--	---	--	--

		in exceptions, and how to use them with examples			built-in exceptions	
		Explain Python code testing and debugging processes with examples			Construct Python programs and manipulate them to test and debug errors	
GENERAL OBJECTIVE: 5.0 Understand python modules, libraries, packages, and PIP						
7-8	5.1 Explain Python module, how to import and use Python modules 5.2 Describe some of the most useful Python standard library modules and how to use them 5.3 Explain how to define and construct Python modules 5.4 Explain Python Packages, how to construct packages and use them 5.6 Explain PIP (Python Installation Package) and how to use it to install and	Discuss Python modules. Discuss the most useful Python standard library modules and different ways of importing the modules Discuss module construction, structure, requirements (e.g.,	White board, PCs/Laptops, Internet, Cisco LMS content, Multimedia, IDE / Cisco Sandbox / Code Editor, and flip chart	Design codes to import Python standard library modules using different import techniques and how to use entities inside the codes Construct modules that will have related functions, variables, as entities Design packages to have the constructed modules as entities	Guide students to: Design codes that import Python standard library modules using different import techniques(use the entities inside the codes)	Perform lab activities on Cisco LMS Sandbox (virtual lab) Take assessment (test) on Cisco NetAcad LMS Take Quiz using tools like slido, kahoot, poll everywhere, etc.

	uninstall ready-to-use packages from PyPI	constructor, classes, etc.) and how to use them Discuss Python packages, and compare with modules Discuss PIP and how to use it to install and uninstall packages.		Install and use PIP to install and uninstall packages	Construct modules that will have related functions and variables as entities Design packages that use the above constructed modules as entities Install PIP, and use PIP to install /uninstall packages	
--	---	--	--	---	---	--

GENERAL OBJECTIVE: 6.0 Understand strings, string and list methods, and exceptions

9-10	6.1 Explain characters, strings and coding standards 6.2 Explain Strings vs. Lists in terms of	Discuss characters and strings Discuss coding	White board, PCs/Laptops, Internet, Cisco LMS content, Multimedia, IDE	Design Python codes to manipulate characters and strings	Guide students to: Design Python codes that	Perform lab activities on Cisco LMS Sandbox (virtual lab)
------	---	--	--	--	--	---

	similarities and differences 6.3 Explain list methods 6.4 Explain string methods 6.5 Describe Python's way of handling runtime errors 6.6 Explain how to control the flow of errors using <i>try</i> and <i>except</i> 6.7 Explain hierarchy of exceptions	standards and how to use them in programs Compare strings and lists in terms of similarities and differences with examples Discuss list methods with examples Discuss string methods with examples Discuss how Python handle runtime errors with examples Discuss how to control the flow of errors using <i>try</i> statement and <i>except</i> clause Discuss hierarchy of exceptions	/ Cisco Sandbox / Code Editor, and flip chart	Construct programs that use list and string methods to manipulate list elements and strings Design Python programs that handle runtime errors Construct Python codes that control the flow of errors by using <i>try</i> and <i>except</i>	manipulate characters and strings Construct programs that use list and string methods to manipulate list elements and strings Design Python programs that handle runtime errors Construct Python codes that control the flow of errors by using <i>try</i> and <i>except</i>	Take assessment (test) on Cisco NetAcad LMS Take Quiz using tools like slido, kahoot, poll everywhere, etc.
--	--	--	---	---	--	---

GENERAL OBJECTIVE: 7.0 Know the concepts and application of Object-Oriented Programming						
11-12	7.1 Explain basic concepts of object-oriented programming (OOP) 7.2 Explain the differences between the procedural and object approaches (motivations and profits) 7.3 Explain Classes, objects, properties, and methods 7.4 Describe Designing reusable classes and creating objects 7.5 Explain abstraction, encapsulation, inheritance and polymorphism 7.6 Explain exceptions as objects 7.8 Explain generators, iterators, and closures 7.9 Describe working with filesystem, directory tree and files 7.10 Explain selected Python Standard	Discuss basic concepts of Object-Oriented Programming (OOP) Discuss the differences between the procedural and object oriented approaches (motivations and profits) Discuss classes, objects, properties and methods Discuss how to design reusable classes and how to create objects with examples Discuss the concepts of abstraction, inheritance,	White board, PCs/Laptops, Internet, Cisco LMS content, Multimedia, Python Interpreter, IDE / Cisco Sandbox / Code Editor, and flip chart	Design Python codes to compare Procedural-Oriented Programming (POP) and Object-Oriented Programming (OOP) approaches Construct Python Classes, objects, and methods as it relates to OOP Design reusable Classes and create objects Write Python codes that demonstrate abstraction, inheritance, encapsulation and polymorphism Construct programs that demonstrate exceptions as objects Design Python codes that illustrate and test Python's generators, iterators, and	Guide students to: Design Python codes that compare Procedural-Oriented Programming (POP) and Object-Oriented Programming (OOP) approaches Construct Python Classes, objects, and methods based on OOP Design reusable Classes and create objects	Perform lab activities on Cisco LMS Sandbox (virtual lab) Take assessment (test) on Cisco NetAcad LMS Take Quiz using tools like slido, kahoot, poll everywhere, etc.

	Library modules (OS, datetime, time, and calendar) encapsulation, and polymorphism with examples Discuss exceptions as objects with examples Discuss generators, iterators and closures with their related functions (lamda, filter, etc.) and examples Discuss how to work with filesystem, directory and files in Python Discuss selected Python Standard Library modules (e.g., OS, datetime, time, and calendar)		closures Design programs that solve real live problems using Python's standard library modules	Write Python codes that demonstrate abstraction, inheritance, encapsulation and polymorphism Construct programs that demonstrate exceptions as objects Design Python codes that illustrate and test Python's generators, iterators, and closures Design programs that solve real live problems using Python's	
--	--	--	--	---	--

					standard library modules	
GENERAL OBJECTIVE: 8.0 Understand working with Databases in Python						
13	8.1 Explain the concepts of API 8.2 Describe the different databases that Python API supports 8.3 Explain MySQL database commands	Discuss the concepts of API Explain the different Databases that Python API supports Discuss database operations, syntaxes and functions of the following database commands: <i>Create Database, Create Table, Insert, Select, Where, Order by, Delete, Drop Table, Update, Join</i>	White board, PCs/Laptops, Internet, Cisco LMS content, Multimedia, Python Interpreter, IDE / Cisco Sandbox / Code Editor, and flip chart	Create a database and name it Create and name a table, having two fields “Name” and “Address” in the database created Insert a record into the table Query the table Delete a record from the table Delete the table from the database	Guide students to : Create a database and name it Create and name a table, having two fields “Name” and “Address” in the database created Insert a record in the table Query the table Delete a record from the table	

					Delete the table from the database	
GENERAL OBJECTIVE: 9.0 Understand Basic Data Analysis with Python						
14-15	9.1 Explain the concept of Big Data 9.2 Outline the strengths of Python as a programming language for Big Data analysis. 9.3 Explain the functions of NumPy, Pandas, and Matplotlib libraries 9.4 Define Dataset and explain how it differs from a Database 9.5 Explain the commands used for importing and exporting datasets. Give their syntaxes 9.6 Explain the need for cleaning data in preparation for analysis 9.7 Explain correlation and why it an important metric in data analysis 9.8 Explain unstructured and semi structured data	Explain Big Data and its characteristics: Volume, Velocity, Variety, and Veracity. Explain why Python is a programming language that is used for Big Data analysis. Explain the functions of essential Python libraries for data analysis such as NumPy, Pandas, and Matplotlib Explain the function of	White board, PCs/Laptops, Internet, Cisco LMS content, Multimedia, Python Interpreter, IDE / Cisco Sandbox / Code Editor, and flip chart	Create a 3-dimensional array using the array command, assign data type to the array using dtype argument, and check its dimension using ndim Method Create a series using the Series() Method and print the series using a Print statement. Create a Dataframe in Pandas using the Dataframe statement and print it using the Print statement Import and open a CSV file using the Import statement and the <i>Open</i> () function.	Guide students to : Demonstrate the functions of NumPy by creating an array using the Array function, assign data type to the array using dtype argument, and check its dimension using ndim() Method. Demonstrate the functions of Pandas by creating a series using	Outline the strengths of Python as a programming language for Big Data analysis Explain unstructured and semi structured data

		Datasets with examples. Compare Datasets and Databases. Explain the process of importing and exporting datasets Explain the process of cleaning and preparing data for analysis. Explain correlation and the different types of correlation with examples. Explain unstructured and semi structured data Introduce NoSQL databases using MongoDB		Export CSV files using the <i>WriteRow()</i> function Import data into a Python dataset using the Import statement and the Insert function Use the <i>Fillna()</i> and <i>Dropna()</i> functions to fill missing values and drop missing values respectively in a dataset. Calculate correlation using the <i>Corr()</i> method. Use the <i>pyMongo</i> function in Python to connect to <i>MongoDB</i> and <i>insert</i>, <i>update</i>, and <i>delete</i> records using the <i>Insert()</i>, <i>Update()</i>, and <i>Delete</i> methods	the <i>Series()</i> Method, print the series using a <i>Print</i> statement. Create a Dataframe in Pandas using the <i>Dataframe</i> statement Import and open CSV files using the <i>Import</i> statement and the <i>Open()</i> function. Demonstrate how to export CSV file. Use the <i>Fillna()</i> and <i>Dropna()</i> functions to fill missing	
--	--	---	--	--	---	--

					values and drop missing values respectively in a dataset. Illustrate how correlation is calculated using the <i>Corr()</i> method. Demonstrate how to work with commands in a <i>NoSQL</i> database like MongoDB	
--	--	--	--	--	---	--

Assessment: Give details of assignments to be used:

Coursework/ Assignments %; Course test %; Practical %; Projects %; Examination %

Type of Assessment	Purpose and Nature of Assessment (SWD 321)	Weighting (%)
--------------------	--	---------------

Examination	Final Examination (written) to assess knowledge and understanding	40
Test	At least 2 progress tests for feedback.	10
Practical	At least 5 hands-on activities to be assessed by the teacher	50
Total		100

PROGRAMME: HIGHER NATIONAL DIPLOMA SOFTWARE AND WEB DEVELOPMENT		
COURSE: Database Design II	COURSE CODE: SWD322	CONTACT HOURS: 5
	CREDIT UNIT: 3	THEORETICAL: 2
YEAR: I SEMESTER: II	PRE-REQUISITE: SWD312	PRACTICAL: 3
GOAL: This course provides students with knowledge and skills to determine an appropriate database model for specific task.		
GENERAL OBJECTIVES: Upon completion of this course, students should be able to: 1. Comprehend Data, its types, structure, and benefits of Data Integrity. 2. Understand Relational Database and Document Database. 3. Know how to use Aggregate Functions in Queries 4. Understand Query Optimization. 5. Understand Object Relational Mappers and their benefits.		

PROGRAMME: HIGHERNATIONAL DIPLOMA SOFTWARE AND WEB DEVELOPMENT						
COURSE: Database Design II			COURSE CODE:SWD322		CONTACT HOURS: 5	
			CREDIT UNIT:3		THEORETICAL:2	
YEAR: I SEMESTER: II			PRE-REQUISITE:SWD312		PRACTICAL:3	
COURSE SPECIFIFCATION: THEORETICAL AND PRACTICAL						
GOAL: This course provides students with knowledge and skills to determine an appropriate database model for specific task.						
GENERAL OBJECTIVE 1.0: Comprehend Data and its types and structure.						
THEORETICAL CONTENT				PRACTICAL CONTENT		
Week	Specific Learning Outcome	Teacher’s Activities	Resources	Specific Learning Outcome	Teacher’s Activities	Evaluation
1-2	1.1 Explain Data and its different structures. 1.2 Explain the difference between structured and unstructured data 1.3 Explain Data Integrity 1.4 Define Atomicity, Consistency, Isolation, Durability. 1.5 Define Referential Integrity, Indexes	Discuss Data and its two main types: structured and unstructured data. Distinguish between structured data, which follows a defined schema, and unstructured data, which lacks a predefined structure. Discuss the challenges and advantages of working with unstructured data in	MongoDB MySQL, Relevant Textbook, Articles, YouTube Videos	Install and configure MongoDB and MySQL correctly	Guide students to set up and install MongoDB and MySQL on their Computers	

		various applications. Discuss the benefits of Data Integrity Describe the application of ACID in Data management. Discuss the benefits of indexes and Referential Integrity in Data.				
GENERAL OBJECTIVE: 2.0 Understand Relational Databases and Document Database						
3-8	2.1 Explain Relational Database 2.2 Explain Document Database 2.3 Explain Collections in Document Databases. 2.4 Explain the concept of SQL and NoSQL	Discuss databases, Explain how a relational database stores information, its strengths and weaknesses using MySQL Discuss document database and how it differs from relational Database. Discuss its flexibility in data schema handling. Discuss the nature of collections in document databases.	MySQL, Relevant Textbook, Articles, YouTube Videos Mongo DB, Relevant Textbook, Articles, YouTube Videos Mongo DB, Relevant Textbook, Articles, YouTube Videos	Create a table in a MySQL Database and populate the table with at least 2 rows of information Create a database in MongoDB and store any information in a collection Create a new collection and save different data with different schemas in the same collection	Guide students to create a simple database in MySQL with at least one table and multiple rows of data Guide Students to verify configuration of MongoDB and Store basic data in a collection Guide students to: Demonstrateth	

		Explain its differences with tables Discuss Structured Query Language and NoSQL			e flexibility of collections Create a new collection and save different data with different schemas in the same collection	
GENERAL OBJECTIVE: 3.0 : Know how to use Aggregate Functions in Queries						
9	3.1 Explain Aggregate Functions 3.2 Describe the ‘group by’, and ‘having’ keywords in aggregate queries	Discuss aggregate functions and their uses. Explain the use of <i>group by</i> in aggregate queries and using <i>having</i> to control selection	MySQL, Relevant Textbook, Articles, YouTube Videos	Utilize aggregate functions to get grouped data in a query Use the <i>having</i> keyword to control selection	Guide students in the: Use of aggregate functions like max, min, avg and more. Use of <i>having</i> to control selection	
GENERAL OBJECTIVE: 4.0						
10- 12	4.1 Define query optimization 4.2 Explain the efficiency of database queries and the significance of query optimization in reducing query execution	Explain query optimization as the process of improving performance.	MySQL, Relevant Textbook, Articles, YouTube Videos	Analyze and evaluate query execution plans. Apply indexing to an existing table to gain performance improvement.	Guide students to: Demonstrate common	

	time and resource utilization. 4.3 Explain indexing and its benefits on query performance 4.4 Explain indexing as a fundamental technique for enhancing query performance by providing rapid data access.	Discuss efficiency of database queries and the significance of query optimization in reducing query execution time and resource utilization. Explain the types of database indexes, and their use cases. Discuss indexing as a fundamental technique for enhancing query performance by providing rapid data access.	MySQL, Relevant Textbook, Articles, YouTube Videos		performance bottlenecks in database queries, such as inefficient joins, excessive data retrieval, and lack of indexes. Guide students in applying indexing to a column of a table and show the performance improvements before and after	
GENERAL OBJECTIVE: 5.0 : Understand Object Relational Mappers(ORM) and their benefits						
13 -15	5.1 Define Object Relational Mapper 5.2 Explain the benefits of using ORM. 5.3 List Different ORMS	Discuss ORM as a programming technique for mapping database tables to object-oriented code structures. Discuss the	MySQL, Relevant Textbook, Articles, YouTube Videos MySQL, Relevant Textbook,	Configure an ORM correctly and use it to make queries to a database	Guide Students to set up an ORM and perform basic queries	

		advantages of using ORM frameworks for abstracting database interactions. Outline various ORMs (like eloquent, entity framework, mongoose) available and compare their flexibility, performance, and usability	Articles, YouTube Videos			
--	--	--	---------------------------------	--	--	--

PROGRAMME: HIGHER NATIONAL DIPLOMA SOFTWARE AND WEB DEVELOPMENT		
COURSE: Front End Development I	COURSE CODE: SWD323	CONTACT HOURS: 6hrs
	CREDIT UNIT: 3	THEORETICAL: 2
YEAR: I SEMESTER: II	PRE-REQUISITE:	PRACTICAL: 4
GOAL: This course provides students with knowledge and skills for Frontend Development		
GENERAL OBJECTIVES: : Upon completion of the course, students should be able to: 1. Understand The basics of Frontend development and The Browser. 2. Know HTML Structure and common HTML tags and their attributes 3. Understand Styling and Different methods used in styling HTML documents. 4. Comprehend Javascript Datatypes, Syntax, and control structure. 5. Understand The DOCUMENT OBJECT MODEL(DOM).		

PROGRAMME: HIGHERNATIONAL DIPLOMA SOFTWARE AND WEB DEVELOPMENT						
COURSE: Front End Development I			COURSE CODE:SWD323		CONTACT HOURS: 6hrs	
			CREDIT UNIT:3		THEORETICAL: 2	
YEAR: I SEMESTER: II			PRE-REQUISITE:		PRACTICAL:4	
COURSE SPECIFIFCATION: THEORETICAL AND PRACTICAL						
GOAL: This course provides students with knowledge and skills for Frontend Development						
GENERAL OBJECTIVE 1.0: Understand the basics of Frontend development and the Browser.						
THEORETICAL CONTENT				PRACTICAL CONTENT		
Week	Specific Learning Outcome	Teacher’s Activities	Resources	Specific Learning Outcome	Teacher’s Activities	Evaluation
1	1.1 Define a browser, its core components, and its functionalities. 1.2 Explain The use of browser developer tools 1.3 Explain The Steps to set up the development environment. 1.4 Explain different web browsers, such	Discuss the functionalities of web browsers including developer tools, and roles of browsers in parsing and rendering HTML, CSS, and JavaScript. Explain the	Textbooks, Web Browser, Relevant YouTube videos, Relevant Articles, PC, Text Editor(VS Code/Vim/Subli	Create a correctly configured development environment Identify different browsers, their strengths and weaknesses Demonstrate the use of Developer tools in web browsers.	Guide Students on Installing web browsers, Text Editors, and Relevant Extensions for the editor. Guide Students to demonstrate	1

	as Chrome, Firefox, Safari, and Edge. 1.5 Explain browser developer tools, including the console, inspector, and network monitor 1.6 Explain client-side and server-side scripting in web development, emphasizing their respective responsibilities and execution environments.	Classification of different browsers showing their strengths and weaknesses Explain the workings of the Browser developer tools Explain the differences between client-side and server-side. Showing their benefits and concerns	me) Web Browsers, PC, Relevant Publications. Chrome Dev tools Sample Source code, Text Editor, Browser, Relevant Material, Text Book		the strengths and weaknesses of various browsers Guide the students to demonstrate the use of developer tools like console, network, and other tabs	
GENERAL OBJECTIVE: 2.0 Know HTML Structure and common HTML tags and their attributes						
2	2.1 Explain the foundational HTML template showing common HTML elements 2.2 Describe attributes of HTML tags showing their actions and use cases. 2.3 Explain common attributes like href, src, alt, and target.	Discuss the basics of HTML and show the commonly used tags, such as headings (<h1> to <h6>), paragraphs (<p>), lists (, ,), and anchor links (<a>) Explain HTML attributes of elements like the href of <a>, showing its use and relevant documentation.	W3schools.com, Text Books, Relevant Videos/Articles W3schools.com, https://developer.mozilla.org, Textbooks, Relevant articles, and videos	Create simple web pages with basic HTML tags Create multiple web pages and link them together, showing media files on the various pages.	Guide students on creating a simple webpage that has some basic header and list tags Guide the students on the creation of linked webpages using <a>, and showing multimedia files such as	2

		Discuss the use of multimedia elements (images, videos) and hyperlinks.			images, videos, and audios	
GENERAL OBJECTIVE: 3.0 Understand Styling and Different methods used in styling HTML documents.						
3 -7	3.1 Explain Styling, CSS3, and its fundamentals in creating visually appealing and dynamic backgrounds. 3.2 Explain and Identify the distinctions between inline, internal (embedded), and external CSS styling methods 3.3 List the different methods of styling 3.4 Explain CSS Selectors 3.5 Explain the various CSS selectors and their methods of targeting HTML elements. 3.6 Explain Layouts 3.7 Discuss layout CSS3 arrangement of content 3.8 Identify the various CSS positioning properties, including relative, absolute, fixed, and sticky. 3.9 Explain CSS transformation and animations	Discuss Styling, CSS3, and the benefits of creating user-friendly and attractive web pages Discuss the three methods of Cascading Style Sheets (CSS), their order of precedence, and their respective use cases. Discuss their advantages on elements to achieve global and consistent styling across multiple pages. Discuss CSS selectors, describing the element, class, id, attribute and pseudo-	W3schools.com, https://developer.mozilla.org, Textbooks, Relevant articles, and videos W3schools.com, https://developer.mozilla.org, Textbooks, Relevant articles, and videos Relevant Documentation W3schools.com, https://developer.mozilla.org, Textbooks, Relevant articles, and	Create and apply any form of styling on existing or new HTML documents. Create at least 3 web pages with inline, internal, and external styles respectively Create a webpage with external styling using selectors that target elements and style them. Create a basic responsive web layout using media queries and CSS rules to adapt content Apply different position properties on elements of a webpage Create a webpage with basic animations and transformations	Guide students to demonstrate the basic styling of HTML documents using typography, fonts, text alignment, color schemes, and background properties, background size. Guide the students to demonstrate implementation of inline, internal, and external styles on HTML files Guide the	

		selectors. Illustrate their role in targeting HTML elements. Explain CSS layouts using the models, such as the box model, flexbox, and grid, Discuss their applicability to designing web page layouts, showcasing their strengths and weaknesses Discuss element positioning in CSS, its types, their differences, and advantages Discuss the concept of CSS transformations, including translate, rotate, scale, and skew, for manipulating elements, positions and shapes.	videos Relevant Documentation W3schools.com, https://developer.mozilla.org , Textbooks, Relevant articles, and videos Relevant Documentation W3schools.com, https://developer.mozilla.org , Textbooks, Relevant articles, and videos Relevant Documentation W3schools.com, https://developer.mozilla.org , Textbooks, Relevant articles, and videos		students in the use of selectors and various pseudo-selectors (e.g., :hover, :focus, :nth-child) and pseudo-elements (e.g., ::before, ::after) Guide students to implement a responsive web layout using media queries and CSS rules to adapt content using grid and flex. Guide students on the various positioning properties of css, showing theirprecedence and effects. Demonstrate CSS transitions and animations to add basic	
--	--	---	--	--	--	--

			Relevant Documentation		animations to web elements, such as smooth transitions for hover effects and fade-ins.	
GENERAL OBJECTIVE: 4.0 Comprehend Javascript Datatypes, Syntax, and control structure.						
8- 12	4.1 Define Javascript 4.2 Define Datatypes 4.3 List data types in JavaScript 4.4 Define control structures in JavaScript 4.5 Explain objects, arrays, and Javascript Object Notation(JSON) 4.6 Explain the let, const, and var keywords in Javascript 4.7 Explain functions, Functions' scope. 4.7 Explain anonymous functions and closures 4.8 Explain basic await on async function calls	Discuss Javascript in Frontend development Explain data types, covering primitive data types (number, string, boolean, undefined, null) and complex data types (object, array, function). Explain control structures including conditional operations, loops, and ternary expressions. Discuss Javascript objects, arrays, and the use of JSON for basic data exchange.	W3schools.com, https://developer.mozilla.org , Textbooks, Relevant articles, and videos Relevant Documentation W3schools.com, https://developer.mozilla.org , Textbooks, Relevant articles, and videos Relevant Documentation W3schools.com, https://developer.mozilla.org	Create a program that works with the primitive data types in Javascript Implement decision-making using if-else and swich statements, discussing scenarios and conditions for branching logic. Create a program that stores values in JSON and iterates through them to output values Create a program demonstrating the advantages of using <i>let</i> for block-level scoping, helping to avoid common issues associated with <i>var</i> . Create reusable functions with parameters and return values, illustrating their	Guide students how to: Demonstrate type conversion and coercion in JavaScript, including explicit and implicit type conversions. Demonstrate extensive usage of different data types within JavaScript Guide students to:	

		Discuss their available APIs, Discuss the differences between the <i>var</i> and <i>let</i> keywords in JavaScript, emphasizing variable scope, hoisting, and block-level scope. Explain Functions and emphasize their significance in code organization, reusability, and maintainability.	.mozilla.org, Textbooks, Relevant articles, and videos Relevant Documentation W3schools.com, https://developer.mozilla.org, Textbooks, Relevant articles, and videos Relevant Documentation W3schools.com, https://developer.mozilla.org, Textbooks, Relevant articles, and videos Relevant Documentation	versatility in performing various tasks.	Demonstrate the use of loops (for and while) to perform iterative tasks, such as array iteration and data processing. Demonstrate the use of switch statements for multi-case branching, (making code more readable and efficient for certain conditions). Guide Students to: Use Arrays, Objects, and JSON in Javascript. Guide students to:	
--	--	--	--	---	--	--

					Refactor code to replace <i>var</i> with <i>let</i> for improved variable management, consistency, and avoidance of potential bugs. Guide students to: Use anonymous and callback functions for asynchronous operations, such as event handling and AJAX requests, showcasing real-world applications of JavaScript functions.	
--	--	--	--	--	--	--

GENERAL OBJECTIVE: 5.0 Understand the Document Object Model (DOM).						
13 - 15	5.1 Explain the the Document Object Model (DOM) and its critical role in web development. 5.2 Explain element selection, attribute modification, and content manipulation.	Discuss the DOM,emphasizing its representation of web page structure as a tree of objects. Explain element selection, attribute modification, and content manipulation.	W3schools.com, https://developer.mozilla.org Textbooks, Relevant articles, and videos Relevant Documentation	Create a program that traverses the DOM tree to target specific elements, their parents, siblings, and modify the content of any.	Guide students to: Demonstrate methods for accessing and manipulating HTML elements within the DOM tree using JavaScript getElementBy APIs and query selectors	5.1 Explain the the Document Object Model (DOM) and its critical role in web development. 5.2 Explain element selection, attribute modification, and content manipulation.

Assessment: Give details of assignments to be used:

Coursework/ Assignments %; Course test %; Practical %; Projects %; Examination %

Type of Assessment	Purpose and Nature of Assessment (SWD 323)	Weighting (%)
Examination	Final Examination (written) to assess knowledge and understanding	40
Test	At least 2 progress tests for feedback.	10

Practical	At least 5 hands-on to be assessed by the teacher	50
Total		100

PROGRAMME: HIGHER NATIONAL DIPLOMA SOFTWARE AND WEB DEVELOPMENT		
COURSE: Back End Development I	COURSE CODE: SWD324	CONTACT HOURS: 6hrs
	CREDIT UNIT: 3	THEORETICAL: 2
YEAR: I SEMESTER: II	PRE-REQUISITE:	PRACTICAL: 4
GOAL: This course provides students with knowledge and skills for Backend Development		
GENERAL OBJECTIVES: Upon completion of the course, students should be able to: 1. Understand Backend development and its functions. 2. Know how to set up a backend development environment. 3. Understand The basics of PHP, Data types, and Control Structure 4. Understand Functions, String Manipulation, and Arrays. 5. Know how to retrieve and clean Request DATA in GET, POST, and other Global variables. 6. Understand Basic Error Handling in PHP. 7. Understand Object Oriented Programming and its principles in PHP.		

PROGRAMME: HIGHERNATIONAL DIPLOMA SOFTWARE AND WEB DEVELOPMENT							
COURSE: Back End Development I			COURSE CODE:SWD324		CONTACT HOURS: 6hrs		
			CREDIT UNIT:3		THEORETICAL: 4		
YEAR: I SEMESTER: II			PRE-REQUISITE:		PRACTICAL:2		
COURSE SPECIFIFCATION: THEORETICAL AND PRACTICAL							
GOAL: This course provides students with knowledge and skills for Backend Development							
GENERAL OBJECTIVE 1.0: Understand Backend development and its functions.							
THEORETICAL CONTENT				PRACTICAL CONTENT			
Week	Specific Learning Outcome		Teacher’s Activities	Resources	Specific Learning Outcome	Teacher’s Activities	Evaluation
1	1.3	Explain Backend Development.	Discuss the reasons for the separation of the backend and front end.	Textbooks, Relevant Material, Articles, or Videos.			
	1.4	Explain their differences and how they work together	Explain their differences and how they work together.				
	1.5	List Benefits of Having Separate backend and frontend	Discuss the Benefits of a Separate Backend from the front end. (such as same APIs for mobile, wearables, and all devices)				

GENERAL OBJECTIVE: 2.0 Know how to set up a backend development environment.						
2	2.1 Explain the purpose of XAMPP or WAMP as local development environments for PHP and MySQL	Discuss XAMPP, LAMP, WAMPP, or any chosen PHP/Apache Server.	Videos, XAMPP/WAMP Or any Apache/PHP Server, Internet	Install and Configure development Server correctly.	Guide students to : Install and setup PHP Servers minimum version 8.2	
	2.2 Explain The Purpose and installation of POSTMAN or Similar Technology	Discuss the relevance of using POSTMAN or similar technology for testing API calls	POSTMAN	Install and Configure POSTMAN correctly	Demonstrate installation and familiarization with POSTMAN	
GENERAL OBJECTIVE: 3.0 Understand the basics of PHP, Data types, and Control Structure						
3-5	3.1 Explain PHP Basic Syntax 3.2 Explain primitive Datatypes, operators, and operator precedence in PHP 3.3 Describe variables in PHP and explain their role in storing and manipulating data. 3.4 Explain Statements in PHP 3.5 Explain Control Structure in PHP 3.6 List the Control Structure available in	Discuss the syntax of PHP Explain Primitive Datatypes, Operators, operator precedence, and Dynamic typing. Discuss variables in PHP.	Textbooks, https://php.net Relevant Articles, YouTube videos, or Material Textbooks, https://php.net Relevant Articles, YouTube videos, or	Demonstrate Variable assignment, data types, and dynamic typing examples of variable scope, including global and local scope. Demonstrate the use of the available control structure in PHP to solve basic problems.	Guide students to: Demonstrate Variable assignment, data types, and dynamic typing examples of variable scope, including global and local scope	

	PHP	Discuss statements in PHP. Discuss the three control Structures in PHP programming, Discuss the control structures syntax, and how to use them.	Material		Demonstrate selection, iteration, and sequential operations with basic examples	
GENERAL OBJECTIVE: 4.0 : Understand Functions, String Manipulation and Arrays.						
6-8	4.1 Explain Functions and Function Syntax 4.2 List types of Function 4.3 List and Explain Basic String Manipulation Functions in PHP 4.4 Explain Arrays 4.5 List various array functions and their uses.	Discuss functions, their syntax, uses, and importance in making code, maintainable, simple, and clean Explain types of functions Explain string manipulation functions like <i>strlen()</i> , <i>substr()</i> Discuss the array data Structure. Discuss Iteration	Textbooks, https://php.net Relevant Articles, YouTube videos, or Material Textbooks, https://php.net Relevant Articles, YouTube videos, or Material	Write custom functions, and utilize built-in functions(<i>substr()</i> , <i>implode()</i> , <i>explode()</i> , <i>strlen()</i>) to manipulate strings Create array variables, manipulate, and iterate through items of the array. Add, remove, and replace values in an array.	Guide Students to: create their custom functions, utilize the built-in function to manipulate strings, and carry out repetitive tasks Guide students to: create an array variable, add	

		through arrays, and various functions in array manipulations			items, remove items, and other array manipulations.	
GENERAL OBJECTIVE: 5.0 : Know how to retrieve and clean Request DATA in GET, POST, and other Global variables.						
9-11	5.1 Explain Requests and Responses 5.2 Explain Global variables. 5.2 List Different Global Variables 5.3 Explain \$_GET and \$_POST. 5.4 Explain how to pass parameters on URL via the postman. 5.5 Explain processes of cleaning user-collected Data	Discuss Requests and global variables like \$_REQUEST, \$_GET, \$_POST. Discuss the use of json_encode and json_decode functions to send a response to the user. Explain how to retrieve values from global variables in PHP Explain the use of the POST and GET global variables Discuss the process of using built-in functions to clean user data.	Textbooks, https://php.net Relevant Articles, YouTube videos, or Material Textbooks, https://php.net Relevant Articles, YouTube videos, or Material	Create a PHP script that will collect data from a user, do some basic manipulation and operations on the data, and return a response to the user. Retrieve data from the GET and POST variables. Write a code sample showing the cleaning of Data from untrusted sources passed as parameters to an endpoint	Guide Students to: Create a PHP script that will collect data from a user, do some basic manipulation and operations on the data, and return a response to the user. Retrieve data from the GET and POST variables. Guide students to: Write a code sample showing the	

		Explain how to avoid execution of dangerous data/scripts passed from a user			cleaning of Data from untrusted sources passed as parameters to an endpoint.	
GENERAL OBJECTIVE: 6.0 Understand basic error handling in PHP.						
12 -13	6..1 Explain errors in PHP, including Exceptions, syntax errors, runtime errors, and logical errors 6.2 List different types of errors 6.3 Show how to throw custom Exceptions in PHP 6.4 Explain Error Handling in PHP using try and catch to handle exceptions	Discuss errors in PHP, including syntax errors, runtime errors, and logical errors. Describe techniques and situations where custom exceptions can be <i>thrown</i> by the developer. Discuss the use of <i>try</i> and <i>catch</i> to handle exceptions for security and return human-readable messages to the user.	http://developers.mozilla.org . W3Schools.com Relevant YouTube Videos, Relevant Articles/Publications http://developers.mozilla.org . W3Schools.com Relevant YouTube Videos, Relevant Articles/Publications	Write programs showing various errors and identify their causes. Demonstrate error handling by using try, catch in PHP. Also, throw and handle custom exceptions that will show relevant errors to the user	Guide students to: Demonstrate the various types of errors and ways of avoiding them and interpreting the error messages. Demonstrate error handling by using <i>try</i> , <i>catch</i> in PHP. Also, <i>throw</i> and handle custom exceptions that will show relevant errors to the user	

GENERAL OBJECTIVE: 7.0 : Understand Object Oriented Programming and its principles in PHP.						
14-15	7.1 Discuss the concepts of OOP	Discuss the concept of classes representing real-life objects to solve problems.	http://developers.mozilla.org .	Develop a program that implements the principles of OOP. Creating an instance of a class that implements inheritance, abstraction, polymorphism, and Encapsulation	Guide students to: Develop a program that implements the principles of OOP. Creating an instance of a class that implements inheritance, abstraction, polymorphism, and Encapsulation	
	7.2 Define class and explain its structure		W3Schools.com Relevant YouTube Videos,			
	7.3 List and Explain the 4 principles of OOP	Discuss the concepts and principles of Abstraction, Inheritance, Encapsulation, and Polymorphism	Relevant Articles/Publications			

Assessment: Give details of assignments to be used:

Coursework/ Assignments %; Course test %; Practical %; Projects %; Examination %

Type of Assessment	Purpose and Nature of Assessment (SWD 324)	Weighting (%)
Examination	Final Examination (written) to assess knowledge and understanding	40

Test	At least 2 progress tests for feedback.	10
Practical	At least 5 hands-on to be assessed by the teacher	50
Total		100

PROGRAMME: HIGHER NATIONAL DIPLOMA SOFTWARE AND WEB DEVELOPMENT		
COURSE: Advanced Statistics for Computing	COURSE CODE: SWD325	CONTACT HOURS: 3hrs
	CREDIT UNIT: 2	THEORETICAL: 2
YEAR: I SEMESTER: II	PRE-REQUISITE:	PRACTICAL: 1
GOAL: This course is designed to produce students with advanced statistical knowledge for Software and Web Development.		
GENERAL OBJECTIVES: On completion of the course, the student should be able to: 1. Comprehend key statistical concepts and their significance in Software and Web Development 2. Utilise statistical methods like regression and ANOVA to address real-world development challenges. 3. Demonstrate proficiency with software tools, and their libraries for data analysis. 4. Analyse time series data for web analytics and software trend prediction. 5. Understand the link between statistics and machine learning, focusing on model performance. 6. Know ethical standards in statistical analysis, emphasizing data privacy and unbiased reporting. 7. Apply advanced statistical methods for complex challenges and deeper domain study. 8.		

PROGRAMME: HIGHERNATIONAL DIPLOMA SOFTWARE AND WEB DEVELOPMENT						
COURSE: Advanced Statistics for Computing		COURSE CODE:SWD325		CONTACT HOURS: 3hrs		
		CREDIT UNIT:2		THEORETICAL: 2		
YEAR: I SEMESTER: II		PRE-REQUISITE:		PRACTICAL:1		
COURSE SPECIFICATION: THEORETICAL AND PRACTICAL						
GOAL: This course is designed to produce students with advanced statistical knowledge for Software and Web Development.						
GENERAL OBJECTIVE 1.0: Comprehend key statistical concepts and their significance in Software and Web Development						
THEORETICAL CONTENT				PRACTICAL CONTENT		
Week	Specific Learning Outcome	Teacher’s Activities	Resources	Specific Learning Outcome	Teacher’s Activities	Evaluation
1-2	1. Define key statistical concepts. 2. Describe the theories behind major statistical methods. 3. Explain the relevance of statistical methods in software and web development.	Discuss the evolution of statistics in software development. Interpret the practical implications of statistical theories. Discuss how statistical methods have influenced software and web development over time.	PC, Projector, Whiteboard, Marker, Textbook chapters on statistical history. E-content on foundational statistical theories. Dataset of past and present software trends.	Develop a timeline on the evolution of statistical methods. Apply statistical theories to real-world software development scenarios. Design a mini project that demonstrates the application of statistics in software development.	Comprehend key statistical concepts and their significance in Software and Web Development Comprehend key statistical concepts and their significance in Software and Web Development Comprehend key statistical concepts and	Written test on the definition and history of statistical concepts. Oral presentation on a chosen statistical theory and its application. Project evaluation based on the correct application of statistical methods.

					their significance in Software and Web Development	
GENERAL OBJECTIVE: 2.0 Utilise statistical methods like regression and ANOVA to address real-world development challenges.						
3-4	1. Define regression analysis and its significance in software development. 2. Describe ANOVA and its applications in web development. 3. Explain non-parametric statistics and its advantages over parametric methods.	Discuss the historical context of regression analysis in software development. Interpret the differences between one-way and two-way ANOVA. Discuss key non-parametric tests like Mann-Whitney U Test and Kruskal-Wallis Test.	PC, Projector, Whiteboard, Marker, Textbook chapters on regression analysis; software like SPSS or R. E-content on ANOVA principles. Dataset of web user behaviors; e-content on non-parametric tests.	Develop a basic model using regression analysis for a given dataset. Apply ANOVA tests on sample web user data to derive insights. Design a study using non-parametric tests on a non-normal dataset.	Guide students to: Develop a basic model using regression analysis for a given dataset. Apply ANOVA tests on sample web user data to derive insights. Design a study using non-parametric tests on a non-normal dataset.	Written test on regression analysis definitions and its principles. Group project and presentation on applying ANOVA in web analytics. Practical examination involving data analysis using non-parametric methods.
GENERAL OBJECTIVE: 3.0 Demonstrate proficiency with software tools, and their libraries for data analysis						
5-7	1. Define the functionalities of statistical software like R, Python, and SQL. 2. Describe the purpose and usage of libraries such as pandas, matplotlib, and ggplot2.	Discuss the evolution and significance of these tools (R, Python, SQL) in data analytics.	PC, Projector, Whiteboard, Marker, Software installations: R,	Develop basic scripts in R/Python, and SQL for data manipulation. Apply pandas for data wrangling and use	Guide students to: Develop basic scripts in R/Python and	Written test on functions and utilities of R/Python and SQL.

	3. Explain the process of data visualization and its relevance in decision-making.	Interpret real-world applications where these libraries (pandas, matplotlib, and ggplot2) have made a significant impact. Discuss various data visualization techniques and their appropriate use-cases.	Python, SQL servers; e-content on software tutorials. Online datasets for practice; library documentation; online courses on platforms like Coursera. Visualization tools like Tableau; datasets related to web analytics.	matplotlib and ggplot2 for visualization on given datasets. Design a comprehensive dashboard depicting user behavior using the learned tools.	SQL for data manipulation. Apply pandas for data wrangling and use matplotlib and ggplot2 for visualization on given datasets. Design a comprehensive dashboard depicting user behavior using the learned tools.	Practical project requiring data cleanup using pandas and visualization using matplotlib/ggplot2. Evaluation based on a presentation of a dashboard and its insights.
GENERAL OBJECTIVE: 4.0 Analyse time series data for web analytics and software trend prediction						
8-9	1. Define time series data and its relevance in web analytics. 2. Describe patterns commonly found in time series data, such as seasonality and trends. 3. Explain techniques for forecasting future data points in a time series.	Discuss the origins and evolution of time series analysis in the computing world. Interpret real-world examples where pattern recognition has aided business decision-making. Discuss various forecasting models	PC, Projector, Whiteboard, Marker, Textbooks on Time Series Analysis; e-content on foundational concepts. Datasets exhibiting	Develop a time series plot for given web analytics data. Apply methods to decompose a time series into its various components. Design a forecasting model for predicting future web traffic.	Guide students to: Develop a time series plot for given web analytics data. Apply methods to decompose a time series into its various	Written test on definitions and fundamental concepts of time series data. Practical project evaluating the decomposition of a time series dataset. Evaluation based

		and their suitability for different scenarios.	different time series patterns; online tutorials. Software like R with "forecast" package; online courses on time series forecasting.		components. Design a forecasting model for predicting future web traffic.	on accuracy and reliability of the developed forecasting model.
GENERAL OBJECTIVE: 5.0 Understand the link between statistics and machine learning, focusing on model performance.						
10-11	1. Understand the foundational differences between statistics and machine learning. 2. Recognize the importance of statistical considerations in machine learning model development. 3. Understand the statistical methods used in model evaluation such as precision, recall, and F1 score.	Discuss the historical developments of both fields and their convergence. Interpret case studies where lack of statistical considerations led to model failures. Discuss the mathematical formulations of these metrics and their significance.	PC, Projector, Whiteboard, Marker, Textbooks on Machine Learning & Statistics; e-articles on their intersection. Online tutorials emphasizing statistical importance in ML; datasets with pitfalls. E-content on model evaluation; online courses	Develop a basic statistical model and a machine learning model for a given dataset. Evaluate the performance of a machine learning model by considering underlying statistical metrics. Apply these statistical methods to evaluate a machine learning model's predictions.	Guide students to: Develop a basic statistical model and a machine learning model for a given dataset. Evaluate the performance of a machine learning model by considering underlying statistical metrics. Apply these statistical	A written test comparing and contrasting statistical models and ML models. Practical assignment assessing model performance and understanding of metrics. Evaluation based on the correct application and interpretation of evaluation metrics.

			on model performance.		methods to evaluate a machine learning model's predictions.	
GENERAL OBJECTIVE: 6.0 Know ethical standards in statistical analysis, emphasizing data privacy and unbiased reporting.						
12-13	1. Recognize the importance of ethics in statistical analysis. 2. Understand the principles of data privacy, especially under GDPR. 3. Identify and prevent biases in statistical analysis.	Discuss real-world examples of ethical breaches in statistical practices. Interpret the key clauses of GDPR related to data analysis and storage. Discuss common biases in data collection and analysis, emphasizing their implications.	PC, Projector, Whiteboard, Marker, Case studies on statistical malpractices; e-content on ethical statistical analysis. NDPR documentation; online courses on NDPR and data privacy. E-articles on bias in statistics; datasets illustrating biased vs unbiased data.	Design a project ensuring ethical considerations in data collection and analysis. Apply NDPR principles in a mock data project. Evaluate a given dataset for potential biases and suggest corrections.	Guide students to: Design a project ensuring ethical considerations in data collection and analysis. Apply NDPR principles in a mock data project. Evaluate a given dataset for potential biases and suggest corrections.	Written assignment evaluating understanding of ethical concerns in statistical practices. Assessment of the mock project's adherence to GDPR principles. Evaluation based on bias identification and appropriate suggestions for corrections.
GENERAL OBJECTIVE: 7.0 Apply advanced statistical methods for complex challenges and deeper domain study.						

14-15	1. Understand the principles of Bayesian statistics. 2. Familiarize with Structural Equation Modeling (SEM) techniques. 3. Recognize the importance and methods of survival analysis. 4. Prepare for complex challenges in advanced statistical studies.	Discuss the foundational concepts of Bayesian thinking. Interpret the foundational and advanced SEM methods using examples. Discuss survival analysis techniques emphasizing their real-world applications. Contrast basic vs advanced statistical techniques for complex challenges.	PC, Projector, Whiteboard, Marker, Textbooks on Bayesian statistics; e-content on Bayesian vs frequentist approach. Software for SEM (e.g., AMOS, LISREL); e-articles and textbooks on SEM. E-books on survival analysis; datasets illustrating censoring and survival events. Advanced statistics textbooks; case studies highlighting complex statistical	Apply Bayesian methods on provided datasets. Develop a structural equation model using a given scenario. Design a survival analysis study based on hypothetical or real-world data. Evaluate complex datasets using advanced statistical methods.	Guide students to: Apply Bayesian inference on datasets. Develop a structural equation model using a given scenario. Design a survival analysis study based on hypothetical or real-world data. Evaluate complex datasets using advanced statistical methods.	Written test on Bayesian principles and their application. Evaluation of the constructed SEM for correctness and interpretation. Assessment based on the design, execution, and interpretation of the survival study. Comprehensive project assessment focusing on advanced methods and their application.
-------	--	---	---	---	--	--

			challenges.			
--	--	--	-------------	--	--	--

PROGRAMME: HIGHER NATIONAL DIPLOMA SOFTWARE AND WEB DEVELOPMENT		
COURSE: Research Methods in SWD	COURSE CODE: SWD326	CONTACT HOURS: 4hrs
	CREDIT UNIT: 3	THEORETICAL: 2
YEAR: I SEMESTER: II	PRE-REQUISITE:	PRACTICAL: 2
GOAL: This is designed to provide students with research methodologies for Software and Web Development.		
GENERAL OBJECTIVES: On completion of the course, the student should be able to: 1. Understand the design of research projects for software and web development. 2. Construct guiding research questions and objectives to expand field knowledge. 3. Understand the research's scope for clarity. 4. Analyse existing literature to spot trends and gaps in aresearch study. 5. Utilise relevant methodologies for effective research implementation 6. . Comprehend and conclude from practical results, suggesting future studies. 7. Demonstrate findings professionally, upholding APA 7th Edition and ethical standards		

PROGRAMME: HIGHERNATIONAL DIPLOMA SOFTWARE AND WEB DEVELOPMENT						
COURSE: Research Methods in SWD			COURSE CODE:SWD326		CONTACT HOURS: 4hrs	
			CREDIT UNIT:3		THEORETICAL: 2	
YEAR: I SEMESTER: II			PRE-REQUISITE:		PRACTICAL:2	
COURSE SPECIFIFCATION: THEORETICAL AND PRACTICAL						
GOAL: To provide students with research methodologies for Software and Web Development.						
GENERAL OBJECTIVE 1.0: Understand the design of research projects for software and web development.						
THEORETICAL CONTENT				PRACTICAL CONTENT		
Week	Specific Learning Outcome	Teacher’s Activities	Resources	Specific Learning Outcome	Teacher’s Activities	Evaluation
1-2	1.1 Explain a research project specific to software and web development. 1.2 Describe the formulation of a relevant title. 1.3 Explain the importance of a clear problem statement. 1.4 Describe the process of identifying research gaps.	Discuss the importance of having a clear research topic. Interpret previous research titles in the field. Discuss the components of an effective problem statement. Contrast well-identified gaps vs. vague gaps in research.	PC, Whiteboard, Projector, Markers, Textbooks on research methodology, Online research databases Examples of previous research titles, Research methodology reference guides Articles with strong problem statements, Research	Develop a draft title for a research project. Design a relevant title for a reaserch project. Create a problem statement for the research project. Evaluate existing literature to find gaps based on the research project.	Guide students to: Develop a draft title for a research project. Design a relevant title for a research project. Create a problem statement for the research project. Evaluate existing literature to	Written evaluation of the proposed research title. Peer review of the formulated titles. Oral defense of the problem statement. Presentation on identified research gaps.

			methodology textbooks Online databases to explore existing research, Journals related to software and web development		find gapbased on the research projects.	
GENERAL OBJECTIVE: 2.0 Construct guiding research questions and objectives to expand field knowledge						
3-4	2.1 Define research questions in the context of software and web development. 2.2 Describe the importance of clear aims in a research project. 2.3 Explain the function of objectives in guiding a research project.	Discuss the characteristics of good research questions. Interpret the aims from various research papers in the field. Discuss steps to derive objectives from the main aim.	PC, Whiteboard, Projector, Markers, Research methodology textbooks, Example research papers with notable questions Academic journals in software and web development, Research methodology workshops Online modules on research	Develop research questions for the reserach project. Create a clear aim for the research project. Design specific objectives that align with the aim of the research project.	Guide students to: Develop research questions for then research project. Create a clear aim for the research project. Design specific objectives that align with the aim of the research project.	Written assessment of the proposed research questions. Peer review of the clarity and relevance of the research aim. Presentation on how the objectives will guide the investigation.

			objectives formulation, Research methodology textbooks			
GENERAL OBJECTIVE: 3.0 Understand the research's scope for clarity.						
5-6	Define the scope of research in the context of software and web development. Describe how boundaries ensure research focus. Explain the potential pitfalls of not having a clear research scope.	Discuss the importance of having a defined scope in research. Interpret how different research papers set their scope and boundaries. Discuss the effects of scope creep on the research process.	PC, Whiteboard, Projector, Markers, Research methodology textbooks, Research scope guidelines from academic bodies Exemplary research papers with well-defined scopes, Online tutorials on scope setting Case studies of projects that suffered from scope creep, Research pitfalls articles	Develop a clear scope for the research project. Apply learned techniques to set boundaries for the research project Evaluate potential challenges they might face if they don't set clear boundaries.	Guide students to: Develop a clear scope for the research project. Apply learned techniques to set boundaries for the research project. Evaluate potential challenges they might face if they don't set clear boundaries.	Written assessment of the defined research scope. Oral presentation defending the chosen scope and boundaries. Peer review focusing on the clarity and appropriateness of the defined scope.
GENERAL OBJECTIVE: 4.0 Analyse existing literature to spot trends and gaps in a research study.						

7-8	Define what a systematic literature review is. Describe key trends and gaps in software and web development based on existing literature. Explain methodologies commonly employed in software and web development research.	Discuss the purpose and importance of systematic literature reviews in research. Interpret key findings from exemplary literature reviews in the field. Discuss various research methodologies and their applicability to different research questions.	PC, Whiteboard, Projector, Markers, Textbooks on research methods, Articles explaining systematic literature reviews Published literature reviews in software and web development, - Journal databases like Google Scholar, IEEE Xplore Research methodology books tailored to software and web development, Methodology webinars, online courses	Develop a plan for conducting a systematic literature review. Apply techniques to identify and annotate key trends and gaps in the chosen research topic. Create a summarized list of methodologies pertinent to the research topic, justifying their relevance.	Guide students to: Develop a plan for conducting a systematic literature review. Apply techniques to identify and annotate key trends and gaps in the chosen research topic. Create a summarized list of methodologies pertinent to the research topic, justifying their relevance.	Submission of a systematic review plan. Oral presentation highlighting key literature findings. Written submission of chosen methodologies with justifications.
-----	--	--	--	---	---	--

GENERAL OBJECTIVE: 5.0 : Utilise relevant methodologies for effective research implementation.						
9-10	Define the various research methodologies suitable for software and web development. Describe the transition process from design phase to implementation. Explain the importance of prototyping, coding, and the build process in research.	Discuss the range of methodologies and their implications for software and web research. Interpret case studies showing the transition from research design to implementation in software projects. Discuss the stages of prototyping, coding, and building, highlighting best practices.	PC, Whiteboard, Projector, Markers, Methodology textbooks, Online courses on research methodologies Case study compendiums, Prototyping tools and software Coding platforms like GitHub, Prototyping tools like Figma, Development environments	Develop a research plan incorporating the chosen methodology. Apply the principles learned to transition from the research design to implementation. Create a working prototype and transition to coding and building processes.	Guide students to: Develop a research plan incorporating the chosen methodology. Apply the principles learned to transition from the research design to implementation. Create a working prototype and transition to coding and building processes.	Submission of a detailed research plan. Demonstration of a working prototype or early software build. Assessment of the prototype and code quality; feedback on the build process.
GENERAL OBJECTIVE: 6.0 Comprehend and conclude from practical results, suggesting future studies.						
11-12	Define the key principles of data analysis in the context of software and web development research. Describe methods for drawing valid	Discuss methodologies and techniques used in data analysis for	Data analysis software such as Statistica, SPSS, R, Journals and	Develop a structured approach to analyze the results of the practical work.	Develop a structured approach to analyze the	Written test on data analysis principles and techniques.

	conclusions from analyzed data. Explain how to derive recommendations for further studies based on research conclusions.	software and web development. Interpret examples of conclusions drawn from various software and web development research projects. Discuss the process of deriving recommendations from research findings.	articles on data analysis techniques Case studies of research projects, Textbooks on research conclusions and recommendations Articles detailing further studies based on past research, Workshops on recommendation derivation	Evaluate the outcomes of their analysis to draw meaningful conclusions. Design a set of actionable recommendations for further studies in the software and web development domain.	results of the practical work. Evaluate the outcomes of their analysis to draw meaningful conclusions. Design a set of actionable recommendations for further studies in the software and web development domain.	Submission and review of a conclusion section for their project. Presentation and defense of recommendations in front of a panel or class.
GENERAL OBJECTIVE: 7.0 Demonstrate findings professionally, upholding APA 7th Edition and ethical standards						
13-15	Define the standards and guidelines of the APA 7th Edition for research documentation. Describe the ethical considerations essential in the reporting of research findings. Explain the importance of presenting research findings in a professional manner.	Discuss the evolution and significance of the APA guidelines in professional research. Interpret real-life cases where ethical considerations were overlooked in research reporting.	PC, Whiteboard, Projector, Markers, APA 7th Edition Manual, Workshops or webinars on APA formatting Case studies of unethical	Develop a research document adhering strictly to the APA 7th Edition guidelines. Apply ethical considerations in documenting and presenting their research findings. Design a presentation of	Guide students to: Develop a research document adhering strictly to the APA 7th Edition guidelines. Apply ethical	Submission of a research paper or section, checked for APA adherence. Ethical review of students' research documentation. Presentation of research findings

		Discuss best practices in compiling, documenting, and presenting research findings.	research reporting, Ethical guidelines from professional bodies Tutorials on professional research presentation, Software tools for professional documentation	the research findings that reflects a high degree of professionalism.	considerations in documenting and presenting their research findings. Design a presentation of the research findings that reflects a high degree of professionalism.	in front of a panel or class.
--	--	---	---	---	---	-------------------------------

YEAR II SEMESTER I

PROGRAMME: HIGHER NATIONAL DIPLOMA SOFTWARE AND WEB DEVELOPMENT		
COURSE: Project Management in Software and Web Development	COURSE CODE: SWD 411	CONTACT HOURS: 3hrs
	CREDIT UNIT: 2	THEORETICAL: 2
YEAR: II SEMESTER: I	PRE-REQUISITE:	PRACTICAL: 1
GOAL: This is designed to provide students with essential project management knowledge and skills for software and web development.		
GENERAL OBJECTIVES: On completion of the course, the student should be able to: 1. Understand the essence of project management in software and web development. 2. Know project needs, objectives, and stakeholder communication essentials. 3. Utilise Project Management tools for resource, risk, and task management. 4. Analyse the Agile and Waterfall methodologies and their application to projects. 5. Comprehend the integration of Software Development Life Cycle (SDLC) with project management. 6. Demonstrate methods for budgeting, testing, and quality assurance in projects. 7. Utilise Microsoft Project for effective project management and monitoring		

PROGRAMME: HIGHERNATIONAL DIPLOMA SOFTWARE AND WEB DEVELOPMENT						
COURSE: Project Management in SWD			COURSE CODE:SWD411		CONTACT HOURS: 3	
			CREDIT UNIT:2		THEORETICAL: 2	
YEAR: II SEMESTER: I			PRE-REQUISITE:		PRACTICAL:1	
COURSE SPECIFIFCATION: THEORETICAL AND PRACTICAL						
GOAL: To provide students with essential project management knowledge and skills for software and web development, using industry-standard tools and methods.						
GENERAL OBJECTIVE 1.0: Understand the essence of project management in software and web development.						
THEORETICAL CONTENT				PRACTICAL CONTENT		
Week	Specific Learning Outcome	Teacher’s Activities	Resources	Specific Learning Outcome	Teacher’s Activities	Evaluation
1-2	1.1 Explain the fundamental principles of project management in software and web development. 1.2 Define project plan and its components 1.3 Explain the significance of project management in ensuring successful software and web development projects. 1.4 Explain different methodologies and their applicability in various software and web development contexts. Explain the difference between traditional and modern project management techniques and tools.	Discuss the importance and evolution of project management in the IT sector. Explain project plan and its components Interpret case studies where project management played a pivotal role in the success or failure of IT projects.	PC, Projector, Whiteboard, Marker, Project Management Books, e-content, PPTs Case study materials, Videos Charts, Comparative tables Software demos, Comparative analysis papers	Develop a basic project plan for a hypothetical software/web development project. Apply project management principles to simulate real-world scenarios. Participate in group discussions to share findings and insights. Evaluate the efficiency of different project management methodologies in varied	Guide students to: Develop a basic project plan for a hypothetical software/web development project Apply project management principles to simulate real-world scenarios	Written test on theoretical concepts of project management. Practical assessment based on the application of project management principles. Group discussion grading based on understanding and insights. Project

		Discuss different methodologies and their applicability in various software and web development contexts. Contrast traditional and modern project management techniques and tools.		scenarios. Select appropriate tools and strategies for project management. Design a project management strategy for software/web development project. Create a detailed project management plan incorporating all learned principles and tools.	Facilitate group discussions to allow students share their findings and insights. Evaluate the efficiency of different project management methodologies in varied scenarios. Select appropriate tools and strategies for project management. Design a project management strategy for software/web development project Create a	presentation and assessment of the designed strategy Continuous assessment and feedback on project plan drafts.
--	--	--	--	---	--	---

					detailed project management plan incorporating all learned principles and tools.	
GENERAL OBJECTIVE: 2.0 Know project needs, objectives, and stakeholder communication essentials.						
3-4	2.1 Identify the various needs of a project in the realm of software and web development. 2.2 Explain the significance of setting clear project objectives. 2.3 Explain the essentials of effective communication with stakeholders in a project. 2.4 Describe the role and interests of various stakeholders in a project.	Discuss the importance of clear project needs definition in ensuring project success. Interpret real-life case studies highlighting the implications of well-defined vs. poorly-defined project objectives. Discuss various communication methods and their efficacy in different project scenarios. Contrast the interests and roles of different stakeholders such as developers, clients, end-users, and managers.	PC, Projector, Whiteboard, Marker, Project Management Books, e-content, PPTs Jira, Microsoft project management software, Trello App, Zoho Project, ProofHub, Gantt Chart, OpenProject, Videos Communication tools, Comparative tables Stakeholder role descriptions, Flowcharts	Develop a detailed needs assessment mapping out the project requirements and dependencies for a hypothetical software/web development project. Apply the principles of objective setting to define clear and measurable goals for a project. Design a comprehensive communication plan tailored to a specific project and its stakeholders. Evaluate the potential impacts of decisions on different stakeholder groups through simulated scenarios. Create stakeholder-specific communication strategies that address individual	Guide students to : Develop a detailed needs assessment mapping out the project requirements and dependencies for a hypothetical software/web development project. Apply the principles of objective setting to define clear, measurable goals following the	Written test on project needs identification concepts. Group activity or quiz based on setting project objectives. Practical assessment based on the robustness and clarity of the communication plan. Role-play or simulation grading based on stakeholder management skills. Continuous assessment and feedback on

				needs and concerns.	SMART (Specific, Measurable, Achievable, Relevant, Time-bound) principle. Facilitate group discussions on best practices in stakeholders' communication. Assess the nuances of stakeholders' management and the implications of their decisions. Create stakeholder-specific communication strategies that address individual needs and concerns.	stakeholder communication strategies.
--	--	--	--	---------------------	---	---------------------------------------

GENERAL OBJECTIVE: 3.0 Utilise Project Management tools for resource, risk, and task management.						
5-6	3.1 Explain the importance of clearly defined project scopes in software and web development. 3.2 Explain the value of efficient resource allocation in project management. 3.3 Explain the essentials of risk management in software and web development projects. 3.4 Explain the importance and usage of Gantt charts in project scheduling.	Discuss the foundational elements of project scope and its significance. Interpret case studies to highlight the benefits and challenges of resource allocation. Discuss various risk identification, assessment, and mitigation techniques. Discuss the structure and utility of Gantt charts in project management.	PC, Projector, Whiteboard, Marker, Project Management Books, e-content, PPTs Case studies, Videos, Charts Risk management tools such as Risk Management Software , Risk Registers, Monte Carlo Simulation, Heat Maps, HAZOP, and Comparative tables Microsoft Project, Primavera, Gantt chart software demonstrations, Examples	Develop a project scope statement for a hypothetical software project. Apply resource allocation techniques to a simulated project scenario. Design a risk management plan, identifying potential risks and mitigation strategies. Create a Gantt chart for a hypothetical project, mapping out task durations and dependencies.	Guide students to: Develop a project scope statement for a hypothetical software project . Apply resource allocation techniques to a simulated project scenario. Design a risk management plan, identifying potential risks and mitigation strategies . Participate in group discussions on risk scenarios and best practices. Create a Gantt	Written test on defining project scopes. Practical assessment based on resource allocation effectiveness. Group activity or quiz based on risk management planning. Evaluation based on the clarity and accuracy of Gantt charts produced.

					chart for a hypothetical project, mapping out task durations and dependencies.	
GENERAL OBJECTIVE: 4.0 Analyse the Agile and Waterfall methodologies and their application to projects.						
7-9	4.1 Explain the core principles and foundations of the Agile methodology. 4.2 Describe the sequential phases and structure of the Waterfall methodology. 4.3 Explain the benefits and challenges of both methodologies. 4.4 Compare and contrast Agile vs. Waterfall.	Discuss the iterative nature, roles, and ceremonies of Agile. Discuss the linear and phased approach of Waterfall. Interpret case studies highlighting successes and failures of each methodology. Contrast the methodologies in terms of flexibility, phases, feedback loops, and suitability	PC, Projector, Whiteboard, Marker, Agile Manifesto, Scrum Guide, e-content, PPTs Waterfall methodology charts, e-content, Diagrams Case studies, Real-world examples, Videos Comparative tables, Venn diagrams	Develop an Agile product backlog and sprint backlog for a hypothetical software project. Design a Waterfall project plan, highlighting each phase's deliverables. Evaluate a given project scenario to determine if Agile or Waterfall is more suitable. Create a presentation comparing Agile and Waterfall, providing recommendations for a given project type.	Guide students to : Develop an Agile product backlog and sprint backlog for a hypothetical software project. Map out a complete Waterfall lifecycle. Determine if Agile or Waterfall is more suitable for a given project	Written test on Agile principles and practices. Practical assessment based on the clarity and structure of the Waterfall plan. Group debate or quiz based on methodology selection for given scenarios. Presentation evaluation based on depth of comparison and suitability of recommendations

					scenario. Participate in group discussions on the merits and demerits of each methodology for various projects. Gather data and articulate their comparisons	
GENERAL OBJECTIVE: 5.0Comprehend the integration of SDLC with project management						
10-11	5.1 Explain the fundamental principles and stages of the Software Development Life Cycle (SDLC). 5.2 Outline the role of project management within SDLC. 5.3 Explain the challenges and pitfalls in aligning SDLC with project management. 5.4 Describe how various SDLC models (like Waterfall, Agile) fit within project management. 5.5 Explain the importance of aligning development practices with project	Discuss the various phases of SDLC and their significance. Discuss how project management practices align with each SDLC phase. Interpret real-world case studies illustrating challenges in SDLC integration with PM.	PC, Projector, Whiteboard, Marker, SDLC diagrams, e-books, PPTs Case studies, Project management tools demos, Videos Real-world examples, Case	Design a flowchart representing the SDLC for a given scenario. Develop a project plan integrating SDLC phases and project management techniques. Evaluate a provided project case, identifying where SDLC and PM integration failed. Create a presentation	Guide students to: Design a flowchart representing the SDLC for a given scenario. Identify key phases and milestones.	Written test on SDLC phases and their functions. Group presentation on the integrated project plan. Assessment based on the depth of analysis in identifying failures.

	objectives.	Contrast the integration of different SDLC models with project management practices. Discuss how project management ensures that SDLC aligns with business and project goals.	study materials Comparative charts, Industry reports, Model diagrams Industry articles, Expert talks, Videos	comparing the integration of various SDLC models with PM. Apply PM techniques to ensure an SDLC model aligns with a given project's objectives.	Develop a project plan integrating SDLC phases and project management techniques. Participate in group discussions on integrating PM with SDLC. Dissect provided project case studies and draw conclusions where PM and SDLC integration failed. Gather data on the integration of various SDLC models with PM and craft their comparisons. Apply PM	Presentation evaluation based on clarity and depth of comparison. Practical assessment of a mock project's alignment with its objectives.
--	-------------	---	---	---	---	---

					techniques to ensure an SDLC model aligns with a given project's objectives.	
GENERAL OBJECTIVE: 6.0 Demonstrate methods for budgeting, testing, and quality assurance in projects.						
12-13	6.1 Describe the techniques for project cost estimation. 6.2 Explain the importance of budget monitoring. 6.3 Explain the principles of software testing. 6.4 Explain the role of quality assurance in project management. 6.5 Explain the significance of delivering high-quality deliverables.	Discuss various models and tools for cost estimation. Interpret real-world cases where budget monitoring led to project success or failure. Discuss the various software testing methodologies and their significance. Discuss the differences between quality assurance and quality control. Discuss the repercussions of poor quality deliverables on project outcomes.	PC, Projector, Whiteboard, Marker, E-books on cost estimation, Cost estimation tools Real-world case studies, Articles Videos on software testing, Testing tools demos Quality assurance guidelines, Charts, e-books Expert talks, Industry reports	Apply a chosen model to estimate costs for a given project. Design a budget monitoring process for a mock project. Develop a test plan for a given software scenario. Evaluate the quality assurance processes of a given project case. Create a quality checklist for ensuring top-notch project deliverables.	Guide students to: Select the right model based on project type. Apply a chosen model to estimate costs for a given project. Design a budget monitoring process for a mock project. Participate in group discussions on budget	Written test on various cost estimation models. Presentation on the designed budget monitoring process. Practical assessment on the comprehensiveness of the test plan. Group discussions and written evaluations on the analyzed cases. Peer review and assessment of the crafted quality checklist.

					challenges and resolutions. Create comprehensive test scenarios. Develop a test plan for a given software scenario Evaluate the quality assurance processes of a given project case. Incorporate industry best practices in the checklist.	
GENERAL OBJECTIVE: 7.0 Utilise Microsoft Project for effective project management and monitoring.						
14-15	7.1 Explain the key features and benefits of Project Software tailored for research projects. 7.2 Describe the process of setting up a research project in Project software such as Microsoft Project, Primavera. 7.3 Explain the concept of task management and its importance in research projects.	Discuss the capabilities of Project Software such as Microsoft Project, Primavera in the context of research project management. Discuss the steps	PC, Projector, Whiteboard, Marker, Microsoft Project, Primavera, E-content on Microsoft	Install, configure and navigate through project software such as Microsoft Project, Primavera in creating a mock research project. Design a project setup for a given research scenario	Guide students to: Install, configure and navigate through Project software such as	Written test on the functionalities of Project software. Practical assessment on project setup. Evaluation of the created task list

	7.4 Describe the process of resource allocation and budgeting in Project software such as Microsoft Project, Primavera. 7.5 Explain monitoring and reporting of research progress using Project software such as Microsoft Project, Primavera. 7.6 Explain the advanced features and customization capabilities of Project software such as Microsoft Project, Primavera for research	involved in initializing a project, setting calendars, and defining working hours. Discuss various task management features including defining tasks, milestones, and understanding dependencies. Discuss the methodology of assigning human and material resources, and budget estimation. Contrast different monitoring tools within Project software such as Microsoft Project, Primavera, Gantt charts and critical paths. Discuss the tailoring of Microsoft Project with custom fields, templates, and integration with	Project overview Step-by-step guides, Video tutorials Dataset of a mock research project, Interactive e-content Microsoft Project e-guides, Financial datasets Contrast different monitoring tools within Project software such as Microsoft Project, Primavera, Gantt charts and critical paths. Discuss the tailoring of Microsoft Project with custom fields, templates, and integration with	using Project software such as Microsoft Project, Primavera. Develop a comprehensive task list with dependencies and milestones for a mock project. Evaluate a given project's resource allocation and budgeting in Project software such as Microsoft Project and Primavera. Create a progress report of a mock research project using Project software such as Microsoft Project, Primavera. Design a customized research project template in Project software such as Microsoft Project, Primavera.	Microsoft Project, Primavera. Design a project setup for a given research scenario using Project software such as Microsoft Project, Primavera. Develop a comprehensive task list with dependencies and milestones for a mock project. Evaluate a given project's resource allocation and budgeting in Project software such as Microsoft Project and Primavera.	and milestones. Group discussions and peer review on resource allocation and budgeting techniques. Assessment of the generated progress reports. Practical evaluation of the tailored templates and features
--	--	--	---	---	--	--

		other Microsoft Office tools.	other Microsoft Office tools.		Create a progress report of a mock research project using Project software such as Microsoft Project, Primavera Participate in group discussions on interpreting and optimizing project progress reports. Design a customized research project template in Project software such as Microsoft Project, Primavera . Explore and	
--	--	-------------------------------	-------------------------------	--	--	--

					utilize advanced features.	
--	--	--	--	--	----------------------------------	--

PROGRAMME: HIGHER NATIONAL DIPLOMA SOFTWARE AND WEB DEVELOPMENT		
COURSE: Embedded System Development	COURSE CODE: SWD412	CONTACT HOURS: 5hrs
	CREDIT UNIT: 3	THEORETICAL: 2
YEAR: II SEMESTER: I	PRE-REQUISITE:	PRACTICAL: 3
GOAL: This course provides the students with knowledge and Skills needed to Design and Develop Embedded Systems		
GENERAL OBJECTIVES: On completion of this course the student should be able to: 1.0 Understand the Fundamentals of Embedded Systems Development 2.0 Know Elements of Embedded Systems Development 3.0 Comprehend Embedded Systems Design Process 4.0 Know Embedded Systems Communication Interface 5.0 Comprehend Embedded Firmware Development 6.0 Understand Real-time operating system(RTOS) Based Embedded System Development		

PROGRAMME: HIGHERNATIONAL DIPLOMA SOFTWARE AND WEB DEVELOPMENT						
COURSE: Embedded System Development		COURSE CODE:SWD412		CONTACT HOURS: 5hrs		
		CREDIT UNIT:3		THEORETICAL: 2		
YEAR: II SEMESTER: I		PRE-REQUISITE:		PRACTICAL:3		
COURSE SPECIFIFCATION: THEORETICAL AND PRACTICAL						
GOAL: This course provides the students with knowledge and Skills needed to Design and Develop Embedded Systems						
GENERAL OBJECTIVE 1.0: Understand the Fundamentals of Embedded Systems Development						
THEORETICAL CONTENT				PRACTICAL CONTENT		
Week	Specific Learning Outcome	Teacher’s Activities	Resources	Specific Learning Outcome	Teacher’s Activities	Evaluation
1-2	1.1 Define Embedded System and give examples 1.2 Explain the Purpose of Embedded Systems 1.3 Explain the Characteristics of Embedded Systems 1.4 Explain the Application Areas of Embedded Systems	Discuss the concept of Embedded System with examples. Discuss the Purpose of Embedded Systems Explain the Characteristics of Embedded Systems Discuss the Application Areas of Embedded Systems,	White Board. PC and multimedia Projector Online lecture notes			Explain the Purpose of Embedded Systems Explain the Characteristics of Embedded Systems Explain Application Areas of Embedded Systems

	1.5 Explain the different classes of Embedded Systems	such as : consumer electronics, Telecom, Banking, healthcare etc. Discuss the classes of Embedded systems: based on generations, triggering, deterministic behaviour etc.				
GENERAL OBJECTIVE: 2.0 Know Elements of Embedded Systems Development						
3- 5	2.1 Explain the different elements of Embedded Systems 2.2 Explain the types of Processors for Embedded Systems Design	Discuss the different elements of Embedded Systems: embedded processors, memory, embedded firmware, sensors, actuators and communication Interface Explain the types of processors for Embedded Systems Design: microprocessor or a microcontroller or digital signal processor.	White Board and marker. PC and multimedia Projector Embedded development boards e.g. Arduino, Raspberry Pi, BBC micro:bit, PyBoard, <u>ESP8266</u> -based boards etc. Embedded	Identify the different elements of embedded systems Connect the different components to an embedded development board Identify the components: ASIC, ASIP, LED, COTS, PLD and CPLD Connect the components to an embedded development	Guide students to: Identify the different elements of embedded systems Connect the different components to an embedded development board Identify the components:	3- 5

	2.3 Describe Embedded Development boards 2.4 Explain the usefulness of the following components: ASIC, ASIP, LED, COTS, PLD and CPLD	Explain Embedded development boards: Arduino, Raspberry Pi etc. Discuss the usefulness of the following components: ASIC, ASIP, LED, COTS, PLD and CPLD	processors, memory, sensors, actuators, LEDs, LCD, Buzzers, push button, switches etc. White Board and marker. PC and multimedia Projector Embedded development boards e.g. Arduino Embedded components: LED, ASIC, ASIP, PLD, COTS, and CPLD	board	ASIC, ASIP, LED, COTS, PLD and CPLD Connect the components to an embedded development board	
GENERAL OBJECTIVE: 3.0 Comprehend Embedded Systems Design Process						
6-7	3.1 Define Embedded System Design Process 3.2 Explain the stages in Embedded Systems Design	Explain Embedded System Design Process	White Board and marker. PC and	i) Write requirement specifications for some selected embedded systems	Guide students to: (i) Write requirement	

	Process	Discuss the stages in Embedded Systems Design Process: requirements Analysis, specifications, Architecture design, system integration etc.	multimedia Projector Online lecture notes	ii) Draw architectural design for the embedded systems specified in (i)	specifications for some selected embedded systems Draw architectural design for the embedded systems specified in (i)	
GENERAL OBJECTIVE: 4.0 Know Embedded Systems Communication Interface						
8-9	4.1 Explain Embedded Communication Interface 4.2 Explain the types of communication Interfaces 4.3 Explain the categories of each type	Explain Embedded Communication Interface Discuss the types of communication Interfaces: Onboard & External Communication Interfaces Discuss the categories of each type of communication interfaces:	White Board and marker. PC and multimedia Projector Wired and wireless communication interfaces	Connect embedded system to different types of wireless interfaces	Guide students to connect embedded system to different types of wireless interfaces	

		1-wireless interface, parallel interface; wired and wireless interfaces – ZigBee, GPRS, Wi-Fi etc.				
GENERAL OBJECTIVE: 5.0 Comprehend Embedded Firmware Development						
10 - 12	5.1 Explain Embedded Firmware and its importance 5.2 List Embedded development languages 5.3 Explain Embedded Firmware Development Approaches	Discuss Embedded Firmware and its importance Explain Embedded development languages Explain Embedded Firmware Development Approaches: Super Loop and Embedded Operating Systems based approaches	White Board and marker. PC and multimedia Projector White Board and marker. PC and multimedia Projector Embedded development boards e.g. Arduino, Raspberry Pi, etc. Embedded programming languages e.g.	Develop firmware for specified embedded systems using Super Loop Approach	Guide students to develop firmware for specified embedded systems using Super Loop Approach	Explain importance of embedded firmware Explain approaches to developing embedded firmware

			C++, Java, Python			
GENERAL OBJECTIVE: 6.0 Understand RTOS Based Embedded System Development						
13 - 15	6.1 Explain Real Time Operating System (RTOS) 6.2 List examples of RTOS 6.3 Explain the Functions of RTOS 6.4 Explain factors that influence the choice of RTOS for Embedded Development	Discuss Real Time Operating System (RTOS) Discuss examples of RTOS: Windows CE, VxWorks, MicroC/OS-II, Symbian, Embedded Linux, QNix etc. Discuss the functions of RTOS: Task management, memory management, time management etc. Discuss factors that influence the choice of RTOS for Embedded Development	White Board and marker. PC and multimedia Projector RTOS e.g.s. Windows CE, Symbian, Embedded Linux, QNix etc. Wired and wireless communication interfaces	Develop firmware for specified embedded systems using Embedded Operating Systems based approach	Guide students to develop firmware for specific embedded systems using Embedded Operating Systems based approach	Explain the functions of RTOS Explain factors that influence choice of RTOS for embedded development Demonstrate how to develop firmware using Embedded Operating Systems based approach

Assessment: Give details of assignments to be used:

Coursework/ Assignments %; Course test %; Practical %; Projects %; Examination %

Type of Assessment	Purpose and Nature of Assessment (SWD 412)	Weighting (%)
Examination	Final Examination (written) to assess knowledge and understanding	40
Test	At least 2 progress tests for feedback.	10
Practical	At least 5 hands-on to be assessed by the teacher	50
Total		100

PROGRAMME: HIGHER NATIONAL DIPLOMA SOFTWARE AND WEB DEVELOPMENT		
COURSE: Front End Development II	COURSE CODE: SWD413	CONTACT HOURS: 6hrs
	CREDIT UNIT: 3	THEORETICAL: 2
YEAR: II SEMESTER: I	PRE-REQUISITE: SWD323	PRACTICAL: 4
GOAL: This course is designed to provides students with Advanced knowledge and skills in frontend development		
GENERAL OBJECTIVES: Upon completion of this course, students should be able to: 1. Know how to Manipulate Document Object Model. 2. Understand Events and Event Handling 3. Comprehend the basics of Error Handling and Logging. 4. Know how to utilise Backend APIs in a Frontend application 5. Know the various frontend libraries and Frameworks		

PROGRAMME: HIGHERNATIONAL DIPLOMA SOFTWARE AND WEB DEVELOPMENT						
COURSE:Front End Development II			COURSE CODE:SWD413		CONTACT HOURS:6hrs	
			CREDIT UNIT:3		THEORETICAL: 2	
YEAR: II SEMESTER: I			PRE-REQUISITE:SWD323		PRACTICAL:4	
COURSE SPECIFIFCATION: THEORETICAL AND PRACTICAL						
GOAL: This course is designed to provides students with Advanced knowledge and skills in frontend development						
GENERAL OBJECTIVE 1.0: Know how to Manipulate The Document Object Model.						
THEORETICAL CONTENT				PRACTICAL CONTENT		
Week	Specific Learning Outcome	Teacher’s Activities	Resources	Specific Learning Outcome	Teacher’s Activities	Evaluation
1-2	1.1 Explain the Document Object Model (DOM) and its role in representing the structure of web documents as a tree of objects.	Discuss the tree structure of the document object model.	http://developers .mozillla.org. W3Schools.com	Utilize DOM traversal methods to navigate the DOM tree, targeting parent, child, and sibling elements Create a webpage and manipulate the contents using the methods explained.	Guide the students to demonstratedy namic changes to web content, such as modifying text, attributes, and styles of elements. Guide students to: Use	
	1.2 Explain inheritance and relationship in DOM.	Discuss inheritance, and relationship in DOM.	Youtube Videos, Relevant Articles/Publicat ions			
	1.3 Describe the use of	Discuss the various methods of getting access to elements in a DOM	http://developers .mozillla.org. W3Schools.com			
	i) getElementById ii) getElementsByClassName, iii) querySelector iv) createElement					

		Discuss creating Elements and adding them to the DOM.	Relevant Youtube Videos, Relevant Articles/Publications		<i>createElement</i> , <i>getElementsByClassName</i> , <i>getElementById</i> , and <i>querySelector</i> methods to select multiple elements based on their shared class attribute, id, or specified selector.	
General Objective 2.0 : Understanding Events and Event Handling						
3 -5	2.1 Explain events in the context of web development, 2.2 List the various events including user interactions like clicks, mouse movements, and keyboard input. 2.3 Explain Event Handling, 2.4 EventListeners, Callback functions, and preventDefault in Event Handling	Discuss the concept of event propagation (capturing and bubbling phases). Discuss common DOM events and their triggers, such as click, mouseover, key down, hover, and so on Discuss the use of event listeners, and callback functions in event handling, showing the use of	http://developers.mozilla.org . W3Schools.com Relevant Youtube Videos, Relevant Articles/Publications http://developers.mozilla.org . W3Schools.com Relevant Youtube Videos,	Create a webpage that responds to user interaction/event Attach event listeners to HTML elements using methods like <i>addEventListener</i> . Write event handler functions to execute desired behavior when events occur (such as form submission or button clicks.)	Guide students to: Demonstrate events and event propagation to create interactive web applications with responsive user interfaces, (showing alerts, prompts, and console logs). Guide students	

v)

		<i>preventDefault</i> to stop event propagation.			to: Demonstrate events triggering, handling and stopping the propagation when needed using <i>callback</i> functions and <i>preventDefault()</i>	
GENERAL OBJECTIVE: 3.0 Comprehend the basics of Error Handling and Logging						
6 - 8	3..1 Explain errors in JavaScript, including Exceptions, syntax errors, runtime errors, and logical errors 3.2 List different types of errors 3.3 Explain Error Handling in Javascript using try and catch to handle exceptions	Discuss errors in JavaScript, including syntax errors, runtime errors, and logical errors. Discuss the use of <i>try</i> , and <i>catch</i> to handle exceptions for security and return human-readable messages to the user.	http://developers.mozilla.org . W3Schools.com Relevant Youtube Videos, Relevant Articles/Publications http://developers.mozilla.org . W3Schools.com Relevant Youtube Videos, Relevant Articles/Publications	Create documents showing various errors and identify their causes. Show error handling using <i>try</i> and <i>catch</i> in JavaScript	Guide students to: Demonstrate the various types of error, ways of avoiding them and interpreting the error messages Demonstrate the use of <i>try</i> , <i>catch</i> , and <i>finally</i> in javascript error handling	

GENERAL OBJECTIVE: 4.0 Know how to consume Backend APIs in a Frontend application

9 -14	4.1 Define Javascript Object Notation (JSON) 4.2 Explain JSON Serialization 4.3 Explain Request Methods 4.4 List the Various Request Methods 4.5 Explain API calls 4.6 Describe the use of fetch function in javascript 4.7 Define Representational State Transfer (REST) and its principles in designing web services.	Discuss JSON and its uses in data interchange between platforms Explain Request methods and the various types(such as GET, POST, PUT, PATCH and DELETE) Explain the <i>fetch</i> API as a modern and efficient method for making network requests in JavaScript. Discuss the concept of RESTful APIs as a means of interacting with remote servers over HTTP.	http://developers.mozilla.org. http://developers.mozilla.org. http://developers.mozilla.org. http://developers.mozilla.org. Relevant Youtube Videos, Relevant Articles/Publications http://developers.mozilla.org. Relevant Youtube Videos, Relevant Articles/Publications	Create dummy data conforming to the JSON format Implement PUT, POST, GET, PATCH and DELETE Requests Demonstrate how to make GET and POST requests to external RESTful APIs using the fetch function. Demonstrate RESTful calls using public or already existing APIs	Guide Students to: Demonstrate the Use of JSON for data exchange between platforms. Demonstrate the various request methods and their use cases. Guide Students to: Implement asynchronous programming techniques, including promises to manage API calls effectively and maintain responsive web applications. Demonstrate RESTful calls	
-------	--	---	---	--	--	--

					using public or already existing APIs	
GENERAL OBJECTIVE: 5.0 Know the Various frontend libraries and Frameworks available						
15	5.1 Explain Libraries and Frameworks 5.2 List Available frontend libraries and Frameworks 5.3 Explain their advantages over development from scratch	Discuss the available frontend libraries like React, jQuery and frameworks like Vue, Angular and more. Discuss their advantages over development from scratch	http://developers.mozilla.org . W3Schools.com Relevant Youtube Videos, Relevant Articles/Publications			

Assessment: Give details of assignments to be used:

Coursework/ Assignments %; Course test %; Practical %; Projects %; Examination %

Type of Assessment	Purpose and Nature of Assessment (SWD 413)	Weighting (%)
Examination	Final Examination (written) to assess knowledge and understanding	40
Test	At least 2 progress tests for feedback.	10
Practical	At least 5 hands-on to be assessed by the teacher	50
Total		100

PROGRAMME: HIGHER NATIONAL DIPLOMA SOFTWARE AND WEB DEVELOPMENT		
COURSE: Back End Development II	COURSE CODE: SWD414	CONTACT HOURS: 6hrs
	CREDIT UNIT: 3	THEORETICAL: 2
YEAR: II SEMESTER: I	PRE-REQUISITE: SWD324	PRACTICAL: 4
GOAL: : This course is designed to provides students with Advanced knowledge and skills in backend development.		
GENERAL OBJECTIVES: Upon completion of the course, students should be able to: 1. Understand Endpoint and its development. 2. Understand Session and Cookies for state management in Web applications. 3. Understand Security Practices Like Authentication, Authorization, Middleware, and Web Tokens 4. Know how to connect to a Database and handle result sets. 5. Know how to comsume other APIs by making cURL requests. 6. Comprehend Version Control and its benefits. 7. Know the available Frameworks for Backend Development. 8. Understand Testing in backend development.		

PROGRAMME: HIGHERNATIONAL DIPLOMA SOFTWARE AND WEB DEVELOPMENT						
COURSE: Back End Development II		COURSE CODE:SWD414		CONTACT HOURS: 6hrs		
		CREDIT UNIT:3		THEORETICAL: 2		
YEAR: II SEMESTER: I		PRE-REQUISITE:SWD324		PRACTICAL:4		
COURSE SPECIFIFCATION: THEORETICAL AND PRACTICAL						
GOAL: : This course is designed to provides students with Advanced knowledge and skills in backend development.						
GENERAL OBJECTIVE 1.0: Know what an Endpoint is and how to develop endpoints						
THEORETICAL CONTENT				PRACTICAL CONTENT		
Week	Specific Learning Outcome	Teacher’s Activities	Resources	Specific Learning Outcome	Teacher’s Activities	Evaluation
1	1.1 Explain Endpoints. 1.2 Explain endpoint documentation and its benefits.	Discuss API Endpoints as specific URLs or routes that allow communication with a web server or service. Discuss the need for documenting APIs for consumption by others.	Textbooks, https://php.net Relevant Articles, YouTube videos, or Material	Create simple API endpoints, specifying their purpose, expected input, and response format.	Guide students to: Implement and document basic endpoints and return JSON responses to the request.	
GENERAL OBJECTIVE: 2.0 : Understand Session and Cookie management for state management in Web applications.						
2-3	2.1 Define Session and Cookies. 2.2 Explain The statelessness of a web application.	Discuss the statelessness of web applications and the concepts of sessions and cookies as mechanisms for	Textbooks, https://php.net Relevant Articles, YouTube videos, or	Develop a simple application that implements session and cookies management. Demonstrate techniques of	Guide students to: Develop a simple application that implements	

	2.3 Differentiate between Cookies and Session	maintaining state and user data. Explain the differences, benefits, and use cases of cookies and sessions.	Material, Text Editor (VS Code/Sublime/Vim/Any)	maintaining user state using Session and Cookies in a secure way.	session and cookies management Demonstrate techniques of maintaining user state using Session and Cookies in a secure way.	
GENERAL OBJECTIVE: 3.0 Understand security practices like Authentication, Authorization, Middleware and Web Tokens						
4 -6	3.1 Explain Endpoint Security and why we need it. 3.2 Explain vulnerabilities such as SQL injection, Cross-Site Scripting (XSS), and Cross-Site Request Forgery (CSRF). 3.2 Explain Authentication Tokens 3.3 List Various Authentication Token Types 3.4 Describe Authorization and its benefits in securing a web application. 3.5 Explain Roles and Permissions in Authorization	Discuss the need for basic security to provide login/ access to resources on an endpoint. Discuss vulnerabilities such as SQL injection, Cross-Site Scripting (XSS), and Cross-Site Request Forgery (CSRF). Discuss Authentication tokens and how they	Textbooks, https://php.net Relevant Articles, YouTube videos, or Material, Text Editor (VS Code/Sublime/Vim/Any) Textbooks, https://php.net Relevant Articles, YouTube videos, or	Develop a simple login app that takes user input and returns a token based on authentication Implement simple tokens with 5 minutes expiration period Demonstrate a basic understanding of Authorization for roles and permissions on Resources	Guide students to: Develop a simple login app that takes user input and returns a token based on authentication Implement simple tokens with 5 minutes expiration period. Guide students	

	3.6 Describe Middleware and its uses in Request Handling	are used. Discuss the various authentication tokens (like OAuth, OAuth 2.0, JWT, and more) Discuss the concept of authorization in securing a web application. Explain Tthe need for Authorization to limit Authenticated users from accessing resources they do not have the privileges to view, modify, or delete Discuss the nature of middleware providing onion-like layers of protection that each request has to pass through.	Material, Text Editor (VS Code/Sublime/Vim/Any) Textbooks, https://php.net Relevant Articles, YouTube videos, or Material Textbooks, https://php.net Relevant Articles, YouTube videos, or Material		to: Implement ways of restricting access to resources that they do not have rights to view, change, or delete,	
GENERAL OBJECTIVE: 4.0 Know how to connect to a Database and handle result sets.						
7 - 8	4.1 Explain Database Connection. 4.2 Describe Connection strings and	Explain Database connections.	Textbooks, https://php.net Relevant	Demonstrate various database connection methods like PDO, mysqli,	Guide students to: Demonstrate	

	Connection Parameters 4.3 Explain various connection methods like PDO, mysqli, and any other useful connection method. 4.3 Execute Basic Queries on a Database and Retrieve Data into a result set variable 4.4 Describe traversal through the result set using loops in PHP	Discuss the use of connection strings to connect to a database in an application. Discuss various connection methods like PDO, mysqli, and any other useful connection method. Discuss the execution of basic queries and the manipulation of result set data after retrieval from the Database system	Articles, YouTube videos, or Material PDO Docs MySql Docs Textbooks, https://php.net Relevant Articles, YouTube videos, or Material PDO Docs MySql Docs	and any other useful database connection method. Develop a simple application that will utilize a database connection object to retrieve, modify, or remove resources on the database using the correct Database User privileges	database various connection methods like PDO, mysqli, and any other useful database connection method. Guide students to: Create an application that executes basic <i>select</i>, <i>update</i>, <i>insert</i>, and <i>delete</i> queries on a Database management system	
GENERAL OBJECTIVE: 5.0 Know how to consume other APIs by making cURL requests.						
9	5.1 Explain cURL in PHP.	Discuss the concept of making external API calls to retrieve relevant Data and process it on the back end before sending it to the front end.	Textbooks, https://php.net Relevant Articles, YouTube videos, or Material, Text Editor(VS	Develop a simple app that makes an external API call and receives its response, parses it, and presents it.	Guide students to: Make external API calls and process the result based on the external API	

	5.2 Describe how to make cURL calls to other endpoints in a web app	Describe the process of making API calls to any gateway in a Backend Application (e.g. payment gateway)	Code/Sublime/Vim/Any)		documentation and retrieve at least one information from the returned Data.	
GENERAL OBJECTIVE: 6.0 Comprehend Version Control and its benefits						
10-11	6.1 Explain Version Control 6.2 Explain GIT 6.3 Describe GitHub, Bitbucket, and other remote repositories	Discuss the need and benefits of version control. Explain Git and its snapshot method of storing information as it is at a point in time and track changes made on documents. Explain the basic Git commands like <i>add</i> , <i>commit</i> , <i>push</i> , <i>pull</i> , <i>merge</i> , and more. Explain the benefits of remote repositories on websites like	Textbooks, https://php.net Relevant Articles, YouTube videos, or Material, Text Editor(VS Code/Sublime/Vim/Any) Git Docs	Create GitHubaccount Demonstrate the use of GitHub for version control using a simple application	Guide students to: Create a GitHub account Demonstrate the use of GitHub for version control using a simple application	

		BitBucket and GitHub for collaborations, tracking changes, and storage of source codes.				
GENERAL OBJECTIVE: 7.0 Know the available Frameworks that are good for Backend Development.						
12-13	7.1 Explore popular web frameworks in various programming languages, such as Laravel, Symphony, ASP.NET Core, and Django.	Discuss web development frameworks as pre-built, reusable structures that simplify application development. Discuss the advantages of frameworks, including rapid development, scalability, and built-in features	Textbooks, https://php.net Relevant Articles, YouTube videos, or Material, Text Editor(VS Code/Sublime/Vim/Any)			
GENERAL OBJECTIVE: 8.0 Understand Testing in backend development						
14-15	8.1 Explain Testing	Discuss testing in backend development.	Textbooks, https://php.net Relevant Articles,	Conduct a simple code testing process using any software testing tools likeXDebug, Selenium or	Guide students to: Demonstrate a simple code	

	8.2 List and Explain Types of Testing	Explain its benefits on workload reduction, code quality improvement, and integrity.	YouTube videos, or Material, Selenium, JMeter, XDebug.	Apache JMeter	testing process using any software testing tools like , XDebugSelenium or Apache JMeter	
	8.3 Discuss Benefits unit and system testing in Backend development	Discuss unit testing and system testing. Discuss their effect on the reduction of development time in collaborations and the improvement of code quality across a team.				

Assessment: Give details of assignments to be used:

Coursework/ Assignments %; Course test %; Practical %; Projects %; Examination %

Type of Assessment	Purpose and Nature of Assessment (SWD 414)	Weighting (%)
Examination	Final Examination (written) to assess knowledge and understanding	40
Test	At least 2 progress tests for feedback.	10
Practical	At least 5 hands-on to be assessed by the teacher	50
Total		100

PROGRAMME: HIGHER NATIONAL DIPLOMA SOFTWARE AND WEB DEVELOPMENT		
COURSE: Software Design and Architecture	COURSE CODE: SWD415	CONTACT HOURS: 6hrs
	CREDIT UNIT: 3	THEORETICAL: 2
YEAR: II SEMESTER: I	PRE-REQUISITE:	PRACTICAL: 4
GOAL: This course is designed to provide students with knowledge and skills on Software Design and Architecture.		
GENERAL OBJECTIVES: On completion of the course, the student should be able to: 1. Understand the essential elements of software design and architecture. 2. Apply design principles, design patterns, architectural patterns and system decomposition: 3. Know scalability, performance and security considerations in software design 4. Comprehend the design of systems for Integration and Interoperability. 5. Understand software documentation, communication, evaluation and analysis 6. Understand case studies and practical applications of software designs while exploring		

PROGRAMME: HIGHERNATIONAL DIPLOMA SOFTWARE AND WEB DEVELOPMENT							
COURSE: Software Design and Architecture			COURSE CODE: SWD415		CONTACT HOURS: 6hrs		
			CREDIT UNIT: 3		THEORETICAL: 2		
YEAR: II SEMESTER: I			PRE-REQUISITE:		PRACTICAL: 4		
COURSE SPECIFIFCATION: THEORETICAL AND PRACTICAL							
GOAL: This course is designed to provide students with knowledge and skills on Software Design and Architecture.							
GENERAL OBJECTIVE 1.0: Understand the essential elements of software architecture concepts:							
THEORETICAL CONTENT				PRACTICAL CONTENT			
Week	Specific Learning Outcome		Teacher’s Activities	Resources	Specific Learning Outcome	Teacher’s Activities	Evaluation
1-3	1.1.	Understand the Definition of Software Architecture	Define and explain fundamental software architecture concepts, such as components, modules, layers, and architectural styles (e.g., client-server, microservices).	Lucidchart, draw.io, or Microsoft Visio for creating architectural diagrams and visual representations.	Create clear and concise documentation of software architecture, including diagrams, charts, and descriptions that effectively communicate the system's structure to team members and stakeholders.	Guide students to: Create clear and concise documentation of software architecture, including diagrams, charts, and descriptions that effectively communicate	
	1.2.	Understand Components, Modules and Architectural Styles					
	1.3.	Understand design principles and Architectural Patterns					
	1.4.	Understand Quality Atributes,	Explain the importance of design	PowerPoint, Google Slides, or Prezi for	Identify the key components and modules		

	documentation and Tradeoffs	principles in software architecture, including principles like separation of concerns, modularity, and abstraction.	creating presentations explaining architectural concepts.	within a software system, and understand how they interact to fulfill the system's functionality.	the system's structure to team members and stakeholders.	
	1.5. Understand Architecture Evaluation			Analyze trade-offs in software architecture decisions.	Identify the key components and modules within a software system, and understand how they interact to fulfill the system's functionality.	
	1.6. Understand communication and ethical considerations in software design	Discuss different architectural styles, (such as client-server, microservices, monolithic, and event-driven architectures) and understand the advantages and disadvantages of each.			Analyze trade-offs in software architecture decisions. For example, understanding that optimizing for one quality attribute (e.g., performance) may lead to	
		Explain the common architectural patterns (such as Model-View-Controller (MVC), Layered Architecture, and Service-Oriented Architecture (SOA))				

		and understand when and how to apply them. Explain how software architecture decisions impact non-functional attributes (quality attributes) (like performance, scalability, security, and maintainability) and be able to make informed design choices to meet specific quality goals. Explain techniques for evaluating and reviewing software architectures, including architectural reviews, code inspections, and architectural modeling tools.			trade-offs in other areas (e.g., maintainability).	
--	--	---	--	--	---	--

		Explain software architecture concepts and explain how to communicate effectively about software architecture concepts with both technical and non-technical stakeholders, facilitating better decision-making and understanding				
GENERAL OBJECTIVE: 2.0Apply design principles, design patterns, architectural patterns and system decomposition:						
4-6	2.1. Explain software design principles such as SOLID, DRY and KISS principles 2.2. Explain design and Architectural patterns. 2.3. Explain System Decomposition 2.4. Explain Trade-offs and how design decisions affect testing strategies and coverage.	Discuss software design principles, including SOLID principle (Single Responsibility, Open-Closed, Liskov Substitution, Interface Segregation, Dependency Inversion) and DRY (Don't Repeat Yourself). Explain how to	PowerPoint, Google Slides, or Prezi for creating presentations explaining architectural concepts. Visual Studio, IntelliJ IDEA, Eclipse, or JetBrains Rider for writing, testing, and	Apply the Don't Repeat Yourself (DRY) principle to eliminate code duplication and promote code reusability. Apply the Keep It Simple, Stupid (KISS) principle to design simple and straightforward solutions rather than overcomplicating designs. Identify common design patterns such as creational,	Guide the student to : Apply the Don't Repeat Yourself (DRY) principle to eliminate code duplication and promote code reusability. Apply the Keep It	

	2.5. Explain Documentation and Communication	evaluate and select the most suitable architectural pattern for a given project based on its requirements and constraints. Explain how to adapt and modify design patterns to suit unique project requirements. Explain how to manage dependencies between modules to minimize coupling and promote reusability. Discuss how to Integrate modules and components into a cohesive system, considering factors like data flow, communication protocols, and error	debugging code Online resources or books about design patterns, such as "Design Patterns: Elements of Reusable Object-Oriented Software" by the Gang of Four. Enterprise Architect, Visual Paradigm, or PlantUML for creating UML diagrams to visualize design patterns and architectural concepts	structural, and behavioral patterns. Design clear and well-defined interfaces between modules to facilitate communication and collaboration Identify various architectural patterns (e.g., Model-View-Controller, Microservices, Layered Architecture). Apply appropriate design patterns to solve specific design problems efficiently. Apply architectural patterns to structure software systems effectively. Decompose complex systems into manageable modules or components, ensuring that each module has a well-defined responsibility.	Simple, Stupid (KISS) principle to design simple and straightforward solutions rather than overcomplicating designs. Identify common design patterns such as creational, structural, and behavioral patterns. Design clear and well-defined interfaces between modules to facilitate communication and collaboration	
--	---	---	---	--	---	--

		handling. Explain how to evaluate trade-offs when choosing design principles, patterns, and architectural patterns. Explain design decisions and how they may impact system qualities like performance, maintainability, and scalability. Explain how to make informed design decisions by considering project requirements, constraints, and the long-term goals of the system. Explain how to apply design principles, patterns, architectural		Design clear and well-defined interfaces between modules to facilitate communication and collaboration. Apply design principles, patterns, architectural patterns, and system decomposition techniques to real-world software design projects. Show how these concepts are used in practice and adopt them to address specific project challenges. Plan and conduct tests to validate that the chosen design principles and patterns meet the desired functional and non-functional requirements.	Identify various architectural patterns (e.g., Model-View-Controller, Microservices, Layered Architecture). Apply appropriate design patterns to solve specific design problems efficiently. Apply architectural patterns to structure software systems effectively. Decompose complex systems into manageable modules or	
--	--	---	--	---	---	--

		patterns, and system decomposition techniques to real-world software design projects. Explain how these concepts are used in practice and adopt them to address specific project challenges.			components, ensuring that each module has a well-defined responsibility. Design clear and well-defined interfaces between modules to facilitate communication and collaboration. Apply design principles, patterns, architectural patterns, and system decomposition techniques to real-world software design projects. Demonstrate	
--	--	--	--	--	---	--

					how these concepts are used in practice and adopt them to address specific project challenges. Plan and conduct tests to validate that the chosen design principles and patterns meet the desired functional and non-functional requirements	
GENERAL OBJECTIVE: 3.0 Know scalability, performance and security considerations in software design						
7-8	3.1. Explain scalability, performance, and security considerations in software design. 3.2. Explain the importance of capacity planning to ensure that a system can scale to meet anticipated future demands. 3.3. Identify and define performance metrics, including response time, throughput, and resource utilization. 3.4. Explain fundamental security	Discuss scalability in the context of software design and architecture, emphasizing the ability of a system to handle increased workloads gracefully. Explain the key factors affecting	PowerPoint, Google Slides, or Prezi for creating presentations explaining architectural concepts. Apache JMeter, LoadRunner, or Gatling.	Plan and conduct scalability testing to evaluate system performance under different levels of load. Conduct load testing to measure system performance under various levels of load and stress. Implement performance	Guide students to: Demonstrate scalability testing to evaluate system performance under different levels of load. Conduct load	7-8

	principles, including confidentiality, integrity, and availability (CIA triad). 3.5. Understand the trade-offs between performance optimization and scalability, as optimizing for one may impact the other. 3.6. Explain ethical and legal responsibilities related to data privacy, security, and responsible technology use. 3.7. Explain the implications of data breaches and security lapses on individuals and organizations.	scalability, including horizontal and vertical scaling, load balancing, and distributed systems. Explain how to identify and address performance bottlenecks in a software system. Explain the strategies for designing scalable systems, (such as partitioning data, caching, and using asynchronous processing). Explain techniques for optimizing software performance, such as code profiling, database indexing, and algorithmic improvements Explain how to perform threat	OWASP ZAP, Nessus, or Burp Suite New Relic, AppDynamics, or Prometheus	monitoring and profiling tools to continuously monitor system performance and diagnose issues. Apply secure design principles, such as the principle of least privilege, input validation, and secure authentication and authorization. Implement secure communication protocols (e.g., HTTPS) and encryption to protect data in transit. Identify the trade-offs between security measures and user-friendly design, and make informed decisions that balance security with usability.	testing to measure system performance under various levels of load and stress. Implement performance monitoring and profiling tools to continuously monitor system performance and diagnose issues. Apply secure design principles, such as the principle of least privilege, input validation, and secure authentication and authorization. Implement secure communication protocols (e.g.,	
--	--	---	--	---	--	--

		modeling to identify potential security threats and vulnerabilities in a software system. Explain security testing techniques, including penetration testing and vulnerability scanning, to assess and improve the security of a system Explain industry-specific security standards, compliance requirements, and legal regulations (e.g., GDPR) that may impact software design and data handling.			HTTPS) and encryption to protect data in transit. Demonstrate the trade-offs between security measures and user-friendly design, and make informed decisions that balance security with usability.	
GENERAL OBJECTIVE: 4.0 Comprehend the design of systems for Integration and Interoperability:						
9-11	4.1. Explain the design of systems for integration and interoperability.	Discuss integration and interoperability	PowerPoint, Google Slides, or	Show how to use various integration technologies,	Guide students to:	

	4.2. Explain the challenges and methods for integrating data from heterogeneous sources, including databases, data warehouses, and external APIs. 4.3. Explain Service-Oriented Architecture (SOA). 4.4. Explain API versioning, authentication, and security considerations in API design. 4.5. Explain Message formats and Protocols. 4.6 Explain middleware and Enterprise Service Buses (ESBs) and Integration patterns. 4.7 Explain how adherence to standards promotes system compatibility.	in the context of software systems. Distinguish between the two concepts (understanding that integration refers to connecting different systems or components, while interoperability focuses on ensuring these systems can work together seamlessly). Explain various integration technologies, such as RESTful APIs, SOAP, message queues, and web services. Explain data transformation techniques, data mapping, and data validation for effective data integration.	Prezi for creating presentations explaining architectural concepts. API Development Tools such as Postman, Swagger, or Insomnia RabbitMQ, Apache Kafka, or AWS SQS for implementing messaging systems Apache Camel, MuleSoft, or Apache ServiceMix for	such as RESTful APIs, SOAP, message queues, and web services Compare and contrast different integration technologies, considering factors like performance, scalability, and compatibility with existing systems Design and implement services using SOA principles, including service discovery, composition, and orchestration. Show how the various message formats (e.g., JSON, XML) and protocols (e.g., HTTP, MQTT) are used in system integration. Demonstrate how middleware and Enterprise Service Buses (ESBs) are used as tools for managing complex integrations and	Use various integration technologies, such as RESTful APIs, SOAP, message queues, and web services Compare and contrast different integration technologies, considering factors like performance, scalability, and compatibility with existing systems Design and implement services using SOA principles, including service discovery,	
--	--	--	--	---	--	--

		Explain the principles of SOA and its role in enabling interoperability through service-based communication. Explain how to design clean and well-documented APIs that promote ease of integration for external developers and systems. Explain various message formats (e.g., JSON, XML) and protocols (e.g., HTTP, MQTT) used in system integration. Explain common integration patterns (such as publish-subscribe, request-		mediating between systems. Demonstrate common integration patterns such as publish-subscribe, request-reply, and point-to-point, and explain when to apply them in integration scenarios. Implement integration patterns using appropriate technologies and tools. Develop strategies for testing and validating integrated systems to ensure data consistency, error handling, fault tolerance and perform integration testing and validation to identify and resolve integration issues. Design robust error handling mechanisms to gracefully handle integration failures and recover from errors.	composition, and orchestration. Demonstrate the various message formats (e.g., JSON, XML) and protocols (e.g., HTTP, MQTT) are used in system integration. Demonstrate how middleware and Enterprise Service Buses (ESBs) are used as tools for managing complex integrations and mediating between systems. Demonstrate common	
--	--	---	--	--	---	--

		reply, and point-to-point) and understand when to apply them in integration scenarios. Explain industry standards and protocols (such as REST, SOAP, and OData) that facilitate interoperability between systems.		Implement effective logging and monitoring practices to track integration activities and diagnose issues.	integration patterns such as publish-subscribe, request-reply, and point-to-point, and explain when to apply them in integration scenarios. Implement integration patterns using appropriate technologies and tools. Develop strategies for testing and validating integrated systems to ensure data consistency, error handling, fault tolerance and perform integration	
--	--	---	--	--	--	--

					testing and validation to identify and resolve integration issues. Design robust error handling mechanisms to gracefully handle integration failures and recover from errors. Implement effective logging and monitoring practices to track integration activities and diagnose issues.	
GENERAL OBJECTIVE: 5.0 Understand software documentation, communication, evaluation and analysis						
12-13	5.1 Explain software documentation Skills 5.2 Explain Effective Communication in	Explain software documentation, communication, evaluation, and	PowerPoint, Google Slides, or Prezi for creating presentations	Create a comprehensive and well-organized software documentation, including technical	Guide students to: Create a comprehensive	

	Software Documentation. 5.3 Explain the importance of version control systems (e.g., Git) in software development. 5.4 Explain the Use of Unified Modeling Language (UML) Diagrams 5.5 Explain API and User documentation	analysis. Explain how to communicate technical concepts, design decisions, and project updates effectively to both technical and non-technical stakeholders Explain Unified Modeling Language (UML) diagrams, including class diagrams, sequence diagrams, and activity diagrams, and how it can be used to visually represent software architecture and behavior. Explain how to produce comprehensive API documentation that includes endpoints, request/response	explaining architectural concepts. Confluence, Microsoft Word, Google Docs Slack, Microsoft Teams, or Zoom JIRA, Trello, or Asana for tracking and managing software development tasks GitHub or GitLab for tracking changes in code and documentation	specifications, user manuals, and design documents. Use appropriate documentation tools and formats to convey information clearly and concisely. Document and communicate version control processes and workflows to team members. Create use case diagrams to represent the interactions between system components and stakeholders, helping to define system functionality. Create flowcharts, data flow diagrams, and other visual aids to illustrate	and well-organized software documentation, including technical specifications, user manuals, and design documents. Use appropriate documentation tools and formats to convey information clearly and concisely. Document and communicate version control processes and workflows to team members.	
--	---	---	---	---	--	--

		formats, authentication methods, and usage examples Explain the importance of keeping documentation up-to-date as the software evolves and changes.		system processes, data flows, and decision-making logic. Produce comprehensive API documentation that includes endpoints, request/response formats, authentication methods, and usage examples. Develop user-friendly documentation that assists end-users in understanding software functionality and how to use the system effectively. Create change logs and release notes that document software updates, bug fixes, and new features, helping users and developers stay informed. Conduct documentation	Create use case diagrams to represent the interactions between system components and stakeholders, helping to define system functionality. Create flowcharts, data flow diagrams, and other visual aids to illustrate system processes, data flows, and decision-making logic. Produce comprehensive API	
--	--	---	--	--	---	--

				audits to ensure compliance with organizational and regulatory standards Create UX documentation that describes user personas, user journeys, and wireframes to inform design decisions and enhance user satisfaction.	documentation that includes endpoints, request/response formats, authentication methods, and usage examples. Develop user-friendly documentation that assists end-users in understanding software functionality and how to use the system effectively. Create change logs and release notes that document software updates, bug fixes, and new	
--	--	--	--	--	---	--

					features, helping users and developers stay informed. Conduct documentation audits to ensure compliance with organizational and regulatory standards Create UX documentation that describes user personas, user journeys, and wireframes to inform design decisions and enhance user satisfaction.	
GENERAL OBJECTIVE: 6.0 Understand case studies and practical applications of software designs while exploring ethical and legal considerations						

14-15		Explain how to analyze and dissect real-world software design case studies to understand the challenges, decision-making processes, and outcomes.	PowerPoint, Google Slides, or Prezi for creating presentations explaining architectural concepts.	Analyze and dissect real-world software design case studies to understand the challenges, decision-making processes, and outcomes.	Guide students to analyze and dissect real-world software design case studies to understand the challenges, decision-making processes, and outcomes.	
	6.1 Explain case study analysis of software design	Discuss and evaluate ethical implications of design decisions, considering potential consequences on users and society.		Identify lessons and best practices from case studies that can be applied to future software design projects.	Identify lessons and best practices from case studies that can be applied to future software design projects.	
	6.2 Explain practical application of Software design.	Explain the legal requirements and regulations that impact software design, such as data protection laws (e.g., GDPR), intellectual property rights, and accessibility standards (e.g., ADA).		Implement secure authentication, authorization, and data encryption mechanisms.	Implement secure authentication, authorization, and data encryption mechanisms.	
	6.3 Explain ethical considerations and legal compliance					
	6.4 Explain Security, Privacy and Risk Assessment					
	6.5 Explain Accessibility and Inclusivity					
	6.6 Explain Professionalism and Accountability in Software Development	Explain how user-				

		centered design principles is used to create software solutions that prioritize user needs, accessibility, and usability. Explain how to incorporate security and privacy considerations into software design, addressing vulnerabilities and safeguarding sensitive data. Explain how to design software with accessibility in mind to ensure that it is usable by individuals with disabilities.				
--	--	---	--	--	--	--

PROGRAMME: HIGHER NATIONAL DIPLOMA SOFTWARE AND WEB DEVELOPMENT		
COURSE: Advanced Data Structures and Algorithms	COURSE CODE: SWD416	CONTACT HOURS: 6hrs
	CREDIT UNIT: 3	THEORETICAL: 2
YEAR: II SEMESTER: I	PRE-REQUISITE:	PRACTICAL: 4
GOAL: This course is designed to provide students with knowledge and skills in Advanced Data Structures and Algorithms		
GENERAL OBJECTIVES: On completion of this course the student should be able to: 1. Comprehend basic concept of data structures 2. Understand Graph data structures 3. Understand Advanced Sorting and Searching Algorithm 4. Know Advanced Tree data structures 5. Know Advanced Stack data structures 6. Comprehend Advanced Queue Data Structures 7. Understand Asymptotic Analysis.		

PROGRAMME: HIGHERNATIONAL DIPLOMA SOFTWARE AND WEB DEVELOPMENT						
COURSE: Advanced Data Structures and Algorithms			COURSE CODE:SWD416		CONTACT HOURS: 6hrs	
			CREDIT UNIT:3		THEORETICAL: 2	
YEAR: II SEMESTER: I			PRE-REQUISITE:		PRACTICAL:4	
COURSE SPECIFIFCATION: THEORETICAL AND PRACTICAL						
GOAL: This course is designed to provide students with knowledge and skills in Advanced Data Structures and Algorithms						
GENERAL OBJECTIVE 1.0: Comprehend Reviewed Data Structure Basics						
THEORETICAL CONTENT				PRACTICAL CONTENT		
Week	Specific Learning Outcome	Teacher’s Activities	Resources	Specific Learning Outcome	Teacher’s Activities	Evaluation
1-3	1.1 Define Data Structure 1.2 Define data attributes; name, value range, data types 1.3 Define unit for Identifying Data,Character, Fields, Subfields, Records, Files 1.4 Define symbols, relations and graph 1.5 Explain the symbols for expressing relations among data 1.6 State properties of graph: routes, edge, sequences, directed and non-directed	Discuss the Concept of Data Structures (eg. data attributes , name, value range and data types (Character fields, sub field, Records and Files)). Discuss symbols, relations and graphs. Explain the properties of graphs(routes, edge	White board multimedia projector Text books	Demonstrate using relevant examples: concepts of attributes, name, value range , data types, character fields, sub fields, records and file Write simple Programs to carry out set operations, Array, Graph, Tree, Stack and Queue Demonstrate using relevant examples on how to use symbols, relations and graph	Guide Students to: Demonstrate using relevant examples: concepts of attributes, name, value range, data types, character fields, sub fields, records and file Write simple	

	1.7 Describe operations such as precede, less than, points to, move to, search, change, entry 1.8 Describe sets and relation, the elements of sets, subsets, super sets, universal set and null set and describe set Operations 1.9 Describe Simple Link List eg Array, double linked list, queues, stack, trees. 1.10 Explain the use of pointers. 1.11 Discribe the concept of trees and Graphs, 1.12 Define sorting and Explain the various Sorting techniques	sequences, directed and non-directed) Discuss operations (such as precede, less than points to move to , search, change, entry) Discuss sets and relation, the elements of sets, subsets, super sets, universal set and null set andset operations. Discuss linked list, types of linked list, and compare it with linear list Discuss the use of pointers Discuss Trees and Graphs Explain Sorting (comparison based		Apply Linked List Demonstrate the Push and Pop operations possibly with diagrams Implement different Sorting and Searching techniques in aprogram	Programs to carry out set operations, Array, Graph, Tree, Stack and Queue Demonstrate using relevant examples on how to use symbols, relations and graph Apply Linked List Demonstrate the Push and Pop operations possibly with diagrams Write programs to implement different	
--	---	--	--	--	--	--

		sorting, bubble sorting algorithm, selection sorting algorithm).			sorting and searching techniques	
GENERAL OBJECTIVE: 2.0 Understand Advanced Graph Data Structures						
4-5	2.1 Describe Depth-First Search (DFS), Breadth-First Search (BFS) 2.2 Describe Dijkstra's Algorithm, Bellman-Ford Algorithm, Floyd-Warshall Algorithm. 2.3 Describe Graph Traversal Algorithms for connectivity 2.4 Explain Graph Algorithms in Natural Language Processing 2.5 Explain Geometric Graph Algorithms 2.6 Describe the Shortest Path Algorithms in Weighted Graphs	Explain the Depth-First Search and Breadth First Search Explain the Dijkstra's, Bellman-Ford and Floyd-Warshall Algorithms Explain the Graph Traversal Algorithms for Connectivity and Graph Algorithms in Natural Language Processing. Describe Geometric Graph Algorithms and Shortest Path Algorithms in Weighted Graph.	White board and multimedia projector Online Code Repositories Visualizations Online Coding Environments like CodePen or Jupyter Notebook Plotting library like Matplotlib.	Create a graphdata structure Implement and print the DFS and BFS traversals for the Custom Graphs. Modify the code to handle Directed Graphs Experiment with larger Graphs to observe the differences in DFS and BFS traversal results. Solve classic Graph problems such as finding the Shortest Path between two nodes and detecting cycles using these Traversal Algorithms.	Guide Students to: Create a graph Implement and print the DFS and BFS Traversals for your Custom Graphs. Modify the code to handle Directed Graphs Experiment with larger Graphs to observe the	

				Apply Dijkstra's Algorithm to Real-World Scenarios. Build a Graph to Identify important keywords. Write code to find the closest pair of points in a set of 2D points using the divide and Conquer Algorithm Use Random Point generation to test the Algorithm's performance with varying input sizes.	differences in DFS and BFS traversal results. Solve Classic Graph problems such as finding the Shortest Path between two nodes and detecting Cycles using these Traversal Algorithms. Apply Dijkstra's Algorithm to Real-World Scenarios. Build a Graph to Identify important keywords. Write code to find the closest pair of points	
--	--	--	--	--	--	--

					in a set of 2D points using the divide and Conquer Algorithm Use Random Point generation to test the Algorithm's performance with varying input sizes.	
GENERAL OBJECTIVE: 3.0 Understand Advanced Sorting and Searching Algorithm						
6-7	3.1 Describe Merge Sort Algorithm, It's time Complexity and Space Complexity 3.2 Describe Quick Sort Algorithm, it's time Complexity and Space Complexity 3.3 Describe Heap Sort Algorithm, it's time Complexity and Space	Explain the various Sorting Algorithms Techniques (e.g Merge Sort,Quick Sort, Heap Sort,Radix Sort, Tim Sort and Intro Sort) in terms of time and space complexity Explain different Searching Techniques (eg:	White board and multimedia projector TextBooks Websites and Online Resources YouTube Channels	Implement Sorting Algorithms like Merge sort, Quicksort, and Heapsort Implement and Compare various searching Algorithms like Binary search, Interpolation search, and Exponential search on different datasets.	Guide Students to: Implement sorting algorithms like merge sort, quicksort, and heapsort. Implement and Compare various	

	Complexity 3.4 Describe Radix Sort Algorithm, it's time Complexity and Space Complexity 3.5 Describe Tim Sort Algorithm, it's time Complexity and Space Complexity 3.6 Describe ..Intro Sort.. Algorithm, it's time Complexity and Space Complexity 3.7 Describe Searching Techniques such as: Fibonacci Search, Trie (Prefix Tree) Search, Suffix Array and Suffix Tree Searches, Bidirectional Search, Hashing, Binary Search, etc	Fibonacci Search, Trie (Prefix Tree) Search, Suffix Array and Suffix Tree Searches, Bidirectional Search, Hashing etc).		Create a simple Text Search Engine that can find keywords in a collection of documents.	searching Algorithms like Binary search, Interpolation search, and Exponential search on different datasets. Create a Simple Text Search Engine that can find keywords in a collection of documents.	
GENERAL OBJECTIVE: 4.0 : Know Advanced Tree Data Structures						

8-9	4.1 Describe Adelson-Velsky and Landis (AVL) and Red-Black Trees 4.2 Describe B-Trees and B+ Trees 4.3 Describe Splay Trees 4.4 Describe Trie (Prefix Tree) 4.5 Describe Segment Trees	Explain AVL, Red-Black Trees, B-Trees and B+ Trees in terms of the overview, properties, balancing properties, advantages, disadvantages and applications Explain Splay Trees the overview, basic operations, splaying operation, advantages, disadvantages, balancing and applications Explain Trie (Prefix Tree) in terms of the overview, properties, advantages, disadvantages and applications Explain Segment trees in terms of the overview, properties, construction,	White board and multimedia projector Textbooks Online resources like GeeksforGeeks Tutorials on YouTube	Use Python or any Programming language to implement the splaying operation for a binary search tree (BST) to bring a specific node to the root. Create a basic Trie to store and search for words	Guide Students to: Use Python or any Programming language to Implement the splaying operation for a binary search tree (BST) to bring a specific node to the root. Create a basic Trie to store and search for words	
-----	---	---	---	---	--	--

		advantages, disadvantages and applications				
GENERAL OBJECTIVE: 5.0 Know Advanced Stack Data Structures						
10-11	5.1 Describe Min Stack and Max Stack 5.2 Describe Expression Evaluation 5.3 Describe Conversion between infix, postfix, and prefix notations. 5.4 Explain Monotonic Stack 5.5 Explain Stack as a Function Call Stack	Explain Min Stack and Max Stack Discuss expression evaluation Explain Conversion between infix, postfix, and prefix notations. Explain Monotonic Stack Explain Stack as a Function Call Stack	White board and multimedia projector Textbooks Online resources Tutorials on YouTube	Create a stack that supports both minimum and maximum value retrieval in constant time. Design a Stack with "GetMin" and "GetMax" Methods Create a program that evaluates simple arithmetic expressions that will support a variety of operators (+, -, *, /, parentheses) in infix notation. Use a stack to handle operator precedence and parentheses. Implement error handling to detect and report syntax	Guide Students to: Create a Stack that supports both minimum and maximum value retrieval in constant time. Design a Stack with "GetMin" and "GetMax" Methods Create a program that evaluates simple arithmetic expressions that will	

				errors in expressions, such as unmatched parentheses or invalid operators Create a program that converts infix expressions to postfix notation, infix expressions to prefix notation and postfix expressions back to infix notation. Verify the correctness of your conversion by comparing the results with manual conversions. Write a Python program to find the nearest smaller element for each element in an input array using a monotonic stack.	support a variety of operators (+, -, *, /, parentheses) in infix notation. Use a stack to handle operator precedence and parentheses. Implement error handling to detect and report syntax errors in expressions, such as unmatched parentheses or invalid operators Create a program that converts infix expressions to postfix notation, infix expressions to	
--	--	--	--	---	---	--

					prefix notation and postfix expressions back to infix notation. Verify the correctness of your conversion by comparing the results with manual conversions. Write a Python program to find the nearest smaller element for each element in an input array using a monotonic stack.	
--	--	--	--	--	---	--

GENERAL OBJECTIVE: 6.0 ComprehendAdvanced Queue Data Structures						
12-13	6.1 Describe Double-ended Queue (Deque) 6.2 Describe Queue Scheduling Algorithms: Priority and Round Robin 6.3 Explain Deque Applications 6.4 Describe Queue Applications in BFS 6.5 Describe the Sliding Window Problems 6.6 Explain Circular Queue	Explain Double-ended Queue (Deque) Discuss Queue Scheduling Algorithms(Priority and Round Robin) Explain Deque Applications Explain Queue Applications in BFS Explain the Sliding Window Problems Explain Circular Queue	White board and multimedia projector Textbooks Online resources Tutorials on YouTube	Create a program that demonstrates deque operations: - Import the deque module - Create a deque - Adding elements - Removing Elements create a simple priority queue using Python or any programming language Simulate Round Robin scheduling using a list in Python. Demonstrate how BFS, using a queue, can be applied to find the shortest path between nodes and determine graph connectivity. Given an array of integers, find the maximum possible sum of a subarray of a	Guide Students to: Create a program that demonstrates deque operations: - Import the deque module - Create a deque - Adding elements - Removing Elements create a simple priority queue using Python or any programming language Simulate Round Robin scheduling	

				fixed size k. Implementing a circular queue in Python	using a list in Python. Demonstrate how BFS, using a queue, can be applied to find the shortest path between nodes and determine graph connectivity. Given an array of integers, find the maximum possible sum of a subarray of a fixed size k. Implement a circular queue in Python	
GENERAL OBJECTIVE: 7.0 Understand Asymptotic Analysis						
14-15	7.1 Describe Asymptotic Analysis, the Purpose and Notations	Explain Asymptotic Analysis, the Purpose and	White board and multimedia projector	Implement and analyze the best case/worst-case/average-case time	Guide Students to:	

	7.2 Describe Best-case, Average-case and worst-Case Analysis 7.3 Describe the Comparative Analysis 7.4 Describe the Common Complexity Classes 7.5 Describe Limitations and Practical Use 7.6 Describe its Application to Data Structures	Notations Explain best-case, Average-case, worst-Case Analysis and Comparative Analysis Explain the Common Complexity Classes, Limitations and Practical Use Explain its Application to Data Structures	Textbooks Benchmarking Libraries (e.g., timeit in Python)	complexity of: - simple sorting Algorithms - Merge Sort and QuickSort - Binary Search - Tree Traversal - Graph Algorithms etc. Use benchmarking libraries (e.g., timeit in Python) to measure the actual runtime of your worst-case scenarios and compare them to the theoretical analysis Implement simple algorithms or functions with constant time complexity. Write code to perform binary search in a sorted array and analyze its logarithmic time complexity Review and analyze code	Implement and analyze the worst-case/average-case/ best-case time complexity of: - simple sorting Algorithms - Merge Sort and QuickSort - Binary Search - Tree Traversal - Graph Algorithms etc. Use Benchmarking Libraries (e.g., timeit in Python) to measure the actual runtime of your worst-case/ average-case/ best-case scenarios and	
--	---	---	---	---	--	--

				written by others or available in open-source projects to identify the complexity classes of the algorithms used. Determine whether the chosen algorithms are appropriate for the given problem.	compare them to the theoretical analysis Implement Simple Algorithms or Functions with Constant Time Complexity. Write code to perform binary search in a sorted array and analyze its Logarithmic time Complexity Review and analyze code written by others or available in open-source projects to identify the complexity classes of the	
--	--	--	--	--	---	--

					Algorithms used. Determine whether the chosen Algorithms are appropriate for the given problem.	
--	--	--	--	--	--	--

Assessment: Give details of assignments to be used:

Coursework/ Assignments %; Course test %; Practical %; Projects %; Examination %

Type of Assessment	Purpose and Nature of Assessment (SWD 416)	Weighting (%)
Examination	Final Examination (written) to assess knowledge and understanding	40
Test	At least 2 progress tests for feedback.	10
Practical	At least 5 hands-on to be assessed by the teacher	50
Total		100

PROGRAMME: HIGHER NATIONAL DIPLOMA SOFTWARE AND WEB DEVELOPMENT

COURSE: Human Computer Interface (HCI)	COURSE CODE: SWD 421	CONTACT HOURS: 3hrs
	CREDIT UNIT: 2	THEORETICAL: 2
YEAR: II SEMESTER: II	PRE-REQUISITE:	PRACTICAL: 1

GOAL: : To Provide students with knowledge and skills in the design and use of Human Computer Interface (HCI)
--

GENERAL OBJECTIVES: On completion of this course the student should be able to:

1. Understand interaction design .
2. Comprehend conceptualized interaction.
3. Understand Principles and application of user centered design
4. Understand HCI design for collaboration and communication.
5. Know how interfaces affect users.
6. Understand the process of interaction design .
7. Understand HCI Design for Emerging Technologies
8. Understand Ethical and societal implications of Human-Computer Interaction

PROGRAMME: HIGHERNATIONAL DIPLOMA SOFTWARE AND WEB DEVELOPMENT						
COURSE: Human Computer Interface (HCI)			COURSE CODE:: SWD 421		CONTACT HOURS: 3hrs	
			CREDIT UNIT:2		THEORETICAL: 2	
YEAR: II SEMESTER: II			PRE-REQUISITE:		PRACTICAL:1	
COURSE SPECIFIFCATION: THEORETICAL AND PRACTICAL						
GOAL: : To Provide students with knowledge and skills in the design and use of Human Computer Interface (HCI)						
GENERAL OBJECTIVE 1.0: Understand interaction design						
THEORETICAL CONTENT				PRACTICAL CONTENT		
Week	Specific Learning Outcome	Teacher’s Activities	Resources	Specific Learning Outcome	Teacher’s Activities	Evaluation
1-2	1.1 Explain Human computer interaction and interaction design. 1.2 Explain Usability and Design Principles. 1.3 Explain Learnability, Memorability, and Interface Metaphors. 1.4 –Explain the importance of user Interface -Explain the benefits of a good design. -Discuss a brief History of Screen Design.	Explain what Human computer interaction is and why it is needed. Discuss interaction design. Discuss the components of interaction design. Discuss why HCI requires working together as a multi	White Board PC Multimedia Projector User Interface prototyping software (like Pencil, Axure RP, Wireframe, Rapid UI, Design Sketch, etc.)	Design a data entry form prototype that incorporates the design principles (Constraining, Consistency, and Affordance). Perform mappig using metaphors.	Guide students to: Design a data entry form prototype that incorporates the design principles (Constraining, Consistency, and Affordance). Perform mappig using metaphors	What is Human computer interaction? Explain Usability and Design Principles. What are the make up of interaction desingn? Use metaphors

	1.5 –Explain Graphical System Characteristics -Discuss Web User Interface Popularity -Explain the Characteristics/Principles of User Interface.	disciplinary team Explain the purpose of interactive design in big business such as website consulting and mobile computing industry. Explain the interaction design process. Discuss the goals of interaction design (usability and user experience). Discuss design and usability principles. Explain how metaphors are used in developing designs through a mapping process.				to create a mapping process in designs. State the importance of user interface. State the benefits of a good design. Discuss a brief history of screen design. Explain Graphical System Characteristics. Discuss web User Interface Popularity.
--	--	---	--	--	--	---

		Explain the importance of user interface. Explain the benefits of a good design. Discuss the brief history of screen design. Explain the characteristics of a Graphical System. Explain Web User Interface Popularity Explain the Characteristics/Principles of User Interface.				
GENERAL OBJECTIVE: 2.0 Comprehend the conceptualize interaction						

3-4	2.1 Explain problem space in HCI. 2.2 Explain Conceptual models based on activities and objects 2.3 Explain interface metaphors and interaction paradigms.	Discuss problem space in HCI Discuss conceptual models based on activities. Discuss the conceptual models based on objects Explain interface metaphors Explain interaction paradigms.	White Board PC Multimedia Projector User Interface prototyping software (like Pencil, Axure RP, Wireframe, Rapid UI, Design Sketch, etc.)	Design an interface prototype that incorporates at least two design metaphors, one of which must be original.	Guide Students to: Demonstrate the use of Interface metaphors. Demonstrate how real life experience is used to come up with Design Metaphors. Identify Interface metaphors incorporated in the design of common software like Operating Systems, Word Processors, and Spreadsheets.	What is a metaphor? Give examples of design metaphors.
GENERAL OBJECTIVE: 3.0 Understand principles and application of user-centered design						
5-6	3.1 Explain conceptual frameworks for cognition such as mental models, information processing and external cognition.	Explain Cognition Discuss conceptual frameworks for	White Board PC	Conduct the Stroop Test in pairs	Guide students to: Demonstrate perception	Explain conceptual frameworks for cognition.

		Cognition. Explain Mental models, Information processing and External Cognition Explain informal design from theory to Practice.	Multimedia Projector User Interface prototyping software. (like Pencil, Axure RP, Wireframe, Rapid UI, Design Sketch, etc.)	Compare Conceptual frameworks for cognition.	interference by conducting Stroop tests. Demonstrate the use of conceptual frameworks for cognition.	Carryout Stoop test. Explain Stroop effect
GENERAL OBJECTIVE: 4.0 Understand HCI designs for collaboration and communication						
7-8	4.1 Explain social mechanisms used in communication and collaboration 4.2 Describe Ethnographic issues in collaboration and communication. 4.3 Explain language framework and distributed cognition 4.4 Explain different interface types and describe their interaction processes. 4.5 Explain the specific design	Explain the social mechanisms used in communication and collaboration. Explain conversational mechanisms and collaborative technologies that support them. Explain collaborative technologies to	White Board PC Multimedia Projector User Interface prototyping software (like Pencil, Axure RP, Wireframe, Rapid UI,	Apply social mechanisms used in communication and collaboration. Design a prototype of a collaborative interface Use real life examples to demonstrate interface types and describe their interactions.	Guide students to: Apply social mechanisms used in communication and collaboration. Illustrate different collaborative systems and identify	What are the social mechanisms used in communication and Collaboration? Explain different interface types and describe their interaction processes

	considerations for different interface types.	support coordination Discuss Ethnographic studies of collaboration and communication. Discuss the language / action framework and distributed cognition Explain different interface types(Graphic User Interfaces, Speech based interfaces, Mobile interfaces, Augmented Reality interfaces, Virtual Reality interfaces, Wearable interfaces, etc.) Explain design considerations for	Design Sketch, etc.) White Board PC Multimedia Projector User Interface prototyping software (like Pencil, Axure RP, Wireframe, Rapid UI, Design	Illustrate design considerations for each interface type.	the design goals for each. Design a prototype of a collaborative interface Use real life examples to demonstrate interface types. Demonstrate design considerations for each interface type	
--	---	--	--	--	---	--

		each interface type.				
GENERAL OBJECTIVE: 5.0 Know how interfaces affect users						
10	5.1 Explain affective and expressive interfaces. 5.2 Explain the application of anthropomorphism to interaction design. 5.3 Define virtual characters and agents.	Discuss affective interfaces. Discuss expressive interfaces Explain user frustrations and how to deal with it. Discuss the application of anthropomorphism to interaction design. Explain virtual characters and agents. Discuss different kinds of characters - Synthetic characters	White Board PC Multimedia Projector User Interface prototyping software (like Pencil, Axure RP, Wireframe, Rapid UI, Design Sketch, etc.)	Design an interface prototype that conveys emotion.	Guide students to: Design expressive interfaces that convey emotion.	Explain affective and expressive interfaces

		- Animated agents - Emotional agents - Embodied conversational interface agent. Explain general design concerns - Believability of virtual characters - Appearance - Behavior of interaction				
GENERAL OBJECTIVE: 6.0 : Understand the process of Interaction Design						
11- 12	6.1 Explain the Waterfall model, Iterative model, and Spiral model of Interaction Design. 6.2 Explain Life cycle models in software engineering and HCI 6.3 Explain user testing. 6.4 Explain Formative evaluation, Field Study, and Controlled Experiment	Explain different interaction design models (Waterfall model, Iterative model, Spiral model). Explain the three characteristics of User-centered design: Iterative Design, Focus on	White Board PC Multimedia Projector User Interface prototyping software (like Pencil, Axure RP, Wireframe,	Conduct user and task analysis for a given project. Design a prototype of the above project using User-centered Design principles. Preparing test conditions. Evaluate and test the project.	Guide student to: Conduct user and task analysis for a given project. Come up with a User-centered Design of the above project. Prepare test	Explain the Waterfall model, Iterative model, and Spiral model of Interaction Design. Prepare test conditions, evaluate and test projects.

	Explain domain analysis of users.	Users and Tasks, Constant Evaluation. Explain the process of identifying users and their needs Explain how to generate alternative designs and choose from the designs. Explain the concept of user testing in HCI Explain different kinds of user testing: (Formative evaluation, Field Study, and Controlled Experiment) Explain the basic issues in designing and testing typical tasks	Rapid UI, Design Sketch, etc.) White Board PC Multimedia Projector User Interface prototyping software (like Pencil, Axure RP, Wireframe, Rapid UI, Design Sketch, etc.)		conditions. Evaluate and test the project.	
--	--	---	---	--	--	--

		Explain the basic issues in selecting typical users and their domains. Explain issues in preparing the test conditions.				
GENERAL OBJECTIVE: 7.0 Understand Human-Computer Interaction Design for Emerging Technologies						
13-14	7.1 Explain the concept of emerging technologies. 7.2 Explain Human Computer Interface design for Virtual Reality 7.3 Explain Human Computer Interaction design for Augmented Reality 7.4 Explain human Computer Interaction Design for Mobile Computer Systems	Explain the term 'emerging technologies'. Explain HCI principles and participatory design approaches for emerging technologies. Explain HCI design requirements for Virtual Reality. Explain HCI design requirements for	White Board PC Multimedia Projector User Interface prototyping software (like Pencil, Axure RP, Wireframe, Rapid UI, Design Sketch, etc.)	Apply principles of HCI in the design of a VR application Apply principles of HCI in the design of Augmented Reality application Apply principles of HCI in mobile applications design	Guide students to Design VR applications applying the principles of HCI Design Augmented Reality applications applying the principles of HCI Design mobile applications	Design HCI application for an emerging technology.

		Augmented Reality Explain HCI design requirements for Mobile devices Explain how HCI can be used with SWD, Virtual Reality and Augmented Reality applications to foster creativity and innovation across different domains and fields (such as education, health, art, etcetera).			applying the principles of HCI	
GENERAL OBJECTIVE: 8.0 Understand Ethical and societal implications of Human-Computer Interaction.						
15	Explain the meaning of Ethics in Human-Computer Interaction. Explain HCI Ethics considerations and concepts such as: human welfare, ownership and property, privacy, freedom from bias,	Discuss ethics in HCI. Discuss various ethical considerations in	White Board PC Multimedia Projector	Examine HCI application to outline certain ethical considerations.	Guide students to examine various HCI applications to enumerate various ethical considerations.	What is ethiCs in HCI? Enumerate ethics considerations

	universal usability, trust, autonomy, informed consent, and accountability. Explain the importance/relevanc of ethics in HCI	HCI applications. Discuss the relevance of ethics in HCI	User Interface prototyping software (like Pencil, Axure RP, Wireframe, Rapid UI, Design Sketch, etc.)			for a given HCI application.
--	---	---	---	--	--	------------------------------

PROGRAMME: HIGHER NATIONAL DIPLOMA SOFTWARE AND WEB DEVELOPMENT		
COURSE: Ethical and Professional Practice in SWD	COURSE CODE: SWD422	CONTACT HOURS: 3hrs
	CREDIT UNIT: 2	THEORETICAL: 2
YEAR: II SEMESTER: II	PRE-REQUISITE:	PRACTICAL: 1
GOAL: This course is designed to provide students with knowledge and skills of ethical and professional practices in Software and Web Development.		
GENERAL OBJECTIVES: : Upon completion of this course, the student should be able to: 1. Comprehend the ethics of software and web development 2. Understand the concepts of IT professional practice 3. Understand legal and Ethical issues in IT practice that relates to software and web development 4. Understand the principles of software and web development group work 5. Understand quality management in IT practice 6. Understand job application process, career paths and requirements 7. Know how to access relevant information for IT practice		

PROGRAMME: HIGHERNATIONAL DIPLOMA SOFTWARE AND WEB DEVELOPMENT						
COURSE: Ethical and Professional Practice in SWD			COURSE CODE:SWD422		CONTACT HOURS: 3hrs	
			CREDIT UNIT:2		THEORETICAL: 2	
YEAR: II SEMESTER: II			PRE-REQUISITE:		PRACTICAL:1	
COURSE SPECIFIFCATION: THEORETICAL AND PRACTICAL						
GOAL: This course is designed to provide students with knowledge and skills of ethical and professional practices in Software and Web Development.						
GENERAL OBJECTIVE 1.0: Comprehend the ethics of Software and Web Development						
THEORETICAL CONTENT				PRACTICAL CONTENT		
Week	Specific Learning Outcome	Teacher’s Activities	Resources	Specific Learning Outcome	Teacher’s Activities	Evaluation
1 - 2	1.1 Define ethics 1.2 Explain the concept of ethics in IT profession and professional behaviour 1.3 Explain the principles of ethical software and web development	Describe ethics with examples Discuss the concept of ethics in IT profession, professional behaviour, and ethical issues related to software and web development Discuss the principles of ethical software and web development (e.g., web applications should work	White board, PCs/Laptops, Internet, Multimedia, and flip chart	Debate on ethical issues related to software and web development Analyse the principles of ethical application development	Guide students to: Brainstorm on software and web development ethical issues Analyse the principles of ethical application development	Outline ethical issues related to software and web development Enumerate the principles of ethical software and web development

		everywhere, software or web applications should work for everyone, web applications should respect a user's privacy and security, developers should be considerate of their peers, etc.)				
GENERAL OBJECTIVE: 2.0 Understand the Concepts of IT Professional Practice						
3 - 4	2.1 Explain the concepts of profession and professional practice 2.2 State the basic requirements for IT professional practice 2.3 Enumerates IT professional bodies and their roles 2.4 Outline the importance and process of registering with IT Professional bodies	Discuss the concepts of profession and professional practice as applied in the IT industry Discuss the basic requirements to practice as an IT professional Discuss IT professional bodies and their roles (e.g., Computer Professionals	White board, PCs/Laptops, Internet, Multimedia, and flip chart	Search and compare at least three IT professional bodies in Nigeria and others elsewhere in terms of 1) membership requirements, 2) Operational roles and responsibilities, 3) impact on ethical practice	Guide students to: Participate in groups to compare at least three IT professional bodies in Nigeria and others elsewhere in terms of 1) membership requirements, 2) Operational roles and responsibilities	

		Registration Council of Nigeria - CPN, Nigeria Computer Society - NCS, Free and Open Source Software Foundation for Africa – FOSSFA, etc.) Explain the importance and process of registering with IT Professional bodies			, 3) impact on ethical practice	
GENERAL OBJECTIVE: 3.0 Understand Legal and Ethical Issues in IT Practice that relates to software and web development						
5 - 6	3.1 Explain professional morals, Ethics and Laws 3.2 Explain the code of ethics/bye laws of relevant professional bodies e.g., CPN, NCS, FOSSFA, etc., as relates to software and web development 3.2 Explain the code of ethics/ bye laws of relevant professional bodies e.g., CPN, FOSSFA, NCS, etc., as relates to software and web development 3.2 Explain the code of ethics/bye laws of	Discuss IT professional morals, ethics and laws for best practices Discuss the code of ethics /bye laws of relevant IT professional bodies (e.g., CPN, NCS, FOSSFA, etc.), as it relates to software and web development	White board, PCs/Laptops, Internet, Multimedia, and flip chart	Brainstorm on the pros and cons of being a member of professional bodies as a developer; with strong reference to Nigerian digital space Create a conceptual ethical guide to support software and web development processes	Guide students to: Brainstorm on the pros and cons of being a member of professional bodies as application developer create a conceptual ethical guide that support	

	relevant professional bodies e.g., CPN, NCS, FOSSFA, etc., as it relates to software and web development 3.3 Explain the obligations of IT professionals to fellow professionals, employers, the nation and society 3.4 Explain legal and ethical issues in IT practice that relates to software and web development 3.5 Define the sanctions for violating code of ethics of the IT profession 3.6 Outline the laws relating to copyright, patent, trademark, intellectual property, access right, piracy and plagiarism etc.	Discuss the obligations of IT professionals to fellow professionals, developers, employers, the nation and society Discuss the sanctions for violating code of ethics of the IT profession with special emphasis on software and web development Discuss legal and ethical issues in IT practice that relates to software and web development (e.g., personal privacy, access right, harmful actions, patents, intellectual property, copyright, trade secrets, liability, and piracy)			software and web development processes	
--	---	---	--	--	--	--

		Discuss the laws relating to copyright, patent, trademark, intellectual property, access right, piracy and plagiarism, etc.				
GENERAL OBJECTIVE: 4.0 Understand the principles of Software and Web Development group work						
7 - 8	4.1 Explain the concept of group and team in software and web development 4.2 Outline the needs for group work in software and web development 4.3 Explain the different types of groups: project group, peer group, in-group, out-group, etc. 4.4 Explain group dynamics and the role of members 4.5 Explain the Stages of group formation (e.g., Tuckman's & Jensen's model): forming, storming, norming, performing and adjourning 4.6 Explain the concept of Balanced Team	Discuss the concept of group, team and their importance in software and web development Discuss the needs for group work in software and web development Discuss the types of groups (project group, peer group, in-group, out-group, etc.), group dynamics and roles of members Discuss the Stages of group formation (Tuckman's/Jen	White board, PCs/Laptops, Internet, Multimedia, and flip chart	Participate in groups using Tuckman's & Jensen's model for simple application development tasks Perform simple application development tasks in groups Assess the group performances over a specified period using Belbin's team roles	Guide students to: Participate in groups using Tuckman's & Jensen's model for simple application development tasks Perform simple application development tasks in groups Assess group performanc	Perform simple application development tasks in groups Discuss how to assess the effectiveness of a team.

	4.7 Describe Belbin's Team Roles: plant, resource investigator, shaper, completer finisher etc. 3.8 Explain the process of assessing the effectiveness of a team using Belbin's Team roles	se') Discuss the concept of Balanced Team Explain Belbin's Team Roles Discuss the process of assessing the effectiveness of a team using Belbin's Team roles			es over a specified period using Belbin's team roles	
GENERAL OBJECTIVE: 5.0 Understand quality management in IT practice						
9 - 10	5.1 Explain the quality management in the context of IT products & services 5.2 Outline the need for quality management in IT practice 5.3 Explain the stages of quality Management: define standards and processes, quality assurance, quality improvement 5.4 Explain the process of setting up	Discuss quality management in the context of IT products & services with specific examples in software & web development Discuss the need for quality management in IT practices.	White board, PCs/Laptops, Internet, Multimedia, and flip chart	Produce quality standards for software and web development products or solutions Create quality management principles for a software and web development company	Guide students to: Produce quality standards for software and web development products or solutions Create quality	Outline and discuss stages of quality management and the processes involved in setting up quality standards What are Quality Management

	quality standards in IT organizations 5.5 Explain Quality Management Principles (ISO 9001 or latest): customer focus, leadership, people involvement, process approach etc. 5.6 Explain the process of monitoring and controlling the quality of products and services in IT organizations	Discuss the stages of quality Management (define standards and processes, quality assurance, quality improvement) with relevant examples. Discuss the process of setting up quality standards in IT organizations based on ISO 9001 Discuss the principles of Quality Management Explain the process of monitoring and controlling the quality of products and services in an organization			management principles for a software and web development company	Principles based on ISO 9001?
GENERAL OBJECTIVE: 6.0 : Understand job application process, career paths and requirements						

11 -12	6.1 Explain the IT job market 6.2 Explain the various roles of IT professionals e.g., Software Developer, Database Administrator, Network Engineer etc. 6.3 Outline the requirements for one's suitability for each role 6.4 Explain Career Paths within each role in IT Profession 6.5 Explain the job application Process: advertisement, curriculum vitae (CV) and covering letter, interview and aptitude tests, etc. 6.6 Describe the structure and Content of a good CV: computer and recruiter friendly 6.7 Explain process of restructuring a CV and covering letter to suit job specifications	Discuss IT job marketplace and roles of IT professionals Discuss the requirements for one's suitability for each role Discuss Career Paths within each role in IT Profession Discuss the job application process Discuss the structure and content of a good CV Discuss the restructuring process of a CV and cover letter to suit job specifications	White board, PCs/Laptops, Internet, Multimedia, and flip chart. Samples of job vacancy advertisement ; good and poor curriculum vitae	Search the Internet for software and web development job advertisement Identify the requirements for each advertised job Develop a good curriculum vitae, cover letter for job application, and an acceptance letter for the job	Guide students to: Search the Internet for software and web development job advertisement Identify the requirements for each advertised job Develop a good curriculum vitae, cover letter for job application, and an acceptance letter for the job	What are the requirements for one's suitability for the Application Developer role in the job market or digital economy? Discuss how to develop a good curriculum vitae and covering letter for job application
--------	--	---	---	---	--	---

GENERAL OBJECTIVE: 7.0 : Know how to access relevant information for IT practice						
13 -15	7.1 Explain data, information, knowledge and understanding 7.2 Describe various types of information needed for IT practice: job (contract & employment) advertisement, training, current trends, conferences, workshops etc. 7.3 Explain various sources of data/information: conferences, workshops, website of professional bodies, print media, government publications, virtual libraries etc. 7.4 Describe the various types of Searches: simple, advanced and Meta searches. 7.5 Describe the various search techniques: Boolean logic, parenthesis, phrase searching, truncation, wildcards, etc. 7.6 Explain stages in information search process 7.7 Discuss the need for information validation and verification 7.8 Discuss the process of information validation and	Discuss the terms data, information, knowledge and understanding in the context of IT practice Discuss various types of information needed for IT practice Discuss relevant sources of data/information Discuss types of search techniques. Discuss the stages in information search process (initiation, selection, exploration, formulation, collection, search closure)		Search the Internet for relevant requests for expression of interest (EoI) or tender bid for software and web development contracts; and outline the company's requirements for each	Guide students to: Search the Internet for relevant requests for expression of interest (EoI) or tender bid for software and web development contracts; and outline the company's requirements for each	What are the types of information and sources for IT practice? Demonstrate how to search, and retrieve relevant information using appropriate search tools

	verification	Explain the needs for information validation and verification. Discuss the process of information validation and verification				
--	--------------	---	--	--	--	--

PROGRAMME: HIGHER NATIONAL DIPLOMA SOFTWARE AND WEB DEVELOPMENT		
COURSE: Software Testing and Quality Assurance	COURSE CODE: SWD423	CONTACT HOURS: 4hrs
	CREDIT UNIT: 3	THEORETICAL: 2
YEAR: II SEMESTER: II	PRE-REQUISITE:	PRACTICAL: 2
GOAL: This course is designed to provide the students with knowledge and Skills on Software Testing and Quality Assurance		
GENERAL OBJECTIVES: On completion of this course the student should be able to: 1.0 Understand the Basic Concepts of Software Testing 2.0 Know the Concepts of Software Quality Assurance 3.0 Understand Stages in Software Testing Life Cycle 4.0 Understand Test Case Execution and Defects Logging 5.0 Know Automation Testing Process 6.0 Understand Test Automation Frameworks		

PROGRAMME: HIGHERNATIONAL DIPLOMA SOFTWARE AND WEB DEVELOPMENT						
COURSE: Software Testing and Quality Assurance		COURSE CODE: SWD423		CONTACT HOURS: 4hrs		
		CREDIT UNIT: 3		THEORETICAL: 2		
YEAR: II SEMESTER: II		PRE-REQUISITE:		PRACTICAL: 2		
COURSE SPECIFIFCATION: THEORETICAL AND PRACTICAL						
GOAL: This course is designed to provide the students with knowledge and Skills on Software Testing and Quality Assurance						
GENERAL OBJECTIVE 1.0: Understand the Basic Concepts of Software Testing						
THEORETICAL CONTENT				PRACTICAL CONTENT		
Week	Specific Learning Outcome	Teacher’s Activities	Resources	Specific Learning Outcome	Teacher’s Activities	Evaluation
1 -3	1.6 Define Software Testing	Explain Software Testing	White Board.	Create sample programs	Guide students to:	Explain Software
	1.7 Outline the Importance of Software Testing	Explain the Importance of Software Testing	PC and multimedia Projector	Identify errors, defects and faults in the sample programs	Create sample programs	Errors, Defects, Bugs, failure and faults with examples
	1.8 Differentiate between Manual Testing and Automation Testing		Online lecture notes	Identify errors, defects and faults in the sample programs	Explain the Importance of Software Testing	
	1.9 Define Software Error		Program Code Samples in any		Explain the Principles of	
	1.10 Explain Software Defects, Bugs,					

	failure and faults with examples 1.11 Explain the Principles of Software Testing 1.12 Explain the Types of Software Testing	Discuss Software Errors, Defects, Bugs, failure and faults with examples Discuss the Principles of Software Testing Discuss the Types of Software Testing (Acceptance, Functional, Unit Testing)	of the following: Java, C++, Python, PHP, Javascript etc.			Software Testing Explain the Types of Software Testing
GENERAL OBJECTIVE: 2.0 Know the Concepts of Software Quality Assurance						
4-5	2.1 Define Software Quality Assurance 2.2 Differentiate between Software Quality Assurance and Software Quality Control 2.3 Explain McCall's Quality Factor Model	Explain Software Quality Assurance Distinguish between Software Quality Assurance and Software Quality Control Discuss McCall's	White Board and marker. PC and multimedia Projector Online lecture notes	Develop Software Quality Assurance Plan	Guide students to: Develop a Software Quality Assurance Plan	Explain McCall's Quality Factor Model Explain Software Quality Assurance Plan (SQAP) Differentiate between Software

	2.4 Explain Software Quality Assurance Plan (SQAP) 2.5 Explain Steps in creating Software Quality Assurance Plan (SQAP)	Quality Factor Model(product operational factors, Revision factors and Transition factors) Discuss Steps in creating Software Quality Assurance Plan (SQAP)				Quality Assurance and Software Quality Control
GENERAL OBJECTIVE: 3.0 Understand the Stages in Software Testing Life Cycle						
6-7	3.1 Define Software Testing Life Cycle 3.2 Explain the stages in Software Testing Life Cycle 3.3 Explain Test Plan and its Importance 3.4 Explain Test Cases and their usefulness 3.5 Explain Continuous Testing and its importance	Explain Software Testing Life Cycle Discuss the stages in Software Testing Life Cycle(requirements analysis, test plan creation, environment setup etc). Discuss Test Plan and its Importance. Discuss Test Cases and their usefulness	White Board and marker. PC and multimedia Projector Online lecture notes Software Testing tools(Selenium, Apache JMeter, etc) Sample Software	Install and configure the Test Environment Create a Test plan for testing Sample Software Develop Test cases required to test the Application	Guide students to: Install and configure the Test Environment Create a Test plan for testing an Application to be created Develop test cases required to test the Application	Explain the stages in Software Testing Life Cycle Explain Test Plan, Test Cases and Continuous Testing

		Discuss Continuous Testing and its importance				
GENERAL OBJECTIVE: 4.0 Understand Test Case Execution and Defects Logging						
8-9	4.1 Define Test Case and Edge test case. 4.2 Explain Test Case Execution its activities 4.3 Explain Requirement Traceability Matrix and its Importance 4.4 Explain Defect logging and its importance	Explain Test Case Explain Edge test cases. Discuss Test Case Execution and its activities (creation of test cases, execution of test cases, mapping of test cases according to requirements). Discuss Defects logging and its Importance Discuss requirement Traceability Matrix and its importance	White Board and marker. PC and multimedia Projector Software Testing tools Sample source code for the Test	Demonstrate Test Case execution and mapping according to requirements Demonstrate Defects logging	Guide students to: Demonstrate Test Case execution and mapping according to requirements Demonstrate Defects logging	
GENERAL OBJECTIVE: 5.0 Know Automation Testing Process						

10 -12	5.1 Explain Automated Testing	Discuss Automated Testing	White Board and marker.	Develop test scripts to test sample programs	Guide students to:	Explain
	5.2 Explain Automated Testing Tools and give examples	Explain Automated Testing Tools and give examples – Selenium, Ranorex etc.	PC and multimedia Projector	Demonstrate Unit Testing, Integration Testing, Smoke Testing, Performance Testing, Regression Testing	Write test scripts to test sample programs	Advantages and disadvantages of Automated Testing
	5.3 Explain Advantages and disadvantages of Automated Testing		Software Automation Testing tools	API Testing	Demonstrate Unit Testing, Integration Testing, Smoke Testing, Performance Testing, Regression Testing	Explain Types of Automated Testing
	5.4 Explain Types of Automated Testing	Discuss Advantages and disadvantages of Automated Testing	Sample source code for the Test	User Interface Testing		Explain the Steps in Automated Testing Process
	5.5 Explain the Steps in Automated Testing Process	Discuss Types of Automated Testing(functional and non-functional testing) Discuss the steps in Automated Testing Process (selecting testing tool, define the scope, write the test script, execute the test etc).				

GENERAL OBJECTIVE: 6.0 Understand Test Automation Frameworks

13-15	6.1 Explain Test Automation Frameworks and its usefulness 6.2 Explain Types of Test Automation Frameworks 6.3 Explain Continuous Testing	Discuss Test Automation Frameworks Discuss Types of Test Automation Frameworks (linear, modular, data driven automation framework etc). Discuss Continuous Testing	White Board and marker. PC and multimedia Projector Online lecture materials Sample source code for the Test	Demonstrate Unit Testing, Integration Testing, Smoke Testing, Performance Testing, Regression Testing API Testing User Interface Testing using Test Automation Frameworks	Guide Students to: Demonstrate Unit Testing, Integration Testing, Smoke Testing, Performance Testing, Regression Testing using Test Automation Frameworks	
-------	--	--	---	---	--	--

Assessment: Give details of assignments to be used:

Coursework/ Assignments %; Course test %; Practical %; Projects %; Examination %

Type of Assessment	Purpose and Nature of Assessment (SWD423)	Weighting (%)
Examination	Final Examination (written) to assess knowledge and understanding	40
Test	At least 2 progress tests for feedback.	10
Practical	At least 5 hands-on to be assessed by the teacher	50
Total		100

PROGRAMME: HIGHER NATIONAL DIPLOMA SOFTWARE AND WEB DEVELOPMENT		
COURSE: Security in Software Development	COURSE CODE: SWD424	CONTACT HOURS: 4hrs
	CREDIT UNIT: 1	THEORETICAL: 2
YEAR: II SEMESTER: II	PRE-REQUISITE:	PRACTICAL: 2
GOAL: : This course is designed to provide the students with knowledge and Skills of Security in Software and Web Development		
GENERAL OBJECTIVES: : On completion of the course, the student should be able to: 1. Understand security fundamentals, threats and vulnerabilities 2. Understand security principles 3. Understand secure development lifecycle (sDLC) 4. Know secure coding practices 5. Understand authentication, authorization, encryption and data protection 6. Understand secure communication, security testing and incident response 7. Understand compliance, legal considerations, ethical and responsible development 8. Understand secure software design, security awareness, secure deployment and maintenance		

PROGRAMME: HIGHERNATIONAL DIPLOMA SOFTWARE AND WEB DEVELOPMENT						
COURSE: Security in SWD			COURSE CODE:SWD424		CONTACT HOURS: 4hrs	
			CREDIT UNIT:1		THEORETICAL: 2	
YEAR: II SEMESTER: II			PRE-REQUISITE:		PRACTICAL:2	
COURSE SPECIFIFCATION: THEORETICAL AND PRACTICAL						
GOAL: : This course is designed to provide the students with knowledge and Skills of Security in Software and Web Development						
GENERAL OBJECTIVE 1.0: Understand security fundamentals, threats and vulnerabilities						
THEORETICAL CONTENT				PRACTICAL CONTENT		
Week	Specific Learning Outcome	Teacher’s Activities	Resources	Specific Learning Outcome	Teacher’s Activities	Evaluation
1-2	Explain CIA triad (Confidentiality, Integrity, Availability) and their significance in software security. Explain common security threats and vulnerabilities in software systems. Explain how to analyze and classify security risks based on their impact and likelihood.	Discuss the CIA triad, defining each component (Confidentiality, Integrity, Availability) and their significance in software security. Discuss common security threats and vulnerabilities in software systems. Discuss scenarios where maintaining confidentiality, ensuring integrity or	PowerPoint and similar presentation packages, Overhead Project, White Board, Access to OWASP (Open Web Application Security Project) for news on security.	Identify CIA triad (Confidentiality, Integrity, Availability) and their significance in software security. Identifysecurity threats and vulnerabilities in software systems. Analyze security risks based on their impact and likelihood.	Guide Students to: Demonstrate real-world examples and case studies where the CIA triad is relevant. Analyze scenarios where maintaining confidentiality, ensuring integrity, or availability is	

		availability is crucial. Discuss how breaches in confidentiality, integrity or availability can impact software systems and organizations. Discuss common security threats (e.g., malware, social engineering, DDoS attacks) and vulnerabilities (e.g., SQL injection, XSS) in software systems. Explain each threat and vulnerability, and how they can be exploited and their potential impact.			crucial and discuss potential security measures. Analyze security threats and vulnerabilities in given software systems or scenarios.	
GENERAL OBJECTIVE: 2.0 Understand security principles						
3-4	Define the principle of least privilege and how it limits access to resources. Explain the concept of defense in depth and	Explain the principle of least privilege and its significance in minimizing security risks by restricting		Identify the principle of least privilege and how it limits access to resources. Demonstrate the concept	Guide students to: Critique the security architecture of	

	its role in layered security. Explain the importance of secure defaults and secure by design principles in software security.	user and system access to only what is necessary. Discuss the concept of defense in depth and its role in layered security. Discuss the importance of secure defaults and secure by design principles in software security. Explain how starting with secure defaults and designing with security in mind can prevent vulnerabilities.		of defense in depth and its role in layered security. Demonstrate the importance of secure defaults and secure by design principles in software security.	a given system, making recommendations to enhance defense in depth. Secure design patterns and best practices, emphasizing how they contribute to secure by design principles. Demonstrate the concept of defense in depth, using visual aids to illustrate how multiple layers of security can protect against different types of threats.	
--	---	--	--	---	--	--

					Apply secure by design principles to create or improve a software system, considering security from the outset. Conduct code reviews or analyses of existing software to identify areas where secure defaults and secure by design principles were implemented effectively or overlooked.	
GENERAL OBJECTIVE: 3.0 Understand secure development lifecycle (sDLC)						
5-6	List the stages of the sDLC.	Discuss the stages of	Integrated Development	Identify the stages of the	Guide students to:	

	Explain security considerations and practices and its integration into each stage of the sDLC. Explain the benefits of incorporating security early in the development process to reduce vulnerabilities and associated costs.	the sDLC. Discuss the purpose, sequence, planning, design, development, testing, deployment, and maintenance in sDLC. Explain using visual aids (like flowcharts or diagrams) the typical phases and their interrelationships within the sDLC. Explain the importance of integration of security testing (e.g., code review, penetration testing) into the testing phase	Environments (IDEs) like Visual Studio Code or JetBrains Jira, Trello, or Microsoft Azure DevOps can help students understand how sDLC is integrated into project management.	sDLC. Identify security considerations and practices and its integration into each stage of the sDLC. Identify the benefits of incorporating security early in the development process to reduce vulnerabilities and associated costs.	Create a simplified sDLC plan for a hypothetical software project, identifying key activities at each stage Gather security requirements and perform threat modeling during the planning and design phases. Identify where security considerations and practices should be integrated into each stage of the sDLC.	
--	--	--	--	---	---	--

		of the sDLC. Discuss the benefits of incorporating security early in the development process to reduce vulnerabilities and associated costs.			Analyze and improve the security of code at the development stage. Demonstrate case studies and statistics that illustrate the higher cost and impact of addressing security issues in later stages of the sDLC compared to addressing them early.	
GENERAL OBJECTIVE: 4.0 Understand secure coding practices						
7-8	Explain secure coding practices and how to	Explain the importance of secure	Static analysis tools eg. SonarQube or	Demonstrate secure coding practices and how to	Guide students	

	mitigate common security risks Explain common coding vulnerabilities, Explain how to write secure code that adheres to industry best practices and coding standards.	coding practices and the role they play in mitigating security risks. Discuss common security risks, such as input validation, output encoding, and error handling. Explain how to identify and avoid common coding vulnerabilities, including SQL injection, XSS, CSRF, and buffer overflows. Explain how to write secure code that adheres to industry best practices and coding standards. Discuss coding standards relevant to secure coding, such as OWASP Secure Coding Guidelines or	ESLint to analyze code for security vulnerabilities. Secure Code Review Tools eg. CodeClimate or Checkmarx	mitigate common security risks Demonstrate common coding vulnerabilities, Demonstrate how to write secure code that adheres to industry best practices and coding standards.	to: Apply secure coding practices to mitigate common security risks, such as input validation, output encoding, and error handling. Perform threat modeling exercises to identify potential security risks in code and suggest mitigations. Identify and avoid common coding vulnerabilities, including SQL injection, XSS, CSRF, and buffer overflows.	
--	---	--	--	---	---	--

		industry-specific standards.			Write a secure code that adhere to industry best practices and coding standards. Conduct code review sessions where students assess code against coding standards, focusing on security-related guidelines.	
GENERAL OBJECTIVE: 5.0 Understand authentication, authorization, encryption and data protection						
9-10	Explain the differences between authentication and authorization. Explain encryption techniques and their applications in securing data. Explain methods for securely storing and handling sensitive data.	Explain authentication and authorization. Explain the differences between authentication and authorization and describe their roles in access control. Discuss common	PC, Projector, Whiteboard, Marker, Relevant Textbook; Passport.js (for Node.js) or Spring Security (for Java) OpenSSL or Keyczar	Demonstrate the differences between authentication and authorization. Demonstrate encryption techniques and their applications in securing data. Demonstrate the methods for securely storing and	Guide student to: Analyze the differences between authentication and authorization, emphasizing their distinct roles in security.	

		traditional access control models(such as discretionary access control (DAC) and mandatory access control (MAC)) and dynamic access control models(like Risk based access control, Adaptive access control) to illustrate the concepts. Explain encryption techniques, including symmetric, asymmetric and hybrid cryptography, and their applications in securing data. Discuss methods for securely storing and handling sensitive data, such as password hashing and encryption.		handling sensitive data. Implement encryption and decryption algorithms in real-world applications to secure data. Demonstrate a practical case study of data breaches, emphasizing how encryption could have prevented or mitigated the impact. Demonstrate encryption tools and libraries in popular programming languages, showing how they can be integrated into software development. Design secure data storage and handling solutions for different scenarios. Analyze the security of data storage and handling practices by acting as ethical hackers.	Conduct hands-on sessions to introduce encryption techniques, including symmetric, asymmetric, and hybrid cryptography, encryption algorithms, and key management. Implement encryption and decryption algorithms in real-world applications to secure data. Demonstrate a practical case study of data breaches, emphasizing how	
--	--	--	--	--	--	--

					encryption could have prevented or mitigated the impact. Demonstrate encryption tools and libraries in popular programming languages, showing how they can be integrated into software development. Design secure data storage and handling solutions for different scenarios. Assess the security of data storage and handling practices by	
--	--	--	--	--	--	--

					acting as ethical hackers.	
GENERAL OBJECTIVE: 6.0 Understand secure communication, security testing and incident response						
11-12	Explain secure communication protocols (e.g., HTTPS) and their role in protecting data in transit. Explain security testing, including penetration testing and vulnerability scanning, to identify and remediate security weaknesses. Explain the components of an incident response plan and the steps to take in response to a security incident.	Explain secure communication protocols, such as HTTPS, SSL/TLS, and their significance in protecting data during transmission. Explain security testing, including penetration testing and vulnerability scanning, to identify and remedy security weaknesses. Explain the components of an incident response plan and the steps to take in response to a security incident.	PC, Projector, Whiteboard, Marker, Relevant Textbook; Splunk or Security Information and Event Management (SIEM) systems	Demonstrate secure communication protocols (e.g., HTTPS) and their role in protecting data in transit. Conduct security testing, including penetration testing and vulnerability scanning, to identify and remediate security weaknesses. Conduct incident response plan and the steps to take in response to a security incident.	Guide students to: Conduct security testing, including penetration testing and vulnerability scanning, to identify and remediate security weaknesses. Demonstrate the use of secure communication protocols in action, showcasing how data is encrypted and secured.	

					Implement secure communication protocols in web applications or network setups. Conduct incident response planning, detailing the components of an incident response plan, including roles, responsibilities, and procedures.	
GENERAL OBJECTIVE: 7.0 Understand compliance, legal considerations, ethical and responsible development						
13-14	Explain the legal and regulatory requirements relevant to software security. Explain the ethical considerations in software development.	Explain the various legal frameworks and regulations relevant to software security, including international laws	PC, Projector, Whiteboard, Marker, Relevant Textbook; KnowBe4 or SANS Security	Conduct presentation on the legal and regulatory requirements relevant to software security.	Guide students to: Explore specific legal requirements and regulations,	

	Explain the consequences of non-compliance and unethical behavior in software development.	like GDPR, HIPAA, NDPR and industry-specific standards. Discuss ethical considerations in software development, including responsible handling of user data and respecting user privacy. Explain the potential legal consequences of non-compliance with regulations and unethical behavior in software development, emphasizing fines, legal actions, and reputational damage.	Awareness; Lucidchart or draw.io; Terraform or Ansible	Identify ethical considerations in software development. Identify the consequences of non-compliance and unethical behavior in software development.	of sample project and analyze their implications for software development, and present their findings. Conduct hypothetical ethical dilemmas and guide them to make and defend ethical decisions based on their understanding of software ethics. Conduct hypothetical scenarios involving ethical breaches and guide them to identify appropriate	
--	---	--	---	--	---	--

					reporting mechanisms and actions, considering whistleblower protection and legal implications	
GENERAL OBJECTIVE: 8.0 Understand secure software design, security awareness, secure deployment and maintenance						
15	Explain the principles of secure software design, including threat modelling, to create software architectures that consider security from the outset. Explain the need for security awareness among development teams to recognize and respond to security risks. Explain secure deployment practices.	Explain secure software design principles, including threat modeling, Discuss how to apply principles of secure software design (including threat modelling, to create software architectures) that consider security from the outset. Discuss common security risks, vulnerabilities, and best practices.	PC, Projector, Whiteboard, Marker, Relevant Textbook; KnowBe4 or SANS Security Awareness; Lucidchart or draw.io; Terraform or Ansible	Identify the principles of secure software design, including threat modelling, to create software architectures that consider security from the onset. Identify potential threats and vulnerabilities in software designs and propose mitigations. Conduct incident response simulations.	Guide students to: Apply secure software design principles, including threat modeling to create secure software architectures. Identify potential threats and vulnerabilities in software designs and propose mitigations.	

	Explain Infrastructure as Code (IaC) and how it can be used to automate and secure deployment environments. Explain the concept of DevSecOps.	Explain secure deployment practices. (containerization, continuous integration/continuous deployment (CI/CD), patch management, monitoring, continuous security) Discuss Infrastructure as Code (IaC) and how it can be used to automate and secure deployment environments. Explain the concept of DevSecOps and how it integrates security into the deployment pipeline, emphasizing collaboration			Conduct incident response simulations in response to security incidents	
--	---	---	--	--	--	--

		between development, security, and operations teams.				
--	--	--	--	--	--	--

LISTOF EQUIPMENT FOR SOFTWARE AND WEB DEVELOPMENT LABORATORY

A. SOFTWARE LABORATORY

S/N	DESCRIPTION OF ITEM	NO REQUIRED
1.	Computer Systems (at least core i3 8G RAM, 2.0GHz)	40 all networked
2.	Server (HP proliant, DL 160, Gen 8, RAID config., 16 GB RAM, 2 TB, a minimum of two network port) KVM Monitor	2 1
3.	Printers	2
4.	Multimedia Projector	1
5.	Generator 5 KVA and 5KVA Solar Inverter	
6.	Software i) Operating system (Windows, Linux, FreeBSD, Kali-Linux) ii) Python (version 3.6 and above) iii) C++ iii) PHP version 8.2 and above v) Text Editors (eg ATOM, VS code, VIM Sublime text, Jupyter Security Onion vi) JAVAScript ES6 and above viii) ArgoUML ix) Magic Draw x) Codelobster	Installer

	xi) JAVA (JDK) xii) Crimson Editor xv) MySQL server and workbench, xvi) MongoDB xvii) Postman xviii) Git vi) Visio, DIA xviii) Rapid UI xix)Open VAS, Wireshark, nmap, nessus, acunetix xx) XAMPP xxi) Virtualization software xxii) Selenium xxiii) Ronorex xxiv) Apache jMeter xxv) SPSS, R, NVIVO xxvi) Matlab, Weka, Tanagra	
--	--	--

B. EMBEDDED SYSTEMS AND NETWORK LABORATORY

S/N	DESCRIPTION OF ITEM	NO REQUIRED
1.	Digital Multimeter	40
2.	Set of Screw Drivers	40
3.	Soldering iron	40
4.	Oscilloscope	2
5.	Development boards: i) Raspberry Pi ii) Arduino board iii) Pyboard iv) ESP8266-based boards v) ESP32 Wifi + Bluetooth	10 10 10 10 10
6.	Vero/Bread Board	10
7.	Jumper Cables	lots
8.	Logic Probe	10
9.	Cleaning Kit	10
10.	Lead sucker	40
11.	Network tool kits: Crimp tools Punch down RJ45	10 10 lots

	Ethernet cable (Cat6 utp)	lots
12.	Network Devices: Router Switch (16 ports managed) Access point Patch Panel (24 ports cat6 utp) 42U Rack (Telecommunication Cabinet) Cable Labeler	10 10 10 10 1 5
13.	Software and Network Simulators Operating system (Windows, Linux, Unix) Python (version 3.6 and above) C++ Proteus GNS3, eNSP, CISCO Packet Tracer	
14.	Blower	2
15.	Circuit Magnifier	40
16.	Discrete components (diode, capacitors, resistors, LEDs, DHT, sound and water sensors, keypads and screen)	Assorted
17.	Computer Systems (at least core i5 16G RAM, 2.5GHz)	5
18.	Multimedia and accessories: Projector Pointer Projector screen	1 each

LIST OF PARTICIPANTS FOR PRE CRITIQUE SOFTWARE AND WEB DEVELOPMENT

S/N	NAME	ADDRESS	E-MAIL
1.	Dr. Bassey A. Ekanem	Novena University Ogume(CPNREP).	Basseykanem99@gmail.com
2.	Dr. Joy Ezeife	Federal Polytechnic Oko	ezeifejoy@gmail.com
3.	Joel Gogwim	University of Jos	gogwim@unijos.edu.ng
4.	AbdulsalamMuhsin Ahmad	Kaduna Polytechnic,	Muhsin661@gmail.com
5.	Dr. Hassan F. Akande	Kaduna Polytechnic	Hassan.akande@kadunapolytechnic.edu.ng
6	Hafsatu Mora	Kaduna Polytechnic	hafsatumora@gmail.com
8.	Prof. Idris M. Bugaje	NBTE, Kaduna	es@nbte.gov.ng
9	Ibrahim Bashir Bello	NBTE, Kaduna	
10.	Dr. Musa Hatim Koko	NBTE, Kaduna	hatimlion@gmail.com
11	BalaDanladiAkut	NBTE, Kaduna	baladanladi2000@yahoo.ca
12	Aliyu I. Hassan	NBTE, Kaduna	
13	Muhammad Bilyaminu Musa	NBTE Kaduna	mohdganybm@gmail.com
14	Mansur Abdullahi	NBTE, Kaduna	Mansurabdullhi33@gmail.com
15.	Muhammad Umar Auna	NBTE, Kaduna	muhammadauna.mu@gmail.com
16	Grace V. Adejumo	NBTE, Kaduna	gracevictor699@gmail.com
17	AbidaAbbati Idris	NBTE, Kaduna	abidabbatidris@gmail.com

LIST OF PARTICIPANTS FOR FINAL CRITIQUE SOFTWARE AND WEB DEVELOPMENT

S/N	NAME	ADDRESS	E-MAIL ADDRESS
1.	Ohwo Onome Blaise	CPN Representative	blaise.ohwo@gmail.com
2.	Dr. Samaila Musa	Abdu Gusau Poly. Talata Mafara Zamfare State	samailaagp@gmail.com
3.	Joel Gogwim Godwin	Cisco Representative	gogwim@gmail.com
4.	Abdulsalam Muhsin Ahmad	Kaduna Polytechnic	muhsin661@gmail.com
5.	Prof. Idris M.Bugaje	Executive Secretary, NBTE.	es@nbte.com
6.	Dr. Ibrahim Bashir Bello	NBTE, Kaduna	
7.	Dr. Musa Hatim Koko	NBTE, Kaduna	hatinilion@gmail.com
8.	Bala Danladi Akut	NBTE, Kaduna	baladanladi2000@yahoo.ca
9.	Miri Ebipade M.P	NBTE, Kaduna	miri.ebipade@gmail.com
10.	Muhammad Bilyaminu Musa	NBTE, Kaduna	muhammadbilyaminu@nbte.gov .ng
11.	Maryam Yusha'u Abubakar	NBTE, Kaduna	myuabubakar@gmail.com
12.	Hassana Abdullahi	NBTE, Kaduna	
13.	Aminu Salihu Bello	NBTE, Kaduna	aminubellosahihu45@gmail.com